norme européenne

NF EN 50128
Octobre 2011

norme française

Indice de classement : **F 74-128**

ICS : 35.240.60; 45.020; 93.100

Applications ferroviaires
Systèmes de signalisation, de télécommunication et de traitement -
Logiciels pour systèmes de commande et de protection ferroviaire

E : Railway applications -
Communication, signalling and processing systems -
Software for railway control and protection systems

D : Bahnanwendungen -
Telekommunikationstechnik, Signaltechnik und Datenverarbeitungssysteme -
Software für Eisenbahnsteuerungs- und Überwachungssysteme

Norme française homologuée

par décision du Directeur Général d'AFNOR le 14 septembre 2011, pour prendre effet à compter du 14 octobre 2011.

Remplace la norme homologuée NF EN 50128, de juillet 2001 qui reste en vigueur jusqu'en avril 2014.

Correspondance

La norme européenne EN 50128:2011 a le statut d'une norme française.

Analyse

Le présent document spécifie les exigences de processus et les techniques applicables au développement de logiciel pour des systèmes électroniques programmables utilisés dans les applications ferroviaires de contrôle-commande et de protection.

Le présent document doit être utilisé conjointement avec la NF EN 50126-1:2000 et la NF EN 50129:2003

Descripteurs

Transport par fer, matériel de chemin de fer, informatique, logiciel, sécurité, qualité, spécification.

Modifications

Par rapport au document remplacé, publication d'une nouvelle édition.

Corrections

Editée et diffusée par l'Union Technique de l'Electricité (UTE) – Tour Chantecoq – 5, rue Chantecoq – 92808 Puteaux Cedex
Tél. : + 33 (0) 1 49 07 62 00 – Télécopie : + 33 (0) 1 47 78 73 51 – Courriel : ute@ute.asso.fr – Internet : http://www.ute-fr.com/
Diffusée également par l'Association Française de Normalisation (AFNOR) – 11, rue Francis de Pressensé –93571 La Plaine Saint-Denis Cedex – Tél. : +33 (0) 1 41 62 80 00

Impr. UTE
© UTE 2011 – Reproduction interdite

AVANT-PROPOS NATIONAL

Ce document constitue la version française complète de la norme européenne EN 50128:2011.

L'Union Technique de l'Électricité a voté favorablement au CENELEC sur le projet de EN 50128, le 14 mars 2011.

———————

Correspondance entre les documents internationaux cités en référence et les documents CENELEC et/ou français à appliquer

Document international cité en référence	**Document correspondant**	
	CENELEC (EN ou HD)	**français** (NF ou UTE)
ISO 9000 (2005)	EN ISO 9000	NF EN 9000 (X 50-130)
ISO 9001 (2008)	EN ISO 9001	NF EN 9001 (X 50-131)
-	EN 50126-1 (1999)	NF EN 50126-1 (2000) (F 00-126-1)
-	EN 50129 (2003)	NF EN 50129 (2003) (F 74-129)

Note : Les documents de la classe C sont en vente à l'Union Technique de l'Électricité – Tour Chantecoq – 5, rue Chantecoq – 92808 Puteaux Cedex – Tél. : + 33 (0) 1 49 07 62 00 ainsi qu'au service diffusion de l'Association Française de Normalisation – 11, rue Francis de Pressensé – 93571 La Plaine Saint-Denis Cedex – Tél. : +33 (0) 1 41 62 80 00.

Les documents CEI sont en vente à l'UTE.

Les documents ISO (et de la classe X) sont en vente à l'AFNOR.

———————

NORME EUROPÉENNE
EUROPÄISCHE NORM
EUROPEAN STANDARD

EN 50128

Juin 2011

ICS 35.240.60; 45.020; 93.100

Remplace EN 50128:2001

Version française

Applications ferroviaires - Systèmes de signalisation, de télécommunication et de traitement - Logiciels pour systèmes de commande et de protection ferroviaire

Bahnanwendungen - Telekommunikationstechnik, Signaltechnik und Datenverarbeitungssysteme - Software für Eisenbahnsteuerungs- und Überwachungssysteme

Railway applications - Communication, signalling and processing systems - Software for railway control and protection systems

UTE

La présente Norme Européenne a été adoptée par le CENELEC le 2011-04-25. Les membres du CENELEC sont tenus de se soumettre au Règlement Intérieur du CEN/CENELEC qui définit les conditions dans lesquelles doit être attribué, sans modification, le statut de norme nationale à la Norme Européenne.

Les listes mises à jour et les références bibliographiques relatives à ces normes nationales peuvent être obtenues auprès du Secrétariat Central ou auprès des membres du CENELEC.

La présente Norme Européenne existe en trois versions officielles (allemand, anglais, français). Une version dans une autre langue faite par traduction sous la responsabilité d'un membre du CENELEC dans sa langue nationale, et notifiée au Secrétariat Central, a le même statut que les versions officielles.

Les membres du CENELEC sont les comités électrotechniques nationaux des pays suivants: Allemagne, Autriche, Belgique, Bulgarie, Chypre, Croatie, Danemark, Espagne, Estonie, Finlande, France, Grèce, Hongrie, Irlande, Islande, Italie, Lettonie, Lituanie, Luxembourg, Malte, Norvège, Pays-Bas, Pologne, Portugal, République Tchèque, Roumanie, Royaume-Uni, Slovaquie, Slovénie, Suède et Suisse.

CENELEC

Comité Européen de Normalisation Electrotechnique
Europäisches Komitee für Elektrotechnische Normung
European Committee for Electrotechnical Standardization

Management Centre: Avenue Marnix 17, B - 1000 Bruxelles

© 2011 CENELEC - Tous droits d'exploitation sous quelque forme et de quelque manière que ce soit réservés dans le monde entier aux membres du CENELEC.

Ref. n° EN 50128:2011 F

Avant-propos

La présente Norme Européenne a été préparée par le SC 9XA, Systèmes de signalisation, de télécommunications et de traitement, du comité technique CENELEC TC 9X, Applications électriques et électroniques dans le domaine ferroviaire.

Le texte du projet a été soumis au vote formel et a été approuvé par le CENELEC comme EN 50128 le 2011-04-25.

Ce document remplace l'EN 50128:2001.

Les principales modifications par rapport à l'EN 50128:2001 sont énumérées ci-après :

- des exigences relatives à la gestion et à l'organisation, à la définition des rôles et des compétences, au déploiement et à la maintenance des logiciels ont été ajoutées;
- un nouvel article concernant les outils a été ajouté, fondé sur l'EN 61508-2:2010;
- les Tableaux dans l'Annexe A ont été mis à jour.

L'attention du lecteur est attiré sur la possibilité que certains éléments de ce document peuvent être couverts par des brevets. Le CEN et le CENELEC ne sauraient être tenus pour responsable de l'identification de tels brevets.

Les dates suivantes ont été fixées:

– date limite à laquelle l'EN doit être mise en application au niveau national par publication d'une norme nationale identique ou par entérinement	(dop)	2012-04-25
– date limite à laquelle les normes nationales conflictuelles doivent être annulées	(dow)	2014-04-25

Il convient de lire la présente Norme Européenne conjointement à l'EN 50126-1:1999 «Applications ferroviaires - Spécification et démonstration de la fiabilité, de la disponibilité, de la maintenabilité et de la sécurité (FDMS) - Partie *1* : *Exigences de base et procédés génériques*» et à l'EN 50129:2003 «*Applications ferroviaires - Systèmes de signalisation, de télécommunications et de traitement - Systèmes électroniques de sécurité pour la signalisation*».

_____

Sommaire

Avant-propos 2
Introduction 7
1 Domaine d'application 10
2 Références normatives 11
3 Termes, définitions et abréviations 11
3.1 Termes et définitions 11
3.2 Abréviations 15
4 Objectifs, conformité et niveaux d'intégrité de sécurité du logiciel 16
5 Organisation et gestion du développement logiciel 17
5.1 Organisation, rôles et responsabilités 17
5.2 Compétence du personnel 22
5.3 Questions relatives au cycle de vie et à la documentation 22
6 Assurance du logiciel 25
6.1 Test du logiciel 25
6.2 Vérification du logiciel 27
6.3 Validation du logiciel 29
6.4 Évaluation du logiciel 30
6.5 Assurance Qualité du Logiciel 32
6.6 Contrôle des modifications et des évolutions 35
6.7 Outils et langages 36
7 Développement de logiciel générique 40
7.1 Cycle de vie et documentation pour logiciel générique 40
7.2 Exigences relatives au logiciel 40
7.3 Architecture et Conception 43
7.4 Conception du Composant 50
7.5 Réalisation et Test des composants 52
7.6 Intégration 53
7.7 Tests d'Ensemble du Logiciel / Validation Finale 55
8 Développement de données d'application ou d'algorithmes d'application : systèmes configurés par des données d'application ou par des algorithmes d'application 58
8.1 Objectifs 58
8.2 Documents en entrée 58

8.3 Documents en sortie 59
8.4 Exigences 59
9 Déploiement et maintenance du logiciel 64
9.1 Déploiement du logiciel 64
9.2 Maintenance du logiciel 66
Annexe A (normative) Critères de sélection des techniques et mesures 70
A.1 Tableaux d'articles 71
A.2 Tableaux détaillés 80
Annexe B (normative) Principaux rôles et responsabilités relatifs au logiciel 86
Annexe C (informative) Résumé du contrôle des documents 96
Annexe D (informative) Bibliographie des techniques 98
D.1 Intelligence artificielle - Correction des défauts 98
D.2 Programmes analysables 98
D.3 Tests en avalanche/en surcharge 99
D.4 Analyse des valeurs aux limites 99
D.5 Rattrapage par régression 100
D.6 Schémas de cause et de conséquence 100
D.7 Listes de contrôle 100
D.8 Analyse de Flux de Contrôle 101
D.9 Analyse des défaillances de mode commun 101
D.10 Analyse du flux de données 102
D.11 Organigrammes des données 102
D.12 Enregistrement et analyse des données 103
D.13 Tables de décision (Tables de vérité) 104
D.14 Programmation défensive 104
D.15 Normes de codage et Guide de style 105
D.16 Programmation diversifiée 106
D.17 Reconfiguration dynamique 106
D.18 Tests de classes d'équivalence et de partition d'entrée 107
D.19 Codes de détection et de correction d'erreurs 107
D.20 Supposition d'erreurs 107
D.21 Insertion d'erreurs 108
D.22 Analyse par arbre des événements 108
D.23 Inspection de Fagan 109
D.24 Programmation par assertion des défaillances 109
D.25 AEEL – Analyse des Effets des Erreurs du Logiciel 109
D.26 Détection des défauts et diagnostic 110
D.27 Automates à états finis/Schémas de transitions d'état 111
D.28 Méthodes formelles 111
D.28.1 CSP - Processus Séquentiels de Communication 112
D.28.2 CCS - Algèbre des Systèmes de Transmission 113
D.28.3 HOL - Logique d'Ordre Supérieur 113
D.28.4 LOTOS 113
D.28.5 OBJ 114

D.28.6 Logique temporelle 114
D.28.7 VDM - Méthode de Développement de Vienne 115
D.28.8 Méthode Z 115
D.28.9 Méthode B 116
D.28.10 Vérification du modèle 116
D.29 Preuve formelle 117
D.30 Rattrapage par progression 117
D.31 Dégradation contrôlée 118
D.32 Analyse d'impact 118
D.33 Masquage d'informations/Encapsulation 118
D.34 Tests d'interface 119
D.35 Sous-ensemble de langage 119
D.36 Mémorisation des cas exécutés 120
D.37 Métriques 120
D.38 Approche modulaire 121
D.39 Modélisation des performances 121
D.40 Exigences en matière de performance 122
D.41 Tests probabilistes 122
D.42 Simulation de processus 123
D.43 Prototypage/Animation 123
D.44 Bloc de rattrapage 124
D.45 Temps de réponse et contraintes de place mémoire 124
D.46 Rattrapage par ré-exécution 124
D.47 Sécurité Contrôlée 125
D.48 Gestion de la configuration du logiciel 125
D.49 Langages de programmation à fort typage 125
D.50 Tests structurels 126
D.51 Schémas de structure 126
D.52 Méthodologie structurée 127
D.53 Programmation structurée 127
D.54 Langages de programmation adaptés 128
D.55 Réseaux de Pétri temporels 129
D.56 Révisions structurées/ Revues de la conception 129
D.57 Programmation orientée objet 129
D.58 Traçabilité 130
D.59 Métaprogrammation 131
D.60 Programmation procédurale 131
D.61 Graphes séquentiels de fonction 132
D.62 Schéma à contact 132
D.63 Diagramme fonctionnel 132
D.64 Graphe d'états ou Diagramme d'états 132
D.65 Modélisation de données 133
D.66 Diagramme de flux de commande/Graphe de flux de commande 133
D.67 Diagramme de séquence 134
D.68 Méthodes de spécification en tableaux 135
D.69 Langage spécifique à l'application 135
D.70 UML (Unified Modeling Language, langage de modélisation unifié) 135

D.71 Langages spécifiques à un domaine 136
Bibliographie 138

Figures

Figure 1 – Démarche illustrative relative au logiciel....9
Figure 2 – Illustration de la structure organisationnelle préférentielle 19
Figure 3 – Cycle de vie de développement 1....24
Figure 4 – Illustration d'un cycle de vie de développement 2 25

Tableaux

Tableau 1 - Relation entre les classe d'outils et les paragraphes applicables 39
Tableau A.1– Problèmes liés au cycle de vie et Documentation (5.3) 71
Tableau A.2 – Spécification des Exigences du Logiciel (7.2) 73
Tableau A.3 – Architecture du Logiciel (7.3)....74
Tableau A.4– Conception et mise en œuvre du logiciel (7.4)....75
Tableau A.5 – Vérification et Tests (6.2 et 7.3) 76
Tableau A.6 – Intégration (7.6) 76
Tableau A.7– Tests d'Ensemble du Logiciel (6.2et 7.7) 77
Tableau A.8 – Techniques d'analyse logicielle (6.3)....77
Tableau A.9 – Assurance Qualité du logiciel (6.5)....77
Tableau A.10 – Maintenance du Logiciel (9.2) 78
Tableau A.11 – Techniques de préparation des données (8.4)....79
Tableau A.12 – Normes de codage 80
Tableau A.13 – Analyse et Tests dynamiques....80
Tableau A.14 – Test fonctionnel/boîte noire 81
Tableau A.15 – Langages de programmation textuels 81
Tableau A.16 – Langages diagrammatiques pour algorithmes d'application 82
Tableau A.17 – Modélisation 82
Tableau A.18 – Tests de Performance....82
Tableau A.19 – Analyse statique 83
Tableau A.20 – Composants 83
Tableau A.21 – Couverture des tests pour le code 84
Tableau A.22 – Architecture de logiciel orienté objet 85
Tableau A.23 – Conception détaillée orientée objet 85
Tableau B.1 — Spécification du Rôle du Gestionnaire des Exigences 87
Tableau B.2 — Spécification du Rôle du Concepteur 88
Tableau B.3 — Spécification du Rôle du Réalisateur....89
Tableau B.4 — Spécification du Rôle du Chargé des tests....90
Tableau B.5 — Spécification du Rôle du Chargé de vérification 91
Tableau B.6 — Spécification du Rôle du Chargé d'intégration....92
Tableau B.7 — Spécification du Rôle du Chargé de Chargé de validation 93
Tableau B.8 — Spécification du Rôle du Chargé d'évaluation 94
Tableau B.9 — Spécification du Rôle du Chef de projet 95
Tableau B.10 — Spécification du Rôle du Gestionnaire de la Configuration 95
Tableau C.1 — Résumé du Contrôle des Documents 96

Introduction

La présente Norme Européenne fait partie intégrante d'un groupe de normes connexes. Les autres documents de ce groupe sont les EN 50126-1:1999 «*Applications ferroviaires - Spécification et démonstration de la fiabilité, de la disponibilité, de la maintenabilité et de la sécurité (FDMS) – Partie 1* : *Exigences de base et procédés génériques*» et EN 50129:2003 «*Applications ferroviaires - Systèmes de signalisation, de télécommunications et de traitement - Systèmes électroniques de sécurité pour la signalisation*».

L'EN 50126-1 traite des systèmes au niveau le plus général, tandis que l'EN 50129 traite des processus d'approbation des systèmes individuels qui peuvent exister dans le cadre du système ferroviaire global de contrôle-commande et de protection. La présente Norme Européenne traite en particulier des méthodes qu'il est nécessaire d'utiliser pour fournir des logiciels répondant aux exigences d'intégrité de la sécurité imposées par ces considérations plus larges.

La présente Norme Européenne fournit un ensemble d'exigences que le développement, le déploiement et la maintenance de tout logiciel de sécurité destiné aux applications ferroviaires de contrôle-commande et de protection doivent respecter. Elle définit les exigences concernant la structure organisationnelle, la relation entre organisations et la répartition des responsabilités impliquées dans les activités de développement, de déploiement et de maintenance. Des critères de qualification et d'expertise du personnel sont également fournis dans la présente Norme Européenne.

Le concept clé de la présente Norme Européenne est celui des niveaux d'intégrité de la sécurité logicielle. La présente Norme Européenne traite de cinq niveaux d'intégrité de sécurité logicielle dans lesquels 0 correspond au niveau le plus bas et 4 au niveau le plus élevé. Plus le risque résultant d'une défaillance logicielle est élevé, plus le niveau d'intégrité de la sécurité logicielle est élevé.

La présente Norme Européenne a identifié des techniques et mesures applicables aux cinq niveaux d'intégrité de la sécurité logicielle. Les techniques et mesures requises pour les niveaux 0 à 4 d'intégrité de la sécurité logicielle sont indiquées dans les tableaux de l'Annexe A (normative). Dans la présente version, les techniques requises pour le niveau 1 sont identiques à celles du niveau 2, et les techniques requises pour le niveau 3 sont identiques à celles du niveau 4. La présente Norme Européenne ne fournit aucune ligne directrice sur le niveau d'intégrité logicielle approprié pour un risque donné. Cette décision sera tributaire de nombreux facteurs, notamment de la nature de l'application, de la limite dans laquelle les autres systèmes assurent des fonctions de sécurité, ainsi que de facteurs socio-économiques.

Le processus de spécification des fonctions de sécurité allouées au logiciel fait partie du domaine d'application des normes EN 50126-1 et EN 50129.

La présente Norme Européenne spécifie les mesures nécessaires au respect de ces exigences.

Les EN 50126-1 et EN 50129 exigent qu'une approche systématique soit adoptée en ce qui concerne :

a) l'identification des situations dangereuses, l'évaluation des risques et la prise de décisions en fonction de critères de risque,

b) l'identification de la réduction des risques nécessaire au respect des critères d'acceptation de risque;

c) la définition d'une Spécification des Exigences de Sécurité du Système, globale, qui décrit les protections indispensables en vue d'atteindre la réduction des risques requise,

d) le choix d'une architecture système adaptée,

e) la planification, le contrôle et la maîtrise des activités techniques et de management nécessaires pour transformer la Spécification des exigences de sécurité du système en un Système de sécurité dont l'intégrité de la sécurité est validée.

Au fur et à mesure que la spécification se décompose en une conception comprenant des composants et des systèmes de sécurité, l'allocation des niveaux d'intégrité de la sécurité est effectuée. Finalement ceci conduit aux niveaux d'intégrité de la sécurité logicielle requis.

L'état actuel de la technique est tel que ni l'application des méthodes d'assurance qualité (mesures d'évitement des défauts et mesures de détection des défauts), ni l'application d'approches logicielles à tolérance aux pannes ne peuvent garantir la sécurité absolue du logiciel. Il n'existe aucun moyen connu de prouver l'absence de défauts dans un logiciel de sécurité même raisonnablement complexe, en particulier l'absence de défauts de spécification et de conception.

Les principes appliqués dans le développement de logiciels à haute intégrité incluent, sans s'y limiter :

- des méthodes de conception descendante,
- la modularité,
- la vérification de chaque phase du cycle de vie du développement,
- des composants vérifiés et des bibliothèques de composants,
- une documentation claire et la traçabilité,
- des documents aptes à être audités,
- la validation,
- l'évaluation,
- la gestion de configuration et le contrôle des modifications, et
- l'étude appropriée des questions de compétence de l'organisation et du personnel.

La Spécification des exigences de sécurité du système identifie toutes les fonctions de sécurité allouées au logiciel et détermine leur niveau d'intégrité de la sécurité du système. Les étapes fonctionnelles successives de l'application de la présente Norme Européenne sont montrées à la Figure 1 et consistent à :

a) définir la Spécification des Exigences du Logiciel et, en parallèle, considérer l'architecture du logiciel. La stratégie de sécurité pour le logiciel et le niveau d'intégrité de la sécurité logicielle (7.2 et 7.3) sont développés dans l'architecturedu logiciel;

b) concevoir, développer et tester le logiciel selon le Plan d'Assurance Qualité du Logiciel, le niveau d'intégrité de la sécurité logicielle et le cycle de vie du logiciel (7.4 et 7.5);

c) intégrer le logiciel sur le matériel cible et vérifier la fonctionnalité (7.6);

d) accepter et déployer le logiciel (7.7 et 9.1);

e) si la maintenance du logiciel est requise pendant la vie opérationnelle, réactiver le cas échéant la présente Norme Européenne (9.2).

Un certain nombre d'activités se déroulent pendant le développement du logiciel, parmi lesquelles les tests (6.1), la vérification (6.2), la validation (6.3), l'évaluation (6.4), l'assurance qualité (6.5) et le contrôle des modifications et des évolutions (6.6).

Des exigences sont données en ce qui concerne les outils (6.7) et les systèmes qui sont configurés par des données d'application ou par des algorithmes d'application (8).

Des exigences sont également fournies en ce qui concerne l'indépendance des rôles et la compétence du personnel impliqué dans le développement du logiciel (5.1, 5.2 et Annexe B).

La présente Norme n'impose pas l'utilisation d'un cycle de vie spécifique de développement du logiciel. Cependant des ensembles illustratifs de cycle de vie et de documentation sont fournis en 5.3, Figure 3, Figure 4 et en 7.1.

Des tableaux ont été établis pour classer diverses techniques/mesures par rapport aux niveaux 0 à 4 d'intégrité de la sécurité logicielle. Les tableaux sont dans l'Annexe A. En référence croisée avec les

tableaux, la bibliographie fournit une brève description de chaque technique/mesure avec des références à des sources complémentaires d'informations. La Bibliographie de techniques est dans l'Annexe D.

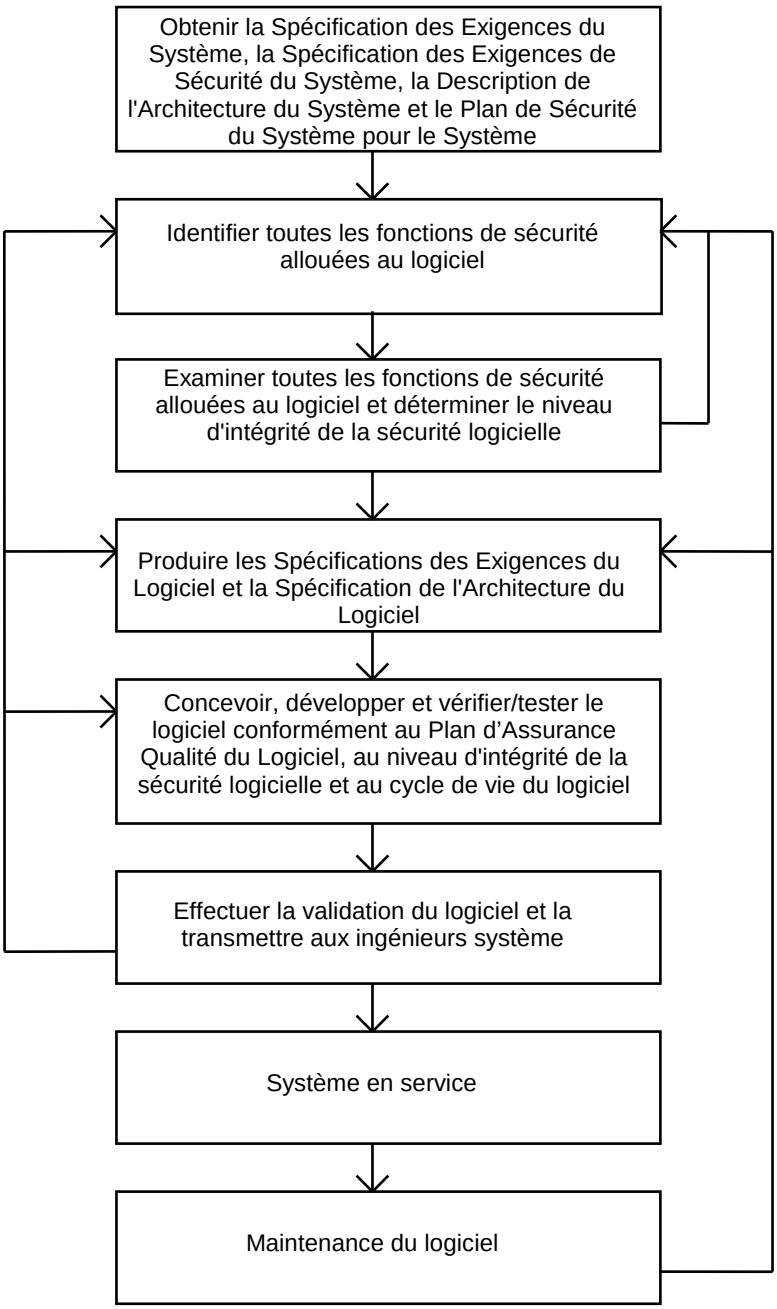


Figure 1 – Démarche illustrative relative au logiciel

1 Domaine d'application

1.1 La présente Norme Européenne spécifie les exigences de processus et les techniques applicables au développement de logiciel pour des systèmes électroniques programmables utilisés dans les applications ferroviaires de contrôle-commande et de protection. Elle est destinée à être utilisée dans tout domaine comportant des implications de sécurité. Ces systèmes peuvent être mis en œuvre à l'aide de microprocesseurs dédiés, de contrôleurs logiques programmables, de systèmes multiprocesseurs distribués, de grands systèmes dotés d'un calculateur central ou à l'aide d'autres architectures.

1.2 La présente Norme Européenne est exclusivement applicable au logiciel et à l'interaction entre le logiciel et le système auquel il appartient.

1.3 La présente Norme Européenne n'est pas pertinente pour les logiciels qui ont été identifiés comme n'ayant aucun impact sur la sécurité, c'est-à-dire pour les logiciels dont les défaillances ne peuvent pas affecter de fonctions de sécurité identifiées.

1.4 La présente Norme Européenne s'applique à tous les logiciels de sécurité utilisés dans des systèmes de contrôle-commande et de protection du ferroviaire, y compris :

- la programmation d'applications,
- les systèmes d'exploitation,
- les outils,
- les microprogrammes.

La programmation d'applications inclut la programmation de haut niveau, la programmation de bas niveau et la programmation spécifique personnalisée (par exemple : la logique à contacts d'un contrôleur logique programmable).

1.5 La présente Norme Européenne traite également de l'utilisation de logiciels et d'outils préexistants. Ces logiciels peuvent être utilisés si les exigences spécifiques en 7.3.4.7 et 6.5.4.16 relatives aux logiciels préexistants et aux outils 6.7 sont satisfaites.

1.6 Un logiciel développé selon une version quelconque de la présente Norme Européenne sera considéré conforme et non soumis aux exigences relatives aux logiciels préexistants.

1.7 La présente Norme Européenne considère que la conception moderne d'applications utilise fréquemment des logiciels génériques qui conviennent comme base pour diverses applications. Ces logiciels génériques sont ensuite configurés par des données et/ou des algorithmes, afin de produire le logiciel exécutable pour l'application. Les Articles généraux 1 à 6 et 9 de la présente Norme Européenne s'appliquent aux logiciels génériques ainsi qu'aux données d'application et algorithmes d'application. L'Article spécifique 7 s'applique uniquement pour les logiciels génériques alors que l'Article 8 fournit les exigences spécifiques pour les données d'application et algorithmes d'application.

1.8 La présente Norme Européenne ne vise pas les problèmes commerciaux. Il convient toutefois de les traiter comme une partie essentielle de tout accord contractuel. Il conviendra également que tous les articles de la présente Norme Européenne soient considérés soigneusement dans toute situation commerciale.

1.9 La présente Norme Européenne n'est pas destinée à être rétroactive. Elle s'applique donc principalement aux nouveaux développements et n'est applicable dans son intégralité aux systèmes existants que s'ils font l'objet de modifications importantes. Pour les modifications mineures, seul le paragraphe 9.2 s'applique. Le Chargé d'évaluation doit analyser les preuves fournies dans la documentation du logiciel pour confirmer si, oui ou non, la détermination de la nature et de l'étendue des modifications du logiciel sont adéquates. Cependant, il est hautement recommandé d'appliquer la présente Norme Européenne pendant les mises à niveau et la maintenance des logiciels existants.

2 Références normatives

Les documents de référence suivants sont indispensables pour l'application du présent document. Pour les références datées, seule l'édition citée s'applique. Pour les références non datées, la dernière édition du document de référence s'applique (y compris les éventuels amendements).

EN 50126-1:1999	Applications ferroviaires - Spécification et démonstration de la fiabilité, de la disponibilité, de la maintenabilité et de la sécurité (FDMS) - Partie 1 : Exigences de base et procédés génériques
EN 50129:2003	Applications ferroviaires - Systèmes de signalisation, de télécommunications et de traitement - Systèmes électroniques de sécurité pour la signalisation
EN ISO 9000	Systèmes de management de la qualité - Principes essentiels et vocabulaire (ISO 9000:2005)
EN ISO 9001	Systèmes de management de la qualité - Exigences (ISO 9001:2008)
ISO/CEI 90003:2004	Ingénierie du logiciel – Lignes directrices pour l'application de l'ISO 9001:2000 aux logiciels informatiques
Série ISO/CEI 9126	Génie du logiciel -- Qualité des produits

3 Termes, définitions et abréviations

3.1 Termes et définitions

Pour les besoins du présent document, les termes et définitions suivants s'appliquent.

3.1.1
évaluation
processus d'analyse afin de déterminer si un logiciel, qui peut inclure des processus, de la documentation, des composants logiciels et/ou matériels de systèmes et sous-systèmes, satisfait aux exigences spécifiées et afin de formuler un jugement sur le fait que le logiciel répond à l'objectif attendu. L'évaluation de la sécurité est une évaluation axée sur les propriétés de sécurité d'un système, mais sans s'y limiter

3.1.2
chargé d'évaluation
entité qui mène à bien une évaluation

3.1.3
logiciel standard disponible dans le commerce (COTS, en anglais commercial off-the-shelf software)

logiciel défini par les besoins du marché, disponible dans le commerce et dont l'adéquation aux besoins a été démontrée par un large éventail d'utilisateurs

3.1.4
composant
partie constitutive de logiciel qui a des interfaces et un comportement bien définis par rapport à la conception et à l'architecture du logiciel et satisfait aux critères suivants :

- elle est conçue conformément à “Composants” (voir Tableau A.20);
- elle couvre un sous-ensemble spécifique des exigences relatives au logiciel;
- elle est clairement identifiée et a une version indépendante au sein du système de gestion de configuration ou est une partie d'un ensemble de composants (par exemple : sous-systèmes) qui ont une version indépendante

3.1.5
gestionnaire de la configuration
entité qui est chargée de mettre en œuvre et d'exécuter les processus pour la gestion de configuration des documents, logiciels et outils connexes, y compris la gestion des modifications et des évolutions

3.1.6
client
entité qui achète un système de contrôle-commande et de protection du ferroviaire comprenant le logiciel

3.1.7
concepteur
entité qui analyse et transforme des exigences spécifiées en solutions de conception acceptables qui ont le niveau prescrit d'intégrité de la sécurité

3.1.8
entité
personne, groupe ou organisation qui remplit un rôle tel que défini dans la présente Norme européenne

3.1.9
erreur, faute
défaut, méprise ou inexactitude pouvant conduire à une défaillance ou à un écart par rapport au comportement ou à la performance prévu(e)

3.1.10
défaillance
différence inacceptable entre la performance requise et la performance observée

3.1.11
tolérance aux "fautes"
capacité propre à un système à délivrer d'une manière correcte et continue un service tel qu'il est spécifié, en présence d'un nombre limité de défauts matériels ou logiciels

3.1.12
micrologiciel
logiciel stocké dans une mémoire morte ou dans une mémoire semi permanente telle qu'une mémoire flash, d'une manière qui est fonctionnellement indépendante du logiciel applicatif

3.1.13
logiciel générique
logiciel pouvant être utilisé pour une grande variété d'installations simplement en fournissant des données et/ou algorithmes propres à l'application

3.1.14
réalisateur
entité qui transforme des choix de conception en leur réalisation physique

3.1.15
intégration
processus d'assemblage d'éléments de logiciel et/ou de matériel, selon la spécification de conception et d'architecture, et de tests de l'ensemble intégré

3.1.16
chargé d'intégration
entité qui mène à bien l'intégration du logiciel

3.1.17
logiciel préexistant
logiciel développé avant l'application dont il est ici question, incluant les logiciels COTS (standards disponibles dans le commerce) et libres.

3.1.18
logiciel libre
code source à la disposition du grand public avec des restrictions de droits d'auteur assouplies ou inexistantes

3.1.19
contrôleur logique programmable
système de commande informatisé, doté d'une mémoire programmable par l'utilisateur pour le stockage des instructions, pour réaliser des fonctions spécifiques

3.1.20
gestion de projet
conduite administrative et/ou technique d'un projet, y compris les aspects de sécurité

3.1.21
chef de projet
entité qui mène à bien la gestion de projet

3.1.22
fiabilité
aptitude d'une entité à réaliser une fonction dans des conditions données, pendant une période de temps donnée

3.1.23
robustesse
aptitude d'un élément à détecter et gérer les situations anormales

3.1.24
gestionnaire des exigences
entité qui mène à bien la gestion des exigences

3.1.25
gestions des exigences
processus consistant à identifier, documenter, analyser, classer par ordre de priorité et accepter par accord les exigences et ensuite contrôler les modifications et évolutions et communiquer avec les parties prenantes. Il s'agit d'un processus continu tout au long d'un projet

3.1.26
risque
combinaison du taux d'occurrence d'accidents et d'incidents occasionnant un dommage (causé par une situation dangereuse) et de la gravité de ce dommage

3.1.27
sécurité
absence de niveaux inacceptables de risque de dommage aux personnes

3.1.28
autorité de tutelle
organisme chargé de certifier que les logiciels ou services relatifs à la sécurité sont conformes aux exigences de sécurité statutaires applicables

3.1.29
fonction de sécurité
fonction qui met en œuvre tout ou partie d'une exigence relative à la sécurité

3.1.30
logiciel de **sécurité**
logiciel qui exécute des fonctions de sécurité

3.1.31
logiciel
création intellectuelle comprenant les programmes, les procédures, les règles, les données et toute documentation associée en rapport avec le fonctionnement d'un système

3.1.32
référentiel logiciel
ensemble complet et cohérent du code source, des fichiers exécutables, des fichiers de configuration, des scripts d'installation et de la documentation qui sont nécessaires à une version diffusée d'un logiciel. Les informations concernant les compilateurs, les systèmes d'exploitation, les logiciels préexistants et les outils indépendants font partie intégrante du référentiel. Elles permettront à l'organisation de reproduire des versions bien définies et seront les données d'entrée pour de futures versions diffusées pour améliorations ou de la mise à niveau dans la phase de maintenance

3.1.33
déploiement du logiciel
transfert, installation et activation d'un référentiel logiciel livrable correspondant à une version déjà diffusée et évaluée

3.1.34
cycle de vie du logiciel
ensemble des activités survenant pendant une période qui commence à la spécification du logiciel et se termine quand le logiciel ne peut plus être utilisé. En principe, le cycle de vie du logiciel inclut une phase de spécification, une phase de conception, une phase de tests, une phase d'intégration, une phase de déploiement et une phase de maintenance

3.1.35
maintenabilité du logiciel
capacité d'un logiciel à être modifié pour corriger les défauts, pour améliorer la performance ou d'autres attributs, ou pour l'adapter à un environnement différent

3.1.36
maintenance du logiciel
action, ou ensemble d'actions, effectuée sur le logiciel après son déploiement afin de renforcer ou corriger sa fonctionnalité

3.1.37
niveau d'intégrité de la sécurité logicielle
numéro de classification déterminant les techniques et mesures à appliquer au logiciel

NOTE Les logiciels relatifs à la sécurité ont été classés en cinq niveaux d'intégrité de sécurité, dans lesquels 0 est le plus bas et 4 le plus élevé.

3.1.38
fournisseur
entité qui conçoit et bâtit un système de contrôle-commande et de protection du ferroviaire comprenant le logiciel ou des parties de celui-ci

3.1.39
niveau d'intégrité de la sécurité du système
numéro indiquant le degré de confiance requis sur le fait qu'un système intégré, comprenant du matériel et du logiciel, respectera ses exigences de sécurité spécifiées

3.1.40
chargé des tests
entité qui mène à bien les tests

3.1.41 tests
processus d'exécution d'un logiciel dans des conditions maîtrisées de manière à s'assurer de son comportement et de sa performance par rapport à la spécification d'exigences correspondante

3.1.42
classe d'outils T1
ne génère pas de sorties susceptibles de contribuer, directement ou indirectement, au code exécutable (y compris les données) du logiciel

NOTE Les exemples de T1 comprennent : un éditeur de texte ou un outil d'aide à la conception ou aux exigences démuni de toute capacité de génération automatique de code; les outils de contrôle de configuration.

3.1.43
classe d'outils T2
permet le test ou la vérification de la conception ou du code exécutable, tandis que des erreurs internes à l'outil peuvent ne pas arriver à révéler des défauts mais sans créer directement des erreurs dans le logiciel exécutable

NOTE Des exemples de T2 comprennent : les générateurs de harnais de test, les outils de mesure de la couverture de tests, les outils d'analyse statique.

3.1.44
classe d'outils T3
génère des sorties susceptibles de contribuer, directement ou indirectement, au code exécutable (y compris les données) du système de sécurité

NOTE Les exemples de T3 comprennent : les compilateurs de code source, les compilateurs de données/algorithmes, les outils pour modifier les points de consigne au cours du fonctionnement du système; les compilateurs d'optimisation lorsque la relation entre le programme de code source et le code objet généré n'est pas évidente; les compilateurs qui incorporent un ensemble d'instructions exécutables dans le code exécutable.

3.1.45
traçabilité
facilité avec laquelle un lien peut être établi entre deux ou plusieurs produits d'un processus de développement, et plus particulièrement, ceux possédant entre eux une relation de prédécesseur à successeur ou de maître à esclave

3.1.46
validation
processus d'analyse suivie d'un jugement fondé sur des preuves afin de déterminer si un élément (par exemple : processus, documentation, logiciel ou application) répond aux besoins d'un utilisateur, en particulier en termes de sécurité et de qualité et en mettant l'accent sur l'adéquation de son fonctionnement selon son objectif dans son environnement prévu

3.1.47
chargé de validation
entité qui est responsable de la validation

3.1.48
vérification
processus d'examen suivi d'un jugement fondé sur des preuves que des éléments de sortie (processus, documentation, logiciel ou application) d'une phase de développement spécifique satisfont aux exigences de la phase en question en termes de complétude, de correction et de cohérence

NOTE La vérification est surtout fondée sur des revues de documents (documents de conception, de réalisation, de test, etc.)

3.1.49
chargé de vérification
entité qui est responsable d'une ou plusieurs activités de vérification

3.2 Abréviations

Pour les besoins du présent document, les abréviations suivantes s'appliquent.

ASR	Assessor (Chargé d'évaluation)
CdP	Chef de projet
COTS	Commercial off-the-shelf (Standard, disponibles dans le commerce)
DES	Designer (Concepteur)
FDMS	Fiabilité, disponibilité, maintenabilité et sécurité
HR	Hautement recommandé
IMP	Implementer (Réalisateur)

INT	Chargé d'intégration
JSD	Méthode Jackson System Development
M	Mandatory (Obligatoire)
MASCOT	Modular Approach to Software Construction, Operation and Test (Approche modulaire à la construction, l'exploitation et l'essai de logiciel)
NR	Non recommandé
R	Recommandé
RQM	Requirements Manager (Gestionnaire des exigences)
SDL	Specification and Description Language (Langage de spécification et de description)
SFC	Sequential Function Charts (Graphes séquentiels de fonction)
SIL	Safety Integrity Level (Niveau d'intégrité de la sécurité)
SOM	Service Oriented Modeling (Modélisation orientée service)
SSADM	Structured Systems Analysis & Design Methodology (Analyse structurée des systèmes et méthodologie de conception)
TST	Chargé des tests
V&V	Vérification et validation
VAL	Chargé de validation
VER	Chargé de vérification

4 Objectifs, conformité et niveaux d'intégrité de sécurité du logiciel

4.1 L'allocation au logiciel de fonctions de sécurité du système, ainsi que d'interfaces de logiciel, doit être identifiée dans la documentation du système. Le système dans lequel le logiciel est intégré doit être complètement défini quant aux éléments suivants :

- les fonctions et interfaces;
- les conditions d'application;
- la configuration ou l'architecture du système;
- les situations dangereuses à maîtriser;
- les exigences d'intégrité de la sécurité;
- l'allocation des exigences et du niveau de SIL au logiciel et au matériel;
- contraintes de temps.

NOTE L'allocation des exigences d'intégrité de la sécurité peut conduire à un SIL différent pour des parties bien distinctes de logiciel et de matériel d'un sous-système. Cette allocation dépend de la contribution des parties de logiciel et de matériel aux fonctions relatives à la sécurité et dépend aussi des mécanismes de limitation des défaillances y compris la séparation de fonction avec un SIL différent.

4.2 L'intégrité de sécurité du logiciel doit être spécifiée comme l'un des cinq niveaux, de SIL 0 (le plus bas) à SIL 4 (le plus élevé).

4.3 Le niveau d'intégrité de la sécurité logicielle requis doit être décidé et évalué au niveau du système sur la base du niveau d'intégrité de la sécurité du système et du niveau de risque associé à l'utilisation du logiciel dans le système.

4.4 Au moins les exigences associées au SIL 0 de la présente Norme européenne doivent être satisfaites pour la partie logicielle des fonctions qui ont un impact sur la sécurité en dessous de SIL 1. Ceci est dû au fait que l'incertitude existe dans l'évaluation du risque, voire dans l'identification des situations dangereuses. En cas d'incertitude, il est prudent de compter sur un faible niveau d'intégrité de sécurité, représenté par SIL 0, plutôt que sur aucun.

4.5 Pour se conformer à la présente Norme européenne, il doit être montré que chacune des exigences a été satisfaite vis à vis du niveau d'intégrité de la sécurité logicielle défini et que l'objectif du paragraphe en question a donc été atteint.

4.6 Là où une exigence est qualifiée par les termes "Dans la limite requise par le niveau d'intégrité de la sécurité logicielle", cela indique qu'une panoplie de techniques et de mesures doit être utilisée pour satisfaire à cette exigence.

4.7 Lorsque le paragraphe 4.6 est appliqué, les tableaux de l'Annexe A normative doivent être utilisés pour faciliter la sélection des techniques et des mesures appropriées au niveau d'intégrité de la sécurité logicielle. La sélection doit être documentée dans le Plan d'Assurance Qualité du Logiciel ou dans un autre document mentionné par celui-ci. Des lignes directrices sur ces techniques sont données dans l'Annexe D informative.

4.8 Si une technique ou une mesure comptant parmi celles hautement recommandées (HR) dans les tableaux n'est pas utilisée, la justification de l'utilisation de techniques alternatives doit être détaillée et enregistrée soit dans le Plan d'Assurance Qualité du Logiciel, soit dans un autre document mentionné par ce dernier. Cette formalité n'est pas nécessaire si l'on a recours à une combinaison approuvée des techniques fournies dans le tableau correspondant. Il doit être démontré que les techniques choisies ont été correctement appliquées.

4.9 Si l'utilisation d'une technique ou d'une mesure non contenue dans les tableaux est proposée, son efficacité et sa pertinence par rapport au respect de l'exigence particulière et de l'objectif global de l'article doivent être justifiées et enregistrées soit dans le Plan d'Assurance Qualité du Logiciel, soit dans un autre document mentionné par le Plan d'Assurance Qualité du Logiciel.

4.10 La conformité aux exigences d'un paragraphe particulier et à leurs techniques et mesures respectives détaillées dans les tableaux doit être vérifiée par l'inspection des documents requis par la présente Norme européenne. Le cas échéant, d'autres preuves tangibles, l'audit et la présence lors des tests doivent aussi être pris en compte.

5 Organisation et gestion du développement logiciel

5.1 Organisation, rôles et responsabilités

5.1.1 Objet

5.1.1.1 Assurer que l'ensemble du personnel responsable du logiciel est organisé, habilité et capable d'assumer ses responsabilités.

5.1.2 Exigences

5.1.2.1 *A minima*, le fournisseur doit mettre en œuvre les parties de l'EN ISO 9001 qui traitent de l'organisation et de la gestion du personnel et des responsabilités.

5.1.2.2 Les responsabilités doivent être conformes aux exigences définies dans l'Annexe B.

5.1.2.3 Le personnel affecté à ces rôles impliqués dans le développement ou la maintenance du logiciel doit être nommé et enregistré.

5.1.2.4 Un Chargé d'évaluation doit être désigné par le fournisseur, le client ou l'autorité de tutelle.

5.1.2.5 Le Chargé d'évaluation doit être indépendant du fournisseur ou, à la discrétion de l'autorité de tutelle, appartenir à l'organisation du fournisseur ou à celle du client.

5.1.2.6 Le Chargé d'évaluation doit être indépendant du projet.

5.1.2.7 Autorité doit être donnée au Chargé d'évaluation pour accomplir l'évaluation du logiciel.

5.1.2.8 Le Chargé de validation doit signifier son accord/désaccord pour toute version diffusée du logiciel.

5.1.2.9 Tout au long du cycle de vie du logiciel, l'attribution des rôles aux personnes doit être conforme aux paragraphes 5.1.2.10 à 5.1.2.14, dans toute la mesure requise par le SIL du logiciel.

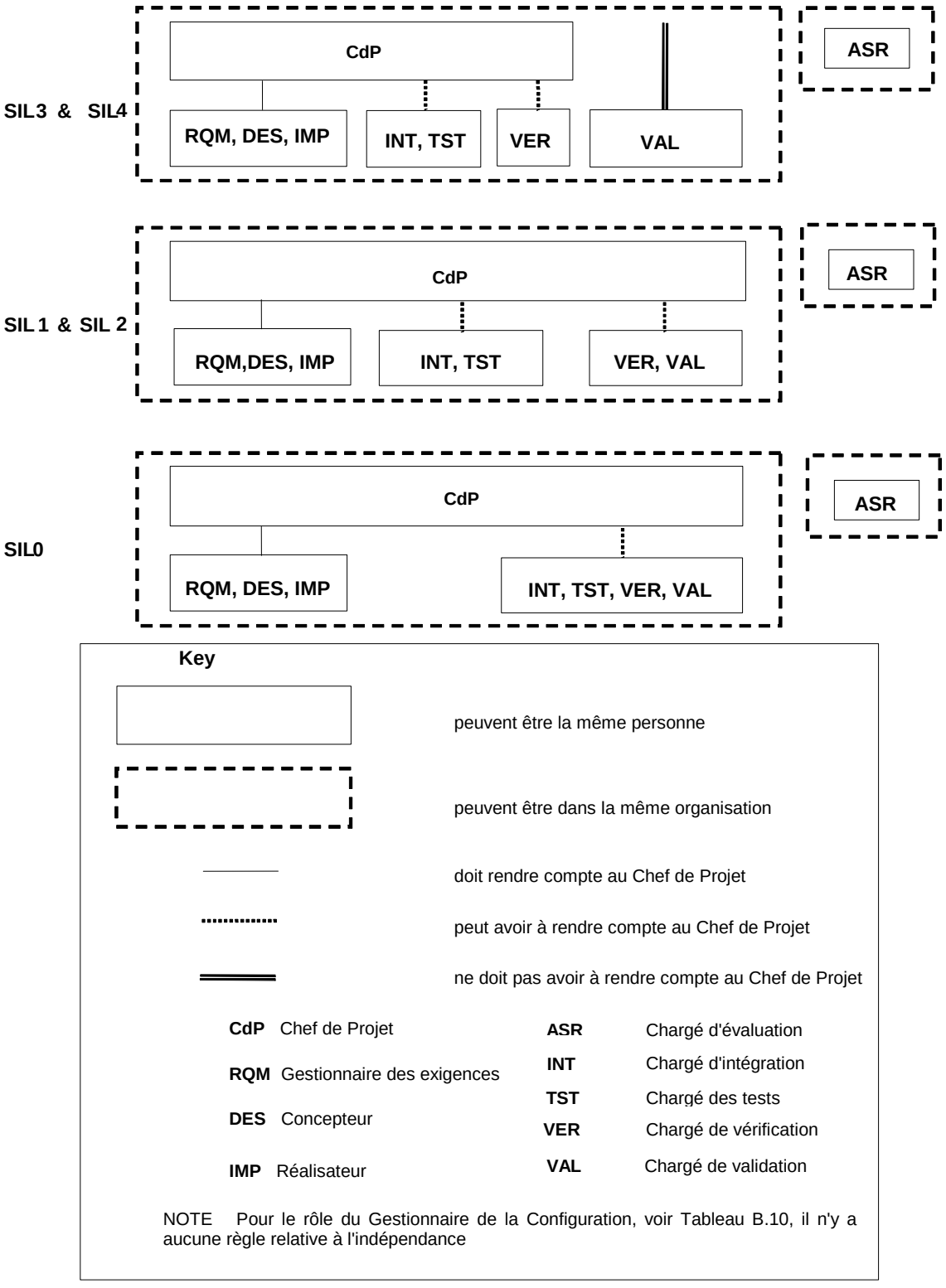


Figure 2 – Illustration de la structure organisationnelle préférentielle

NOTE La Figure 2 est uniquement une illustration de la structure organisationnelle préférentielle.

5.1.2.10 La structure organisationnelle préférentielle pour le SIL 3 et le SIL 4 correspond à ce que :

a) Le Gestionnaire des exigences, le Concepteur et le Réalisateur pour un composant logiciel peuvent être la même personne.
b) Le Gestionnaire des exigences, le Concepteur et le Réalisateur pour un composant logiciel doivent rendre compte au Chef de projet.
c) Le Chargé d'intégration et le Chargé des tests pour un composant logiciel peuvent être la même personne.
d) Le Chargé d'intégration et le Chargé des tests pour un composant logiciel peuvent rendre compte au Chef de projet ou au Chargé de validation.
e) Le Chargé de vérification peut rendre compte au Chef de projet ou au Chargé de validation.
f) Le Chargé de validation ne doit pas rendre compte au Chef de projet, à savoir le Chef de projet ne doit avoir aucune influence sur les décisions du Chargé de validation mais le Chargé de validation fournit au Chef de projet des informations sur ses décisions.
g) Une personne qui est un Gestionnaire des exigences, un Concepteur ou un Réalisateur pour un composant logiciel ne doit être ni Chargé des tests ni Chargé d'intégration pour le même composant logiciel.
h) Une personne qui est un Chargé d'intégration ou Chargé des tests pour un composant logiciel ne doit être ni Gestionnaire des exigences ni Concepteur ni Réalisateur pour le même composant logiciel.
i) Une personne qui est Chargé de vérification ne doit être ni Gestionnaire des exigences, Concepteur, Réalisateur, Chargé d'intégration, Chargé des tests ni Chargé de validation.
j) Une personne qui est Chargé de validation ne doit être ni Gestionnaire des exigences, Concepteur, Réalisateur, Chargé d'intégration, Chargé des tests ni Chargé de vérification.
k) Une personne qui est Chef de projet peut en plus remplir les rôles de Gestionnaire des exigences, de Concepteur, de Réalisateur, de Chargé d'intégration, de Chargé des tests ou de Chargé de vérification à la condition que les exigences relatives à l'indépendance entre ces rôles supplémentaires soient respectées.
l) Le Chef de projet, le Gestionnaire des exigences, le Concepteur, le Réalisateur, le Chargé d'intégration, le Chargé des tests, le Chargé de vérification et le Chargé de validation peuvent appartenir à la même organisation.
m) Le Chargé d'évaluation doit être indépendant et, du point de vue organisationnel, indépendant par rapport aux rôles de Chef de projet, Gestionnaire des exigences, Concepteur, Réalisateur, Chargé d'intégration, Chargé des essais, Chargé de vérification et Chargé de validation.

Cependant, les options suivantes peuvent s'appliquer :

n) Une personne qui est Chargé de validation peut aussi remplir le rôle de Chargé de vérification mais en maintenant toujours l'indépendance par rapport au Chef de projet. Dans ce cas, les documents produits par le Chargé de vérification doivent être revus par une autre personne compétente ayant le même niveau d'indépendance que le Chargé de validation. Cette option organisationnelle doit être soumise à l'approbation du Chargé d'évaluation.

o) Une personne qui est Chargé de vérification peut également remplir le rôle de Chargé d'intégration et de Chargé des tests. Dans ce cas, le rôle du Chargé de validation doit vérifier l'adéquation des preuves documentées obtenues par intégration et tests avec les objectifs de vérification spécifiés, conservant donc deux niveaux de contrôle au sein de l'organisation du projet.

5.1.2.11 La structure organisationnelle préférentielle pour le SIL 1 et le SIL 2 correspond à ce que :

a) Le Gestionnaire des exigences, le Concepteur et le Réalisateur pour un composant logiciel peuvent être la même personne et doivent rendre compte au Chef de projet.
b) Le Chargé d'intégration et le Chargé des tests pour un composant logiciel peuvent être la même personne.
c) Le Chargé d'intégration et le Chargé des tests pour un composant logiciel peuvent rendre compte au Chef de projet ou au Chargé de validation.
d) Le Chargé de vérification et le Chargé de validation peuvent être la même personne.
e) Le Chargé de vérification et le Chargé de validation peut rendre compte au Chef de projet.
f) Une personne qui est Gestionnaire des exigences, Concepteur ou Réalisateur pour un composant logiciel ne doit être ni Chargé des tests ni Chargé d'intégration pour le même composant logiciel.
g) Une personne qui est un Chargé d'intégration ou Chargé des tests pour un composant logiciel ne doit être ni Gestionnaire des exigences ni Concepteur ni Réalisateur pour le même composant logiciel.

h) Une personne qui est Chargé de vérification ou Chargé de validation ne doit être ni Gestionnaire des exigences, Concepteur, Réalisateur, Chargé d'intégration ni Chargé des tests.
i) Une personne qui est Chef de projet peut en plus remplir les rôles de Gestionnaire des exigences, de Concepteur, de Réalisateur, de Chargé d'intégration, de Chargé des tests, de Chargé de vérification ou de Chargé de validation à la condition que les exigences relatives à l'indépendance entre ces rôles supplémentaires soient respectées.
j) Le Chef de projet, le Gestionnaire des exigences, le Concepteur, le Réalisateur, le Chargé d'intégration, le Chargé des tests, le Chargé de vérification et le Chargé de validation peuvent appartenir à la même organisation.
k) Le Chargé d'évaluation doit être indépendant et, du point de vue organisationnel, indépendant par rapport aux rôles de Chef de projet, Gestionnaire des exigences, Concepteur, Réalisateur, Chargé d'intégration, Chargé des tests, Chargé de vérification et Chargé de validation.

Cependant, les options suivantes peuvent s'appliquer :

l) Une personne qui est Chargé de vérification peut aussi remplir le rôle de Chargé d'intégration et de Chargé des tests. Dans ce cas, le rôle du Chargé de validation doit inclure la revue des documents produits par le Chargé de vérification, conservant donc deux niveaux de contrôle au sein de l'organisation du projet.

m) Une personne qui est Chargé de validation peut aussi remplir le rôle de Chargé de vérification, de Chargé d'intégration et de Chargé des tests. Dans ce cas les documents produits par le Chargé de vérification doivent être revus par une autre personne compétente ayant le même niveau d'indépendance que le Chargé de validation. Cette option organisationnelle doit être soumise à l'approbation du Chargé d'évaluation.

5.1.2.12 La structure organisationnelle préférentielle pour le SIL 0 correspond à ce que :

a) Le Gestionnaire des exigences, le Concepteur et le Réalisateur pour un composant logiciel peuvent être la même personne et doivent être gérés par le Chef de projet.
b) Le Chargé d'intégration, le Chargé des tests, le Chargé de vérification et le Chargé de validation pour un composant logiciel peuvent être la même personne.
c) Le Chargé d'intégration, le Chargé des tests, le Chargé de vérification et le Chargé de validation peuvent être gérés par le Chef de projet.
d) Une personne qui est Gestionnaire des exigences, Concepteur ou Réalisateur pour un composant logiciel ne doit être ni Chargé des tests ni Chargé d'intégration pour le même composant logiciel.
e) Une personne qui est Chargé de vérification ou Chargé de validation ne doit être ni Gestionnaire des exigences, Concepteur ni Réalisateur.
f) Une personne qui est Chef de projet peut en plus remplir les rôles de Gestionnaire des exigences, de Concepteur, de Réalisateur, de Chargé d'intégration, de Chargé des tests, de Chargé de vérification ou de Charge de validation à la condition que les exigences relatives à l'indépendance entre ces rôles supplémentaires soient respectées.
g) Le Chef de projet, le Gestionnaire des exigences, le Concepteur, le Réalisateur, le Chargé d'intégration, le Chargé des tests, le Chargé de vérification et le Chargé de validation peuvent appartenir à la même organisation.
h) Le Chargé d'évaluation doit être indépendant et, du point de vue organisationnel, indépendant par rapport aux rôles de Chef de projet, Gestionnaire des exigences, Concepteur, Réalisateur, Chargé d'intégration, Chargé des tests, Chargé de vérification et Chargé de validation.

Cependant, les variantes suivantes peuvent s'appliquer :

i) le Gestionnaire des exigences, le Concepteur, le Réalisateur, le Chargé d'intégrationet le Chargé des tests peuvent être la même personne.
j) Le Chargé de validation et le Chargé de vérification peuvent aussi être la même personne.
k) Une personne qui est Chargé de vérification ou Chargé de validation ne doit être ni Gestionnaire des exigences, Concepteur ni Réalisateur.

5.1.2.13 Les rôles de Gestionnaire des exigences, de Concepteur et de Réalisateur pour un certain composant peuvent exercer les rôles de Chargé des tests et de Chargé d'intégration pour un composant différent.

5.1.2.14 Les rôles du Chargé de vérification et du Chargé de validation doivent être définis au niveau du projet et doivent rester inchangés tout au long du projet de développement.

5.2 Compétence du personnel

5.2.1 Objectifs

5.2.1.1 Assurer que l'ensemble du personnel responsable du logiciel a les compétences pour s'acquitter de ces responsabilités en démontrant la capacité d'exécuter les tâches correspondantes de manière correcte, efficace et cohérente à un niveau de qualité élevé et dans des conditions variables.

5.2.2 Exigences

5.2.2.1 Les principales compétences requises pour chaque rôle dans le développement du logiciel sont définies dans l'Annexe B. Si une expérience, des capacités ou des qualifications supplémentaires sont requises pour un rôle dans le cycle de vie du logiciel, celles-ci doivent être définies dans le Plan d'Assurance Qualité du Logiciel.

5.2.2.2 La preuve documentée de la compétence du personnel, y compris les connaissances techniques, les qualifications, l'expérience adéquate et la formation appropriée, doit être conservée par l'organisation du fournisseur afin de démontrer une organisation de sécurité appropriée.

5.2.2.3 L'organisation doit maintenir les procédures de gestion des compétences du personnel à être apte aux rôles appropriés en conformité à des normes de qualité existantes.

5.2.2.4 Une fois apportée, à la satisfaction d'un chargé d'évaluation ou par certification, la preuve de la démonstration des compétences de l'ensemble du personnel affecté aux divers rôles, il sera nécessaire pour chaque individu de démontrer un maintien et un développement continus des compétences. Cela peut se démontrer par la tenue d'un journal de bord montrant que l'activité est effectuée correctement sur une base régulière et qu'une formation complémentaire est dispensée conformément à l'EN ISO 9001 et à 6.2.2 «Compétence, sensibilisation et formation» de la CEI 90003:2004.

5.3 Questions relatives au cycle de vie et à la documentation

5.3.1 Objectifs

5.3.1.1 Structurer le développement du logiciel en phases et activités définies.

5.3.1.2 Enregistrer toutes les informations en rapport avec le logiciel tout au long de son cycle de vie.

5.3.2 Exigences

5.3.2.1 Un modèle de cycle de vie pour le développement du logiciel doit être choisi. Ce modèle doit être détaillé dans le Plan d'Assurance Qualité du Logiciel conformément à 6.5.

Deux exemples de modèle de cycle de vie sont présentés à la Figure 3 et à la Figure 4.

5.3.2.2 Le modèle de cycle de vie doit prendre en compte la possibilité d'itérations à l'intérieur des phases et entre celles-ci.

5.3.2.3 Les procédures d'Assurance Qualité doivent être menées à bien parallèlement aux activités du cycle de vie et doivent utiliser les mêmes termes.

5.3.2.4 Le Plan d'Assurance Qualité du Logiciel, le Plan de Vérification du Logiciel, le Plan de Validation du Logiciel et le Plan de Gestion de Configuration du Logiciel doivent être rédigés au début du projet et maintenus tout au long du cycle de vie de développement du logiciel.

5.3.2.5 Toutes les activités à exécuter pendant une phase doivent être définies et planifiées avant le commencement de la phase.

5.3.2.6 Tous les documents doivent être structurés de manière à permettre un enrichissement continu parallèlement au processus de développement.

5.3.2.7 La traçabilité de chaque document doit être assurée en termes d'un numéro de référence unique et d'une relation définie et documentée à d'autres documents.

5.3.2.8 Chaque terme, acronyme ou abréviation doit avoir le même sens dans tous les documents. Si cela n'est pas possible pour des raisons historiques, les différents sens doivent être cités avec leurs références.

5.3.2.9 À l'exception de la documentation liée aux logiciels préexistants (voir 7.3.4.7), chaque document doit être rédigé conformément aux règles suivantes :

- il doit contenir ou mettre en œuvre toutes les conditions et exigences applicables du document précédent avec lequel il a une relation hiérarchique;
- il ne doit pas être en contradiction avec le document précédent.

5.3.2.10 Il doit être fait référence à chaque entité ou concept en utilisant le même nom ou la même description dans tous les documents.

5.3.2.11 Le contenu de tous les documents doit être enregistré dans un format approprié à la manipulation, au traitement et au stockage.

5.3.2.12 Lorsque des documents produits par des rôles indépendants sont combinés en un seul document, la relation aux parties produites par un rôle indépendant quelconque doit être tracée au sein du document.

5.3.2.13 Les documents peuvent être regroupés ou divisés conformément à 5.3.2.12. Certaines étapes du développement peuvent être combinées, divisées ou, lorsque cela est justifié, éliminée, à la discrétion du Chef de projet et avec l'accord du Chargé de validation.

5.3.2.14 Si une variante de cycle de vie ou de structure de documentation a été adoptée, il doit être établi qu'elle satisfait à tous les objectifs et exigences de la présente Norme européenne.

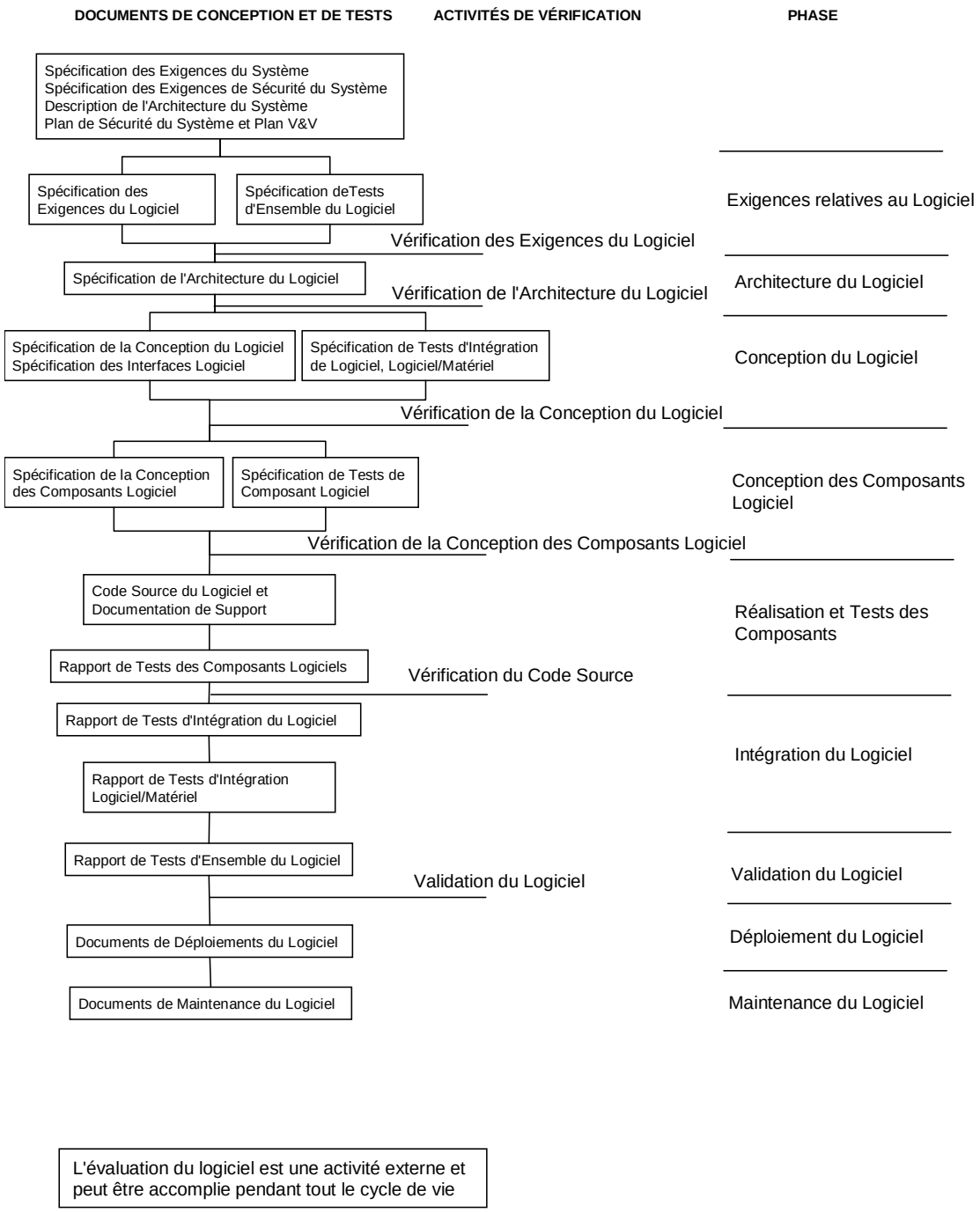


Figure 3 – Cycle de vie de développement 1

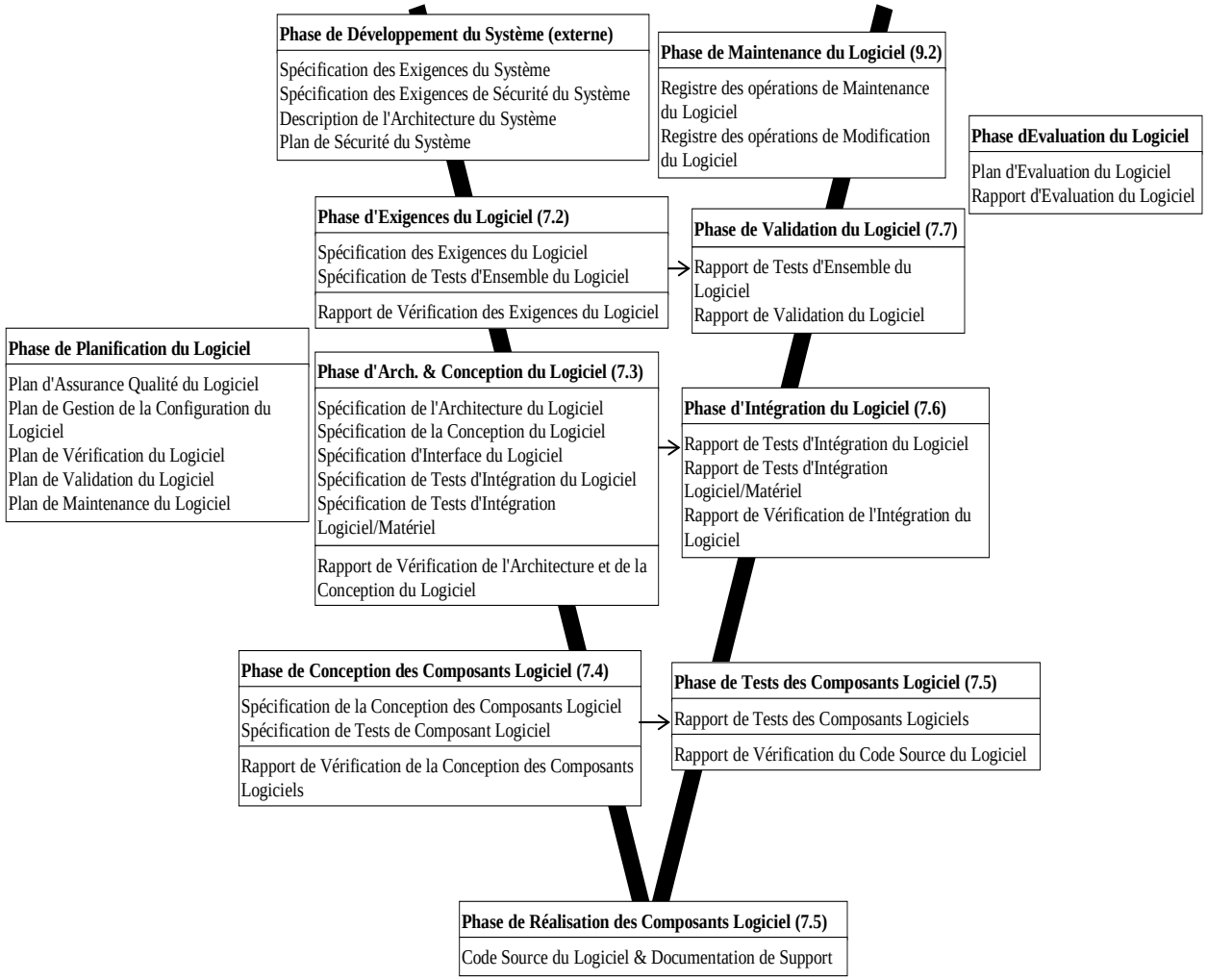


Figure 4 – Illustration d'un cycle de vie de développement 2

6 Assurance du logiciel

6.1 Test du logiciel

6.1.1 Objet

6.1.1.1 Le but des tests du logiciel, tel qu'effectué par le Chargé des tests et/ou le Chargé d'intégration, est de s'assurer du comportement ou de la performance du logiciel par rapport à la spécification de tests correspondante jusqu'au périmètre atteignable par la couverture de test choisie.

6.1.2 Documents en entrée

1) L'ensemble de la documentation nécessaire du système, du matériel et du logiciel telle que spécifiée dans le Plan de vérification du logiciel.

6.1.3 Documents en sortie

1) Spécification de Tests d'Ensemble du Logiciel

2) Rapport de Tests d'Ensemble du Logiciel

3) Spécification de Tests d'Intégration du Logiciel

4) Rapport de Tests d'Intégration du Logiciel

5) Tests d'Intégration Logiciel/Matériel

6) Rapport de Tests d'Intégration Logiciel/Matériel

7) Spécification de Tests de Composant Logiciel

8) Rapport de Tests des Composants Logiciels

6.1.4 Exigences

6.1.4.1 Les tests effectués par d'autres entités telles que le Gestionnaire des exigences, le Concepteur ou le Réalisateur, s'ils sont pleinement documentés et totalement conformes aux exigences suivantes, peuvent être acceptés par le Chargé de vérification.

6.1.4.2 Le matériel de mesure utilisé pour les tests doit être étalonné de manière appropriée. tous les outils, matériels ou logiciels utilisés pour les tests doivent être démontrés adaptés aux objectifs.

6.1.4.3 Les tests du logiciel doivent être documentés par une Spécification de Tests et un Rapport de Tests, tels que définis dans ce qui suit.

6.1.4.4 Chaque Spécification de Tests doit documenter ce qui suit :

a) les objectifs de test;

b) les scénarios de test, les données de test et les résultats attendus;

c) les types de test à effectuer;

d) l'environnement de test, les outils, la configuration et les programmes;

e) les critères de test sur lesquels l'achèvement du test sera jugé;

f) les critères à satisfaire et les degrés de couverture des tests à atteindre;

g) les rôles et responsabilités du personnel impliqué dans le processus de test;

h) les exigences couvertes par la Spécification de Tests;

i) la sélection et l'utilisation du matériel de test du logiciel.

6.1.4.5 Un Rapport de Tests doit être émis comme suit :

a) Le Rapport de Tests doit mentionner les noms des Chargés des tests, énoncer les résultats de tests et indiquer si, oui ou non, les objectifs des tests et les critères de tests selon la Spécification de Tests ont été satisfaits. Les échecs doivent être documentés et récapitulés;

b) les scénarios de test et leurs résultats doivent être consignés, de préférence dans un format lisible par la machine en vue d'une analyse ultérieure;

c) Les tests doivent être reproductibles et, dans la mesure du possible, exécutés par des moyens automatiques;

d) Les scripts de test pour l'exécution automatique des tests doivent être vérifiés;

e) l'identité et la configuration de tous les éléments concernés (matériel utilisé, logiciel utilisé, équipements utilisés, étalonnage des équipements, ainsi que les informations de version de la spécification d'essai) doivent être documentées;

f) une évaluation de la couverture des tests et de l'achèvement des tests doit être fournie et tout écart consigné.

6.2 Vérification du logiciel

6.2.1 Objet

6.2.1.1 L'objectif de la vérification du logiciel est d'examiner et parvenir à un jugement fondé sur la preuve que les éléments de sortie (processus, documentation, logiciel ou application) d'une phase de développement spécifique satisfont aux exigences et aux plans en ce qui concerne la complétude, l'exactitude et la cohérence. Ces activités sont gérées par le Chargé de vérification.

6.2.2 Documents en entrée

1) Toute la documentation nécessaire concernant le système, le matériel et le logiciel.

6.2.3 Documents en sortie

1) Plan de Vérification du Logiciel

2) Rapport(s) de Vérification du Logiciel

3) Rapport de vérification concernant l'Assurance Qualité du logiciel

6.2.4 Exigences

6.2.4.1 La vérification doit être documentée par au moins un Plan de Vérification du Logiciel et un ou plusieurs Rapports de vérification (relatifs au processus).

6.2.4.2 Un Plan de Vérification du Logiciel doit être rédigé, sous la responsabilité du Chargé de vérification, sur la base de la documentation nécessaire.

Les exigences de 6.2.4.3 à 6.2.4.9 se rapportent au Plan de Vérification du Logiciel.

6.2.4.3 Le Plan de Vérification du Logiciel doit décrire les activités à exécuter pour assurer une vérification correcte et la prise en charge correcte des besoins particuliers en matière de conception ou autre vérification.

6.2.4.4 Pendant le développement (et selon la taille du système), il est admis que le plan soit divisé en un certain nombre de sous-documents qui seront complétés à mesure que les besoins détaillés de vérification deviennent plus clairs.

6.2.4.5 Le Plan de Vérification du Logiciel doit documenter tous les critères, techniques et outils à utiliser dans le processus de vérification. Le Plan de Vérification du Logiciel doit inclure les techniques et mesures choisies dans le Tableau A.5, le Tableau A.6, le Tableau A.7 et le Tableau A.8. La combinaison choisie doit être justifiée comme un ensemble satisfaisant à 4.8, 4.9 et 4.10

6.2.4.6 Le Plan de Vérification du Logiciel doit décrire les activités à exécuter pour assurer l'exactitude et la cohérence vis-à-vis des entrées de chaque phase. Ces activités comprennent la revue, les tests et l'intégration.

6.2.4.7 Dans chaque phase de développement, il doit être montré que les exigences en matière de fonctionnement, de performance et de sécurité sont respectées.

6.2.4.8 Les résultats de chaque vérification doivent être conservés dans un format défini ou référencé dans le Plan de Vérification du Logiciel.

6.2.4.9 Le Plan de Vérification du Logiciel doit inclure les éléments suivants :

a) la sélection des stratégies et des techniques de vérification (pour éviter de compliquer inutilement l'évaluation de la vérification et des tests, la préférence doit être donnée à la sélection de techniques qui sont elles-mêmes facilement analysables);

b) sélection de techniques issues du Tableau A.5, du Tableau A.6, du Tableau A.7 et du Tableau A.8;

c) la sélection et la documentation des activités de vérification;

d) l'évaluation des résultats de vérification obtenus;

e) l'évaluation des exigences de sécurité et de robustesse;

f) les rôles et responsabilités du personnel impliqué dans le processus de vérification;

g) le degré de couverture des tests fonctionnels qu'il est requis d'atteindre;

h) la structure et le contenu de chaque étape de vérification, notamment pour la Vérification des Exigences du Logiciel (7.2.4.22), l'Architecture du Logiciel et la Vérification de la Conception (7.3.4.41, 7.3.4.42), la Vérification des Composants Logiciels (7.4.4.13), la Vérification du Code Source du Logiciel (7.5.4.10) et la Vérification de l'Intégration (7.6.4.13) d'une manière qui facilite la revue par rapport au Plan de Vérification du Logiciel.

6.2.4.10 Un Rapport de Vérification d'Assurance Qualité du Logiciel doit être rédigé, sous la responsabilité du Chargé de vérification, sur la base des documents en entrée de 6.2.2.

L'exigence en 6.2.4.11 se rapporte au Rapport de Vérification d'Assurance Qualité du logiciel.

6.2.4.11 Une fois que le Plan de Vérification du Logiciel a été défini, la vérification doit porter sur :

a) le fait que le Plan de Vérification du Logiciel satisfait aux exigences générales de lisibilité et de traçabilité de 5.3.2.7 à 5.3.2.10 et de 6.5.4.14 à 6.5.4.17 ainsi qu'aux exigences spécifiques dans 6.2.4.3 à 6.2.4.9,

b) la cohérence interne du Plan de Vérification du Logiciel.

Les résultats doivent être enregistrés dans un Rapport de Vérification d'Assurance Qualité du Logiciel.

6.2.4.12 Tous les éventuels Rapports de Vérification du Logiciel doivent être rédigés, sous la responsabilité du Chargé de vérification, sur la base des documents en entrée. Ces rapports sont partitionnés pour la clarté et la commodité et doivent suivre le Plan de Vérification du Logiciel. L'exigence 6.2.4.13 se rapporte aux Rapports de Vérification du Logiciel.

6.2.4.13 Chaque Rapport de Vérification du Logiciel doit documenter ce qui suit :

a) l'identité et la configuration des éléments vérifiés, ainsi que les noms des Chargés de vérification;

b) les éléments qui ne sont pas conformes aux spécifications;

c) les composants, les données, les structures et les algorithmes mal adaptés au problème;

d) les erreurs ou déficiences détectées;

e) le respect du Plan de Vérification du Logiciel ou l'écart par rapport à celui-ci (en cas d'écart le Rapport de Vérification doit expliquer s'il est critique ou non);

f) les hypothèses éventuelles;

g) un récapitulatif des résultats de vérification.

6.3 Validation du logiciel

6.3.1 Objet

6.3.1.1 L'objectif de la validation du logiciel est de démontrer que les processus et leurs sorties sont tels que le logiciel a le niveau défini d'intégrité de sécurité logicielle, qu'il satisfait aux exigences relatives au logiciel et qu'il est adapté à l'application prévue. Cette activité est exécutée par le Chargé de validation.

6.3.1.2 Les principales activités de validation sont de démontrer par l'analyse et/ou des tests que toutes les exigences relatives au logiciel sont spécifiées, testées et satisfaites comme requis par le niveau de SIL applicable et d'évaluer la criticité pour la sécurité de toutes les anomalies et non-conformités en se basant sur les résultats des revues, des analyses et des tests.

6.3.2 Documents en entrée

L'ensemble de la documentation du système, du matériel et du logiciel telle que spécifiée dans la présente Norme européenne.

6.3.3 Documents en sortie

1) Plan de Validation du Logiciel

2) Rapport de Validation du Logiciel

3) Rapport de Vérification de la Validation du Logiciel

6.3.4 Exigences

6.3.4.1 Les activités de Validation du Logiciel doivent être développées et exécutées, avec leurs résultats évalués, par un Chargé de validation ayant un niveau approprié d'indépendance tel que défini en 5.1.

6.3.4.2 La validation doit être documentée avec, au moins, un Plan de Validation du Logiciel et un Rapport de Validation du Logiciel, tels que définis ci-après.

6.3.4.3 Un Plan de Validation du Logiciel doit être rédigé, sous la responsabilité du Chargé de validation, sur la base des documents en entrée.

Les exigences de 6.3.4.4 à 6.3.4.6 se rapportent au Plan de Validation du Logiciel.

6.3.4.4 Le Plan de Validation du Logiciel doit inclure un résumé justifiant le choix de la stratégie de validation. La justification doit inclure, selon le niveau requis d'intégrité de la sécurité logicielle, la prise en considération :

a) de techniques manuelles et/ou automatisées;

b) de techniques statiques et/ou dynamiques;

c) de techniques analytiques et/ou statistiques;

d) des tests dans un environnement réel et/ou simulé.

6.3.4.5 Le Plan de Validation du Logiciel doit identifier les étapes nécessaires pour démontrer l'adéquation de toute Spécification de Logiciel pour satisfaire aux exigences de sécurité exposées dans la Spécification des exigences de sécurité du système.

6.3.4.6 Le Plan de Validation du Logiciel doit identifier les étapes nécessaires pour démontrer l'adéquation de la Spécification de Tests d'Ensemble du Logiciel en tant que test par rapport à la Spécification des Exigences du logiciel.

6.3.4.7 Un Rapport de Validation du Logiciel doit être rédigé, sous la responsabilité du Chargé de validation, sur la base des documents en entrée.

Les exigences de 6.3.4.8 à 6.3.4.11 se rapportent au Rapport de Validation du Logiciel.

6.3.4.8 Les résultats de la validation doivent être documentés dans le Rapport de Validation du Logiciel.

6.3.4.9 Le chargé de validation doit s'assurer que le processus de vérification est complet.

6.3.4.10 Le Rapport de Validation du Logiciel doit indiquer de manière exhaustive la configuration du logiciel qui a été validé.

6.3.4.11 Le Rapport de Validation doit clairement identifier toutes les éventuelles déficiences connues du logiciel et l'impact qu'elles peuvent avoir sur l'utilisation du logiciel.

6.3.4.12 Un Rapport de Vérification de la Validation du Logiciel doit être rédigé, sous la responsabilité du Chargé de vérification, sur la base des documents en entrée de 6.3.2.

Les exigences de 6.3.4.13 à 6.3.4.14 se rapportent au Rapport de Vérification de la Validation du Logiciel.

6.3.4.13 Une fois que le Plan de Validation du Logiciel a été défini, la vérification doit porter sur :

a) le fait que le Plan de Validation du Logiciel satisfait aux exigences générales de lisibilité et de traçabilité de 5.3.2.7 à 5.3.2.10 et de 6.5.4.14 à 6.5.4.17 ainsi qu'aux exigences spécifiques dans 6.3.4.4 à 6.3.4.6,

b) la cohérence interne du Plan de Validation du Logiciel.

6.3.4.14 Une fois que le Rapport de Validation du Logiciel a été défini, la vérification doit porter sur :

a) le fait que le Rapport de Validation du Logiciel satisfait aux exigences générales de lisibilité et de traçabilité de 5.3.2.7 à 5.3.2.10 et de 6.5.4.14 à 6.5.4.17 ainsi qu'aux exigences spécifiques dans 6.3.4.8 à 6.3.4.11 et dans 7.7.4.7 à 7.7.4.11,

b) la cohérence interne du Rapport de Validation du Logiciel.

Les résultats doivent être enregistrés dans un Rapport de Vérification de la Validation du Logiciel.

6.3.4.15 Le Chargé de validation doit être habilité à exiger ou exécuter des revues, analyses et tests supplémentaires.

6.3.4.16 Le logiciel ne doit être diffusé pour l'exploitation qu'après autorisation par le Chargé de validation.

6.3.4.17 Il est permis d'utiliser la simulation et la modélisation pour compléter le processus de validation.

6.4 Évaluation du logiciel

6.4.1 Objet

6.4.1.1 Évaluer que les processus du cycle de vie et leurs sorties sont tels que le logiciel est conforme aux niveaux définis d'intégrité de la sécurité logicielle 1 à 4 et qu'il est adapté à son application prévue.

6.4.1.2 Pour les logiciels SIL 0, les exigences de la présente Norme doivent être satisfaites, mais lorsqu'un certificat énonçant la conformité à l'EN ISO 9001 est disponible, aucune évaluation ne sera requise.

6.4.2 Documents en entrée

1) Spécification des Exigences de Sécurité du Système

2) Spécification des Exigences du Logiciel

3) Tous les autres documents nécessaires pour mener à bien le processus d'évaluation

6.4.3 Documents en sortie

1) Plan d'Évaluation du Logiciel

2) Rapport d'Évaluation du Logiciel

3) Rapport de Vérification de l'Évaluation du Logiciel

6.4.4 Exigences

6.4.4.1 L'évaluation du logiciel doit être menée à bien par un Chargé d'évaluation qui est indépendant comme décrit en 5.1.2.6 et en 5.1.2.7

6.4.4.2 Un logiciel muni d'un Rapport d'Évaluation de Logiciel délivré par un autre Chargé d'évaluation peut ne pas faire l'objet d'une nouvelle évaluation. Le Chargé d'évaluation doit s'assurer que le logiciel est adapté à son utilisation prévue dans l'environnement prévu et que l'évaluation précédente énonçait que le logiciel a atteint un niveau d'intégrité de sécurité au moins égal au niveau requis.

6.4.4.3 Le Chargé d'évaluation doit avoir accès à l'ensemble de la documentation associée au projet tout au long du processus de développement.

6.4.4.4 Un Plan d'Évaluation du Logiciel doit être rédigé, sous la responsabilité du Chargé d'évaluation, sur la base des documents en entrée de 6.4.2. Lorsque cela est approprié, il est permis d'utiliser une procédure ou Plan d'évaluation de logiciel générique documenté(e) existant(e). L'exigence en 6.4.4.5 se rapporte au Plan d'Évaluation du Logiciel.

6.4.4.5 Le Plan d'Évaluation du Logiciel doit comporter le périmètre suivant :

a) les aspects dont traite l'évaluation;

b) les activités tout au long du processus d'évaluation et leur lien séquentiel aux activités d'ingénierie;

c) les documents à prendre en considération;

d) les déclarations des critères d'acceptation et de rejet et la manière de traiter des cas de non-conformité;

e) les exigences relatives au contenu et à la forme du Rapport d'Évaluation du Logiciel.

6.4.4.6 Un Rapport de Vérification de l'Évaluation du Logiciel doit être rédigé, sous la responsabilité du Chargé de vérification, sur la base des documents en entrée de 6.4.2.

L'exigence en 6.4.4.7 se rapporte au Rapport de Vérification de l'Évaluation du Logiciel.

6.4.4.7 Une fois que le Plan d'Évaluation du Logiciel a été défini, la vérification doit porter sur :

a) le fait que le Plan d'Évaluation du Logiciel satisfait aux exigences générales de lisibilité et de traçabilité de 5.3.2.7 à 5.3.2.10 et de 6.5.4.14 à 6.5.4.17 ainsi qu'aux exigences spécifiques dans 6.4.4.5,

b) la cohérence interne du Plan d'Évaluation du Logiciel.

Les résultats doivent être enregistrés dans un Rapport de Vérification de l'évaluation du Logiciel.

6.4.4.8 Le Chargé d'évaluation doit évaluer si le logiciel du système est adapté à l'objectif prévu et s'il répond correctement aux problèmes de sécurité dérivés de la Spécification des Exigences de Sécurité du Système.

6.4.4.9 Le Chargé d'évaluation doit évaluer si un ensemble approprié de techniques issues de l'Annexe A, qui sont adaptées pour le développement prévu, a été sélectionné et appliqué en fonction du niveau requis d'intégrité de sécurité.

En outre, le Chargé d'évaluation doit tenir compte de la mesure dans laquelle chaque technique issue de l'Annexe A est appliquée, c'est-à-dire si elle s'applique à la totalité ou seulement à une partie du logiciel, et également rechercher des preuves indiquant qu'elle est appliquée correctement.

6.4.4.10 Le Chargé d'évaluation doit évaluer le système de gestion de la configuration et des modifications ainsi que les preuves de son utilisation et son application.

6.4.4.11 Le Chargé d'évaluation doit examiner la preuve de la compétence de l'équipe de projet selon l'Annexe B et doit évaluer l'organisation quant au développement du logiciel selon 5.1.

6.4.4.12 Pour tout logiciel contenant des conditions d'application relatives à la sécurité, le Chargé d'évaluation doit rechercher les écarts consignés, les non-conformités aux exigences et les non-conformités enregistrées si celles-ci ont un impact sur la sécurité et juger si la justification fournie par le projet est acceptable. Le résultat doit être consigné dans le rapport d'évaluation.

6.4.4.13 Le Chargé d'évaluation doit évaluer les activités de vérification et de validation et les pièces justificatives.

6.4.4.14 Le Chargé d'évaluation doit donner son accord sur le périmètre et le contenu du Plan de Validation du Logiciel. Cet accord doit également stipuler la présence du Chargé d'évaluation pendant les tests.

6.4.4.15 Le Chargé d'évaluation peut conduire des audits et inspections (par exemple : présence lors des tests) tout au long du processus de développement entier. Le Chargé d'évaluation peut demander un travail supplémentaire de vérification et de validation.

NOTE Il est avantageux d'impliquer le Chargé d'évaluation à un stade précoce du projet.

6.4.4.16 Un Rapport d'Évaluation du Logiciel doit être rédigé sous la responsabilité du Chargé d'évaluation. Les exigences de 6.4.4.17 à 6.4.4.19 se rapportent au Rapport d'Évaluation du Logiciel.

6.4.4.17 Le Rapport d'Évaluation du Logiciel doit satisfaire aux exigences du Plan d'Évaluation du Logiciel et fournir une conclusion et des recommandations.

6.4.4.18 Le Chargé d'évaluation doit enregistrer ses activités comme une base cohérente pour le Rapport d'Évaluation du Logiciel. Celles-ci doivent être résumées dans le Rapport d'Évaluation du Logiciel.

6.4.4.19 Le Chargé d'évaluation doit identifier et évaluer toute non-conformité aux exigences de la présente Norme européenne et juger l'impact sur le résultat final. Ces non-conformités et le jugement à leur sujet doivent être énumérés dans le Rapport d'Évaluation du Logiciel.

6.5 Assurance Qualité du Logiciel

6.5.1 Objectifs

6.5.1.1 Identifier, superviser et contrôler toutes les activités, à la fois techniques et de gestion, qui sont nécessaires pour assurer que le logiciel possède la qualité requise. Il est nécessaire de fournir la démarche qualitative requise contre les défauts systématiques et de s'assurer qu'une trace puisse être laissée afin de permettre un déroulement efficace des activités de vérification et de validation.

6.5.1.2 Apporter la preuve que toutes les activités ci-dessus ont été menées à bien.

6.5.2 Documents en entrée

Tous les documents disponibles à chaque étape du cycle de vie.

6.5.3 Documents en sortie

1) Plan d'Assurance Qualité du Logiciel

2) Plan de Gestion de la Configuration du Logiciel, s'il n'est pas disponible au niveau du système

3) Rapport de vérification concernant l'Assurance Qualité du logiciel

6.5.4 Exigences

6.5.4.1 Tous les plans doivent être communiqués au commencement du projet et mis à jour pendant le cycle de vie.

6.5.4.2 Les organisations prenant part au développement du logiciel doivent mettre en œuvre et utiliser un Système d'Assurance Qualité conforme à l'EN ISO 9000, pour satisfaire aux exigences de la présente Norme européenne. La certification EN ISO 9001 est hautement recommandée.

6.5.4.3 Un Plan de Vérification d'Assurance Qualité du Logiciel doit être rédigé, sous la responsabilité du Chargé de vérification, sur la base des documents en entrée de 6.5.2.

Les exigences de 6.5.4.4 à 6.5.4.6 se rapportent au Plan d'Assurance Qualité du Logiciel.

6.5.4.4 Un Plan d'Assurance Qualité du Logiciel doit être rédigé et doit être spécifique au projet. Il doit mettre en œuvre les exigences de 6.5.4.5.

6.5.4.5 Pour le moins, les éléments suivants doivent être spécifiés ou référencés dans le Plan d'Assurance Qualité du Logiciel.

a) Définition du modèle de cycle de vie

   1) activités et tâches élémentaires compatibles avec les plans (par exemple : Plan de Sécurité) qui ont été établis au niveau Système;

   2) critères d'entrée et de sortie de chaque activité;

   3) entrées et sorties de chaque activité;

   4) principales activités qualité;

   5) l'entité responsable de chaque activité.

b) Structure de la documentation.

c) Contrôle de la documentation :

   1) rôles impliqués dans la rédaction, le contrôle et l'approbation;

   2) étendue de la distribution;

   3) archivage.

d) Suivi et traçabilité des écarts.

e) Méthodes, mesures et outils pour l'assurance qualité en fonction des niveaux alloués d'intégrité de la sécurité (voir Annexe A).

f) Justifications, telles que définies de 4.7 à 4.9, que chaque combinaison de techniques ou de mesures sélectionnées conformément à l'Annexe A est appropriée au niveau défini d'intégrité de la sécurité logicielle.

Certaines des informations requises du Plan d'Assurance Qualité du Logiciel peuvent être contenues dans d'autres documents, tels qu'un Plan de Gestion de la Configuration de Logiciel séparé, un Plan de Maintenance, un plan de Vérification du Logiciel et un Plan de Validation du Logiciel. Les paragraphes du Plan d'Assurance Qualité du Logiciel doivent référencer les documents qui contiennent les informations. Dans tous les cas, le contenu de chaque paragraphe du Plan d'Assurance Qualité du Logiciel doit être spécifié, directement ou par référence à un autre document.

Les documents référencés doivent être revus afin de s'assurer qu'ils fournissent toutes les informations requises et qu'ils répondent complètement aux exigences de la présente Norme européenne.

6.5.4.6 Les activités, actions, documents, etc. de l'assurance qualité requis par tous les paragraphes normatifs de la présente Norme européenne doivent être spécifiés ou référencés dans le Plan d'Assurance Qualité du Logiciel et adaptés en fonction du projet spécifique.

6.5.4.7 Un Rapport de Vérification d'Assurance Qualité du Logiciel doit être rédigé, sous la responsabilité du Chargé de vérification, sur la base des documents en entrée de 6.5.2.

L'exigence en 6.5.4.8 se rapporte au Rapport de Vérification d'Assurance Qualité du Logiciel.

6.5.4.8 Une fois que le Plan d'Assurance Qualité du Logiciel a été défini, la vérification doit porter sur :

a) le fait que le Plan d'Assurance Qualité du Logiciel satisfait aux exigences générales de lisibilité et de traçabilité de 5.3.2.7 à 5.3.2.10 et de 6.5.4.14 à 6.5.4.17 ainsi qu'aux exigences spécifiques dans 6.5.4.4 à 6.5.4.6,

b) la cohérence interne du Plan d'Assurance Qualité du Logiciel.

Les résultats doivent être enregistrés dans un Rapport de Vérification d'Assurance Qualité du Logiciel.

6.5.4.9 Chaque document de planification doit comporter un alinéa spécifiant les détails relatifs à sa propre mise à jour tout au long du projet : fréquence, responsabilité, méthode.

6.5.4.10 Chaque document du logiciel et chaque produit à fournir doivent être soumis au contrôle de configuration à partir du moment de sa première diffusion.

6.5.4.11 Les modifications apportées à tous les éléments sous le contrôle de la Gestion de Configuration doivent être autorisées et enregistrées.

6.5.4.12 En plus du développement de logiciel, le Système de Gestion de Configuration doit aussi couvrir l'environnement de développement du logiciel utilisé pendant le cycle de vie complet

Cette extension, nécessaire pour la reproductibilité du développement et pour les activités de maintenance, doit concerner tous les outils, traducteurs, fichiers de données et de tests, fichiers de paramétrage et plateformes matérielles supportant ces derniers.

6.5.4.13 Le fournisseur doit établir, documenter et maintenir des procédures pour le contrôle des fournisseurs externes, notamment

- des méthodes et des enregistrements significatifs visant à assurer que les logiciels fournis par des fournisseurs externes respectent les exigences définies. Les logiciels développés préalablement doivent

être garantis conformes au niveau requis d'intégrité de la sécurité logicielle et de sûreté de fonctionnement. Les nouveaux logiciels doivent être développés et maintenus en conformité avec le Plan d'Assurance Qualité du Logiciel du fournisseur ou avec un Plan d'Assurance Qualité du Logiciel spécifique, préparé par le fournisseur externe en accord avec le Plan d'Assurance Qualité Logiciel du fournisseur,

- des méthodes et enregistrements correspondants visant à assurer que les exigences transmises au fournisseur externe sont adéquates et complètes.

6.5.4.14 La traçabilité des exigences doit être largement prise en considération dans la validation d'un système de sécurité et des moyens doivent être fournis pour permettre de le démontrer d'un bout à l'autre de toutes les phases du cycle de vie.

6.5.4.15 Dans le contexte de la présente Norme européenne et dans la limite appropriée au niveau spécifié d'intégrité de la sécurité logicielle, la traçabilité doit concerner particulièrement :

a) la traçabilité des exigences par rapport à la conception ou aux autres objets qui y répondent,

b) la traçabilité des objets de la conception vers les objets de la réalisation qui les instancient,

c) la traçabilité des exigences et des objets de conception vers les tests (composant, intégration, test d'ensemble) et les analyses qui les vérifient.

La traçabilité doit être soumise à la Gestion de Configuration.

6.5.4.16 Dans des cas particuliers (par exemple celui des logiciels préexistants ou logiciels prototypés), la traçabilité peut être établie après la mise en œuvre et/ou la documentation du code, mais avant la vérification/validation. Dans ces cas, il doit être démontré que la vérification/validation est aussi efficace qu'elle l'aurait été avec la traçabilité sur toutes les phases.

6.5.4.17 Il doit être démontré que les objets de spécification des exigences, de conception ou de réalisation qui ne peuvent pas être correctement tracés n'ont aucune influence sur la sécurité ou l'intégrité du système.

6.6 Contrôle des modifications et des évolutions

6.6.1 Objectifs

6.6.1.1 Assurer que le logiciel fonctionne comme requis, en préservant l'intégrité de la sécurité logicielle et la sûreté de fonctionnement lorsqu'on procède à des modifications du logiciel.

6.6.1.2 Ces objectifs sont gérés par le Gestionnaire de la Configuration.

6.6.2 Documents en entrée

1) Plan d'Assurance Qualité du Logiciel

2) Plan de Gestion de la Configuration du Logiciel

3) Tout document approprié de conception, de réalisation et d'analyse

4) Demandes de modification

5) Analyse de l'impact des modifications et autorisation des modifications

6.6.3 Documents en sortie

1) Tous les documents en entrée modifiés

2) Registres des modifications du logiciel (voir 9.2.4.11)

3) Enregistrements de la nouvelle Configuration

6.6.4 Exigences

6.6.4.1 Le Processus de Gestion des Modifications doit définir au moins les aspects suivants :

a) la documentation nécessaire pour le compte-rendu des problèmes et/ou les actions correctives, dans le but de permettre le retour d'informations vers la direction responsable;

b) l'analyse des informations recueillies dans les comptes-rendus de problèmes pour en identifier les causes;

c) les pratiques à suivre pour le compte rendu, le suivi et la résolution des problèmes identifiés à la fois au cours de la phase de développement et au cours de la phase de maintenance du logiciel;

d) les responsabilités organisationnelles spécifiques par rapport au développement et à la maintenance du logiciel;

e) la manière d'appliquer des contrôles pour s'assurer que des actions correctives sont prises et qu'elles sont efficaces;

f) l'analyse d'impact de l'effet des modifications sur le composant de logiciel en développement ou déjà livré;

g) l'analyse d'impact doit énoncer la revérification, la revalidation et la réévaluation nécessaires pour le changement;

h) lorsque plusieurs modifications sont apportées, l'analyse d'impact doit envisager l'impact cumulé;

NOTE Plusieurs modifications peuvent nécessiter cumulativement un nouvel essai complet.

i) l'autorisation avant la réalisation.

6.6.4.2 Toutes les modifications doivent déclencher un retour à une phase appropriée du cycle de vie. Toutes les phases ultérieures doivent être menées à bien selon les procédures spécifiées pour les phases spécifiques en conformité avec les exigences de la présente Norme européenne.

6.7 Outils et langages

6.7.1 Objectifs

6.7.1.1 L'objectif est de fournir la preuve que les défaillances potentielles des outils ne nuisent pas à la sécurité des données produites par l'ensemble des outils sans que cela ne soit détecté par des mesures techniques et/ou organisationnelles extérieures à l'outil. Pour ce faire, les outils logiciels sont catégorisés en trois classes, à savoir T1, T2 et T3 (voir les définitions en 3.1).

Lorsque des outils sont utilisés en remplacement d'opérations manuelles, la preuve de l'intégrité de la sortie du jeu d'outils peut être présentée par les mêmes étapes de processus que si la sortie était produite dans une opération manuelle. Ces étapes de processus pourraient être remplacées par des méthodes en variante si une argumentation sur l'intégrité de la sortie des outils est donnée et le niveau d'intégrité du logiciel n'est pas diminué par le remplacement.

6.7.2 Documents en entrée

Spécification des outils ou manuel.

6.7.3 Documents en sortie

Rapport de validation des outils (si cela s'avère nécessaire, voir 6.7.4.4 ou 6.7.4.6)

6.7.4 Exigences

6.7.4.1 Les outils logiciels doivent être choisis comme partie cohérente des activités de développement du logiciel.

NOTE Il convient d'utiliser des outils appropriés d'aide au développement du logiciel afin d'augmenter l'intégrité du logiciel en réduisant la probabilité d'introduction ou de non-détection de défauts au cours du développement. Les exemples d'outils pertinents pour les phases de cycle de vie de développement du logiciel sont notamment

a) les outils de transformation ou de traduction qui convertissent une représentation de logiciel ou de conception (par exemple : un texte ou un schéma) d'un niveau d'abstraction en un autre les outils d'affinement de la conception, les compilateurs, les assembleurs, les éditeurs de liens, les chargeurs et les outils de génération de code,

b) les outils de vérification et de validation tels que les logiciels d'analyse statique, les contrôleurs de couverture de test, les aides de démonstration de théorèmes, les simulateurs et les vérificateurs de modèle,

c) les outils de diagnostic utilisés pour maintenir et surveiller le logiciel dans des conditions de fonctionnement,

d) les outils d'infrastructure tels que les systèmes d'aide au développement,

e) les outils de contrôle de configuration tels que les outils de contrôles des versions,

f) les outils de données d'application qui produisent ou maintiennent les données qui sont requises pour définir les paramètres et instancier les fonctions du système, par exemple, paramètres de fonction, gammes d'instruments, niveaux d'alarme et de déclenchement, états de sortie à adopter à la défaillance, disposition géographique.

Il convient que les outils sélectionnés soient capables de coopérer. Dans ce contexte, des outils coopèrent si les grandeurs de sortie d'un outil ont le contenu et le format adéquats pour être injectées automatiquement en entrée dans un outil suivant, réduisant ainsi au maximum la possibilité d'introduire une erreur humaine dans la manipulation des résultats intermédiaires.

Les outils doivent être choisis et prouvés compatibles avec les besoins de l'application.

La disponibilité des outils adéquats pour fournir les services nécessaires tout au long de la durée de vie du logiciel doit être prise en considération.

6.7.4.2 Le choix des outils des classes T2 et T3 doit être justifié (voir 7.3.4.12). La justification doit inclure l'identification de défaillances potentielles qui peuvent être injectées dans la sortie des outils et les mesures pour éviter ou gérer ces défaillances.

6.7.4.3 Tous les outils des classes T2 et T3 doivent avoir une spécification ou un manuel qui définit le comportement de l'outil et les éventuelles instructions ou contraintes concernant son utilisation.

6.7.4.4 Pour chaque outil de la classe T3, la preuve que sa sortie est conforme à la spécification de la sortie ou que ses défaillances sont détectées doit être disponible. La preuve peut être fondée sur les mêmes justifications qui seraent fournies pour un processus manuel mise en œuvre à la place de l'outil ou sur un argumentaire si des types de preuve différents sont possibles (par exemple : validation de l'outil). La preuve peut aussi être basée sur

a) une combinaison appropriée d'utilisations antérieures réussies dans des environnements similaires et pour des applications similaires (au sein de l'organisation ou d'autres organisations),

b) la validation d'outil telle que spécifiée en 6.7.4.5,

c) divers codes redondants qui permettent la détection et la maîtrise des défaillances provoquant l'introduction de défauts par un outil,

d) la conformité aux niveaux d'intégrité de sécurité dérivés de l'analyse de risque appliquée au processus et aux procédures, y compris les outils,

e) d'autres méthodes appropriées pour éviter ou gérer les défaillances introduites par des outils.

NOTE 1 Un historique des versions peut apporter l'assurance de la maturité de l'outil et un enregistrement des erreurs/ambiguïtés associées à son utilisation dans l'environnement.

NOTE 2 La preuve énumérée pour T3 peut aussi servir pour les outils T2 pour juger du caractère correct des résultats.

6.7.4.5 Les résultats de la validation d'outils doivent être documentés et couvrir les résultats suivants :

a) un enregistrement des activités de validation;

b) la version du manuel d'outils utilisé;

c) les fonctions de l'outil validées;

d) les outils et le matériel utilisés;

e) les résultats des activités de validation; les résultats documentés de la validation doivent indiquer soit que le logiciel a passé avec succès la validation, soit les raisons de son rejet;

f) les scénarios de test et leurs résultats, en vue d'une analyse ultérieure;

g) les divergences entre les résultats attendus et les résultats réels.

6.7.4.6 Lorsque la preuve de la conformité en 6.7.4.4 n'est pas disponible, des mesures effectives doivent exister pour maîtriser les défaillances du logiciel exécutable de sécurité qui résultent de défauts imputables à l'outil.

NOTE 1 Un exemple en est la génération de codes redondants divers qui permettent la détection et la maîtrise de défaillances provoquant l'introduction de défauts par un traducteur.

NOTE 2 À titre d'exemple, l'adéquation aux besoins d'un compilateur non certifié peut être justifiée comme suit.

Le code objet produit par le compilateur a été soumis à une combinaison de tests, de contrôles et d'analyses qui sont capables d'assurer le caractère correct du code dans la mesure où il est compatible avec le niveau visé d'intégrité de sécurité. En particulier, les points ci-après s'appliquent à tous les tests, contrôles et analyses.

- Il a été montré que les tests avaient une couverture suffisamment élevée du code mis en œuvre. S'il existe un code inaccessible par des tests, il a été montré par des contrôles ou des analyses que la fonction concernée s'exécutait correctement lorsque le code était atteint sur la cible.
- Des contrôles et analyses ont été appliqués au code objet et se sont avérés capables de détecter les types d'erreurs susceptibles de résulter d'un défaut du compilateur.
- Plus aucune traduction avec le compilateur n'a eu lieu après les tests, le contrôle et l'analyse.
- Si une compilation ou traduction supplémentaire est effectuée, tous les tests, contrôles et analyses seront répétés.

6.7.4.7 La représentation du logiciel ou de la conception (y compris le langage de programmation) choisie doit

a) avoir un traducteur qui a été évalué quant à son adéquation aux besoins, y compris, le cas échéant, l'évaluation par rapport aux normes internationales ou nationales,

b) être compatible avec les caractéristiques de l'application,

c) contenir des caractéristiques qui facilitent la détection des erreurs de conception ou de programmation,

d) prendre en charge des caractéristiques qui sont compatibles avec la méthode de conception.

Un langage de programmation est l'une des classes de représentation de logiciel ou de conception. Un Traducteur convertit une représentation de logiciel ou de conception (par exemple : un texte ou un schéma) d'un niveau d'abstraction en un autre. Les exemples de Traducteurs sont notamment les outils d'affinement de la conception, les compilateurs, les assembleurs, les éditeurs de liens, les chargeurs et les outils de génération de code.

L'évaluation d'un Traducteur peut être effectuée pour un projet d'application spécifique ou pour une classe d'applications. Dans le second cas, toutes les informations nécessaires sur l'outil concernant son utilisation prévue et appropriée doivent être mises à la disposition de l'utilisateur de l'outil. L'évaluation de l'outil pour un projet spécifique peut alors se réduire à une vérification de l'adéquation générale de l'outil au projet et de la conformité à la «spécification ou manuel» (c'est-à-dire l'utilisation correcte de l'outil). L'utilisation correcte pourrait inclure des activités de vérification supplémentaires dans les limites du projet spécifique.

Un dispositif de validation peut être utilisé pour évaluer l'adéquation aux besoins d'un traducteur en fonction de critères définis, qui doivent inclure des exigences fonctionnelles et non fonctionnelles. Pour les exigences fonctionnelles du traducteur, les tests dynamiques peuvent constituer la technique de validation principale. Si cela est possible, une série de tests automatiques doit être utilisée.

6.7.4.8 Lorsque 6.7.4.7 ne peut pas être complètement satisfait, l'adéquation aux besoins du langage et toute mesure complémentaire traitant des éventuelles insuffisances identifiées du langage doivent être justifiées et évaluées.

NOTE Voir NOTE 2 en 6.7.4.6.

6.7.4.9 Lorsqu'une génération automatique de code ou une traduction automatique similaire a lieu, l'adéquation du Traducteur automatique au développement de logiciel de sécurité doit être évaluée à l'endroit du cycle de vie de développement auquel les outils au développement sont choisis.

6.7.4.10 La gestion de configuration doit assurer que pour les outils des classes T2 et T3, seules des versions justifiées sont utilisées.

6.7.4.11 Chaque nouvelle version d'un outil utilisé doit être justifiée (voir Tableau 1). Cette justification peut reposer sur une preuve fournie pour une version antérieure s'il est fourni des preuves suffisantes que

a) les différences fonctionnelles (éventuelles) n'affecteront pas la compatibilité de l'outil avec le reste du jeu d'outils,

b) la nouvelle version n'est pas susceptible de contenir de nouveaux défauts inconnus significatifs.

NOTE La preuve que la nouvelle version n'est pas susceptible de contenir de nouveaux défauts inconnus significatifs peut reposer sur une identification crédible des changements apportés et sur une analyse des actions de vérification et de validation exécutées.

6.7.4.12 La relation entre les classes d'outils et les paragraphes applicables est définie au Tableau 1.

Tableau 1 - Relation entre les classe d'outils et les paragraphes applicables

Classe d'outils	Paragraphes applicables
T1	6.7.4.1
T2	6.7.4.1, 6.7.4.2, 6.7.4.3, 6.7.4.10, 6.7.4.11
T3	6.7.4.1, 6.7.4.2, 6.7.4.3, 6.7.4.4, 6.7.4.5 ou 6.7.4.6, 6.7.4.7, 6.7.4.8, 6.7.4.9, 6.7.4.10, 6.7.4.11

7 Développement de logiciel générique

7.1 Cycle de vie et documentation pour logiciel générique

7.1.1 Objectifs

7.1.1.1 Fournir une description du logiciel lui-même, depuis les niveaux supérieurs d'abstraction jusqu'à l'affinement des détails, afin de créer un cadre pour la démonstration de la sécurité obtenue ainsi que pour les actions de maintenance futures.

7.1.2 Exigences

7.1.2.1 Dans la limite requise par le niveau d'intégrité de sécurité logicielle, les documents énumérés au Tableau A.1 doivent être produits pour un logiciel générique.

7.1.2.2 La séquence de documents à fournir tels qu'ils sont décrits au Tableau A.1 reflète un modèle de cascade linéaire idéal. Ce modèle n'est toutefois pas destiné à servir de référence dans le sens de programmation et de liaison d'activités, car il serait difficile d'obtenir une stricte conformité dans la pratique. Des phases peuvent se chevaucher, mais les activités de vérification et de validation doivent démontrer la cohérence des entrées et des sorties (documents et logiciels) au sein des phases et entre les phases.

Cependant, le but principal de la documentation prévue est de fournir une description du logiciel lui-même, depuis les niveaux supérieurs d'abstraction jusqu'à l'affinement des détails, afin de créer un cadre pour la démonstration de la sécurité obtenue ainsi que pour les actions de maintenance futures.

7.2 Exigences relatives au logiciel

7.2.1 Objectifs

7.2.1.1 Décrire un jeu complet d'exigences pour les logiciels respectant les exigences relatives au système et à la sécurité et fournir un jeu complet de documents pour chaque phase ultérieure.

7.2.1.2 Décrire la Spécification de Tests d'Ensemble du Logiciel.

7.2.2 Documents en entrée

1) Spécification des Exigences du Système
2) Spécification des Exigences de Sécurité du Système
3) Description de l'Architecture du Système
4) Spécifications d'interface externe (par exemple : Spécification d'interface Logiciel/Logiciel, Spécification d'interface Logiciel/Matériel)
5) Plan d'Assurance Qualité du Logiciel
6) Plan de Validation du Logiciel

7.2.3 Documents en sortie

1) Spécification des Exigences du Logiciel
2) Spécification de Tests d'Ensemble du Logiciel
3) Rapport de Vérification des Exigences du Logiciel

7.2.4 Exigences

7.2.4.1 Une Spécification d'exigences du Logiciel doit être rédigée, sous la responsabilité du Gestionnaires des Exigences, sur la base des documents en entrée de 7.2.2.

Les exigences de 7.2.4.2 à 7.2.4.15 se rapportent à la Spécification des Exigences du Logiciel.

7.2.4.2 La Spécification des Exigences du Logiciel doit exprimer les propriétés requises du logiciel développé. Ces propriétés, qui sont toutes (à l'exception de la sécurité) définies dans la série ISO/CEI 9126, doivent inclure

a) la fonctionnalité (y compris le dimensionnement et la performance en temps de réponse),

b) la robustesse et la maintenabilité,

c) la sécurité (y compris les fonctions de sécurité et les niveaux d'intégrité de la sécurité logicielle qui leur sont associés),

d) l'efficacité,

e) l'utilisabilité,

f) la portabilité.

7.2.4.3 Le niveau d'intégrité de la sécurité logicielle doit être dérivé comme indiqué dans l'Article 4 et enregistré dans la Spécification des Exigences du Logiciel.

7.2.4.4 Dans la limite requise par le niveau d'intégrité de la sécurité logicielle, la Spécification des Exigences du Logiciel doit être exprimée et structurée de manière à être

a) complète, claire, précise, sans équivoque, susceptible d'être vérifiée, testée, maintenue et réalisée,

b) traçable en remontant à tous les documents en entrée.

7.2.4.5 La Spécification des Exigences du Logiciel doit inclure des modes d'expression et des descriptions qui sont compréhensibles par le personnel responsable impliqué dans le cycle de vie du logiciel.

7.2.4.6 La Spécification des Exigences du Logiciel doit identifier et documenter toutes les interfaces avec tout autre système, qu'il se trouve à l'intérieur ou à l'extérieur de l'équipement sous contrôle, y compris les opérateurs, chaque fois qu'une connexion directe existe ou qu'elle est prévue.

7.2.4.7 Tous les modes de fonctionnement applicables doivent être détaillés dans la Spécification des Exigences du Logiciel.

7.2.4.8 Tous les modes de comportement applicables des systèmes électroniques programmables, notamment le comportement de défaillance, doivent être documentés ou référencés (par exemple : documentation au niveau système) dans la Spécification des Exigences du Logiciel.

7.2.4.9 Toute contrainte entre le matériel et le logiciel doit être documentée ou référencée (par exemple : documentation au niveau système) dans la Spécification des Exigences du Logiciel.

7.2.4.10 Dans la limite requise par la description de la documentation du système, la Spécification des Exigences du Logiciel doit envisager l'autocontrôle logiciel et le contrôle du matériel par le logiciel. L'autocontrôle logiciel consiste en la détection et le compte-rendu par le logiciel de ses propres défaillances et erreurs.

7.2.4.11 La Spécification des Exigences du Logiciel doit inclure des exigences concernant le test périodique des fonctions, dans la limite requise par la Spécification des Exigences de Sécurité du Système.

7.2.4.12 La Spécification des Exigences du Logiciel doit inclure des exigences permettant la testabilité de toutes les fonctions de sécurité pendant le fonctionnement global du système, dans la limite requise par la Spécification des Exigences de Sécurité du Système.

7.2.4.13 Toutes les fonctions que le logiciel est tenu d'exécuter, notamment celles liées à la réalisation du niveau requis d'intégrité de la sécurité du système, doivent être clairement identifiées dans la Spécification des Exigences du Logiciel.

7.2.4.14 Les éventuelles fonctions non liées à la sécurité que le logiciel est tenu d'exécuter doivent être clairement identifiées dans la Spécification des Exigences du Logiciel.

7.2.4.15 La Spécification des Exigences du Logiciel doit être appuyée par des techniques et des mesures issues du Tableau A.2. La combinaison choisie doit être justifiée comme un ensemble satisfaisant à 4.8 et 4.9.

7.2.4.16 Une Spécification de Tests d'Ensemble du Logiciel doit être rédigée, sous la responsabilité du Chargé des tests, sur la base de la Spécification des Exigences du Logiciel.

Les exigences de 7.2.4.17 à 7.2.4.19 se rapportent à la Spécification de Tests d'Ensemble du Logiciel.

7.2.4.17 La Spécification de Tests d'Ensemble du Logiciel doit être une description des tests à effectuer sur le logiciel complet.

7.2.4.18 La Spécification de Tests d'Ensemble du Logiciel doit choisir des techniques et mesures issues du Tableau A.7. La combinaison choisie doit être justifiée comme un ensemble satisfaisant à 4.8 et 4.9.

7.2.4.19 La Spécification de Tests d'Ensemble du Logiciel doit identifier les scénarios de test pour chacune des fonctions requises, y compris

a) les signaux d'entrée requis avec leurs séquences et leurs valeurs,

b) les signaux de sortie attendus avec leurs séquences et leurs valeurs,

c) les critères de succès aux tests, en incluant les aspects performance et qualité.

7.2.4.20 Un Rapport de Vérification des Exigences du Logiciel doit être rédigé, sous la responsabilité du Chargé de vérification, sur la base de la Spécification des Exigences de Sécurité du Système, de Spécification des Exigences du Logiciel, de la Spécification de Tests d'Ensemble du Logiciel et du Plan d'Assurance Qualité du Logiciel.

Les exigences de 7.2.4.21 à 7.2.4.22 se rapportent au Rapport de Vérification des Exigences du Logiciel.

7.2.4.21 Le Rapport de Vérification des Exigences du Logiciel doit être rédigé conformément aux exigences génériques établies pour tous les Rapports de Vérification (voir 6.2.4.13).

7.2.4.22 Une fois que la Spécification des Exigences du Logiciel a été définie, la vérification doit porter sur :

a) l'adéquation de la Spécification des Exigences du Logiciel pour satisfaire aux exigences exposées dans la Spécification des Exigences du Système, la Spécification des Exigences de Sécurité du Système et le Plan d'Assurance Qualité du Logiciel,

b) le fait que la Spécification des Exigences du Logiciel satisfait aux exigences générales de lisibilité et de traçabilité de 5.3.2.7 à 5.3.2.10 et de 6.5.4.14 à 6.5.4.17 ainsi qu'aux exigences spécifiques dans 7.2.4.2 à 7.2.4.15,

c) l'adéquation de la Spécification de Tests d'Ensemble du Logiciel pour le test par rapport à la Spécification des Exigences du Logiciel,

d) la définition de toute activité supplémentaire afin de démontrer la couverture correcte des exigences qui ne se prêtent pas à des tests,

e) la cohérence interne de la Spécification des Exigences du Logiciel,

f) l'adéquation de la Spécification des Exigences du Logiciel pour respecter ou prendre en compte les contraintes entre matériel et logiciel.

Les résultats doivent être enregistrés dans un Rapport de Vérification des Exigences du Logiciel.

7.3 Architecture et Conception

7.3.1 Objectifs

7.3.1.1 Développer une architecture de logiciel qui satisfait aux exigences du Logiciel.

7.3.1.2 Identifier et évaluer l'importance des interactions matériel/logiciel pour la sécurité.

7.3.1.3 Choisir une méthode de conception si aucune n'a été définie au préalable.

7.3.1.4 Concevoir un logiciel d'un niveau défini d'intégrité de sécurité logicielle à partir des documents en entrée.

7.3.1.5 Assurer que le système résultant et son logiciel se prêteront facilement à des tests depuis le commencement. Puisque la vérification et les tests seront un élément crucial de la validation, un intérêt tout particulier doit être porté aux besoins en matière de vérification et de test tout au long de la réalisation.

7.3.2 Documents en entrée

1) Spécification des Exigences du Logiciel

7.3.3 Documents en sortie

1) Spécification de l'Architecture du Logiciel

2) Spécifications de la Conception du Logiciel

3) Spécification d'Interface du Logiciel

4) Spécification de Tests d'Intégration du Logiciel

5) Spécification de Tests d'Intégration Logiciel/Matériel

6) Rapport de Vérification de l'Architecture et de la Conception du Logiciel

7.3.4 Exigences

7.3.4.1 Une Spécification d'Architecture de Logiciel doit être rédigée, sous la responsabilité du Concepteur, sur la base de la Spécification des Exigences du Logiciel.

Les exigences de 7.3.4.2 à 7.3.4.14 se rapportent à la Spécification de l'Architecture du Logiciel.

7.3.4.2 L'architecture de logiciel proposée doit être établie et détaillée dans la Spécification d'Architecture du Logiciel.

7.3.4.3 La Spécification de l'Architecture du Logiciel doit prendre en considération la faisabilité de la réalisation de la Spécification des Exigences du Logiciel au niveau requis d'intégrité de la sécurité logicielle.

NOTE L'Architecture du Logiciel vise à réduire au maximum la taille et la complexité de la partie sécurité de l'application.

7.3.4.4 La Spécification de l'Architecture du Logiciel doit identifier, analyser et détailler l'importance de toutes les interactions matériel/logiciel.

7.3.4.5 La Spécification de l'Architecture du Logiciel doit identifier tous les composants logiciels et déterminer pour ces composants :

a) si ces composants sont nouveaux ou existants;

b) si ces composants ont été précédemment validés et, si tel est le cas, leurs conditions de validation;

c) le niveau d'intégrité de la sécurité logicielle du composant.

7.3.4.6 Les composants logiciels doivent

a) couvrir un sous-ensemble défini d'exigences relatives au logiciel,

b) être clairement identifiés et constituer des versions indépendantes dans le système de gestion de la configuration.

7.3.4.7 L'utilisation de logiciels préexistants doit être soumise aux restrictions suivantes.

a) Pour tous les niveaux d'intégrité de sécurité logicielle, les informations suivantes doivent être clairement identifiées et documentées :
   - les exigences auxquelles les logiciels préexistants sont censés satisfaire;
   - les hypothèses relatives à l'environnement des logiciels préexistants;
   - les interfaces à d'autres parties du logiciel.

b) Pour tous les niveaux d'intégrité de sécurité logicielle, les logiciels préexistants doivent être inclus dans le processus de validation du logiciel complet.

c) Pour les niveaux d'intégrité de sécurité logicielle SIL 3 et SIL 4, les précautions suivantes doivent être prises :
   - une analyse des défaillances possibles des logiciels préexistants et de leurs conséquences sur le logiciel complet doit être menée à bien;
   - une stratégie doit être définie en vue de détecter les défaillances des logiciels préexistants et de protéger le système contre ces défaillances;
   - le processus de vérification et de validation doit assurer
     1) que les logiciels préexistants satisfont aux exigences allouées,
     2) que les défaillances des logiciels préexistants sont détectées et le système dans lequel les logiciels préexistants sont intégrés est protégé de ces défaillances,
     3) que les hypothèses relatives à l'environnement des logiciels préexistants sont respectées.

d) Les logiciels préexistants doivent être accompagnés d'une description (c'est-à-dire fonctions, contraintes et preuves) suffisamment précise (par exemple, limitée aux fonctions utilisées) et complète. La description doit contenir les contraintes relatives au matériel et/ou logiciel que le Chargé d'intégration doit connaître et prendre en considération au cours de l'application. En particulier, elle constitue le moyen d'informer le Chargé d'intégration des raisons de la conception du logiciel, de ses propriétés, de son comportement et de ses caractéristiques.

NOTE Des preuves statistiques peuvent être utilisées dans la stratégie de validation des logiciels préexistants.

7.3.4.8 L'utilisation des composants logiciels vérifiés existants développés conformément à la présente Norme européenne dans la conception est à préférer chaque fois que possible.

7.3.4.9 Lorsque le logiciel est constitué de composants de niveaux d'intégrité de la sécurité logicielle différents, tous les composants logiciels doivent alors être traités comme s'ils appartenaient au plus élevé de ces niveaux, à moins qu'il n'existe une preuve d'indépendance entre les composants de plus haut niveau d'intégrité de la sécurité logicielle et les composants du plus bas niveau d'intégrité de la sécurité logicielle. Cette preuve doit être enregistrée dans la Spécification de l'Architecture du Logiciel.

7.3.4.10 La Spécification de l'Architecture du Logiciel doit décrire la stratégie de développement du logiciel dans la limite requise par le niveau d'intégrité de la sécurité logicielle. La Spécification de l'Architecture du Logiciel doit être exprimée et structurée de manière à être

a) complète, cohérente, claire, précise, sans équivoque, susceptible d'être vérifiée, testée, maintenue et réalisée,

b) traçable depuis la Spécification des Exigences du Logiciel.

7.3.4.11 Les mesures pour gérer les défauts doivent être incluses dans la Spécification de l'Architecture du Logiciel afin d'assurer l'équilibre entre les stratégies d'évitement de "fautes" et de tolérance aux "fautes".

7.3.4.12 La Spécification de l'Architecture du Logiciel doit démontrer que les techniques, les mesures et les outils choisis forment un ensemble qui satisfait à la Spécification des Exigences du Logiciel au niveau requis d'intégrité de la sécurité logicielle.

7.3.4.13 La Spécification de l'Architecture du Logiciel doit prendre en compte les exigences de 8.4.8 lorsque le logiciel est configuré par des données ou algorithmes d'application.

7.3.4.14 La Spécification de l'Architecture du Logiciel doit choisir des techniques et mesures issues du Tableau A.3. La combinaison choisie doit être justifiée comme un ensemble satisfaisant à 4.8 et 4.9.

7.3.4.15 La taille et la complexité de l'architecture de logiciel développée doivent être équilibrées.

7.3.4.16 Le prototypage peut être utilisé dans n'importe quelle phase pour identifier les exigences ou obtenir une vue plus détaillée des exigences et de leurs conséquences.

7.3.4.17 Un code issu d'un prototype ne peut être utilisé dans le système cible que s'il est démontré que le code et son développement ainsi que sa documentation satisfont à la présente Norme européenne.

7.3.4.18 Une Spécification d'Interface de Logiciel pour toutes les interfaces entre les composants logiciels et la frontière du logiciel global doit être rédigée, sous la responsabilité du Concepteur, sur la base de la Spécification des Exigences du Logiciel et de la Spécification de l'Architecture du Logiciel.

L'exigence 7.3.4.19 se rapporte à la Spécification d'Interface du Logiciel.

7.3.4.19 La description des interfaces doit porter sur

a) les préconditions/postconditions,

b) la définition et la description de toutes les valeurs aux limites pour toutes les données spécifiées,

c) le comportement lorsque la valeur à la limite est dépassée,

d) le comportement lorsque la valeur est à la limite,

e) pour les données d'entrée et de sortie à temps critique

   1) contraintes de temps et exigences pour un fonctionnement correct,

   2) gestion des exceptions,

f) la mémoire allouée pour les tampons d'interface et les mécanismes pour détecter que la mémoire ne peut pas être allouée ou tous les tampons sont pleins, selon le cas,

g) l'existence de mécanismes de synchronisation entre les fonctions (voir le point e).

Toutes les données en provenance et à destination des interfaces doivent être définies pour la plage complète de valeurs définies par le type des données, y compris les plages qui ne sont pas utilisées lorsqu'elles sont traitées par les fonctions :

a) définition et description de toutes les classes d'équivalence pour toutes les données spécifiées et de chaque fonction logicielle les utilisant,

b) définition des classes d'équivalence inutilisées ou interdites.

NOTE Le type de données comprend les éléments suivants :

1) les paramètres d'entrée et les résultats de sortie des fonctions et/ou procédures;

2) les données spécifiées dans les télégrammes ou paquets de communication;

3) les données du matériel.

7.3.4.20 Une Spécification de la Conception du Logiciel doit être rédigée, sous la responsabilité du Concepteur, sur la base de la Spécification des Exigences du Logiciel, de la Spécification de l'Architecture du Logiciel et de la Spécification d'Interface du Logiciel.

Les exigences de 7.3.4.21 à 7.3.4.24 se rapportent à la Spécification de la Conception du Logiciel.

7.3.4.21 Les documents en entrée doivent être disponibles, bien que pas nécessairement finalisés, avant le début du processus de conception.

7.3.4.22 La Spécification de la Conception du Logiciel doit décrire la conception du logiciel basée sur une décomposition en composants chaque composant comportant une Spécification de la Conception de Composant Logiciel et une Spécification de Tests de Composant Logiciel.

7.3.4.23 La Spécification de la Conception du Logiciel doit décrire

a) les composants logiciels tracés par rapport à l'architecture du logiciel et leur niveau d'intégrité de la sécurité,

b) les interfaces des composants logiciels avec l'environnement,

c) les interfaces entre les composants logiciels,

d) les structures de données,

e) l'allocation et la traçabilité des exigences sur les composants,

f) les algorithmes principaux et le séquencement,

g) les mécanismes de compte-rendu des erreurs.

7.3.4.24 La Spécification de la Conception du Logiciel doit choisir des techniques et mesures issues du Tableau A.4. La combinaison choisie doit être justifiée comme un ensemble satisfaisant à 4.8 et 4.9.

7.3.4.25 Des règles de codage doivent être développées et spécifier

a) les bonnes pratiques de programmation, telles que définies par Tableau A.12,

b) les mesures pour éviter ou détecter les erreurs qui peuvent être commises dans l'application du langage et qui ne sont pas détectables pendant la vérification (voir 7.5 et 7.6). De telles défaillances sont dérivées par analyse sur toutes les caractéristiques du langage,

c) les procédures pour la documentation du code source.

7.3.4.26 Le choix d'une règle de codage doit être justifié dans la limite requise par le niveau d'intégrité de sécurité logicielle.

7.3.4.27 Les règles de codage doivent être utilisées pour le développement de tous les logiciels et être référencées dans le Plan d'Assurance Qualité du Logiciel.

7.3.4.28 Conformément au niveau d'intégrité de la sécurité logicielle, la méthode de conception choisie doit offrir des caractéristiques qui facilitent

a) l'abstraction, la modularité et d'autres caractéristiques pour maîtriser la complexité,

b) l'expression claire et précise des éléments suivants :

   1) la fonctionnalité;

   2) le flux d'informations entre les composants;

   3) les séquencements et les informations temporelles;

   4) le parallélisme;

   5) la structure et les propriétés des données;

c) la compréhension humaine,

d) la vérification et la validation,

e) la maintenance du logiciel.

7.3.4.29 Une Spécification de Tests d'Intégration du Logiciel doit être rédigée, sous la responsabilité du Chargé d'intégration, sur la base de la Spécification des Exigences du Logiciel, de la Spécification de l'Architecture du Logiciel, de la Spécification de la Conception du Logiciel et de la Spécification d'Interface du Logiciel.

Les exigences de 7.3.4.30 à 7.3.4.32 se rapportent à la Spécification de Tests d'Intégration du Logiciel.

7.3.4.30 La Spécification de Tests d'Intégration du Logiciel doit être rédigée conformément aux exigences génériques établies pour une Spécification de Tests (voir 6.1.4.4).

7.3.4.31 La Spécification de Tests d'Intégration du Logiciel doit traiter de ce qui suit :

a) il doit être montré que chaque composant logiciel fournit les interfaces spécifiées pour d'autres composants en exécutant les composants ensemble;

b) il doit être montré que le logiciel se comporte d'une façon appropriée lorsque l'interface est soumise à des entrées qui sont hors spécification;

c) les données en entrée requises avec leurs séquences et leurs valeurs doivent être la base des scénarios de test;

d) les données de sortie prévues avec leurs séquences et leurs valeurs doivent être la base des scénarios de test;

e) il doit être indiqué quels résultats de test des composants (voir 7.5.4.5 et 7.5.4.7) sont censés être réutilisés pour les tests d'intégration du logiciel.

7.3.4.32 La Spécification de Tests d'Intégration du Logiciel doit choisir des techniques et des mesures issues du Tableau A.5. La combinaison choisie doit être justifiée comme un ensemble satisfaisant à 4.8 et 4.9.

7.3.4.33 Une Spécification de Tests d'Intégration Logiciel/Matériel doit être rédigée, sous la responsabilité du Chargé d'intégration, sur la base de la Description de la Conception du Système, de la Spécification des Exigences du Logiciel, de la Spécification de l'Architecture du Logiciel et de la Spécification de la Conception du Logiciel.

Les exigences de 7.3.4.34 à 7.3.4.39 se rapportent à la Spécification de Tests d'Intégration Logiciel/Matériel.

7.3.4.34 Une Spécification de Tests d'Intégration Logiciel/Matériel sera créée tôt dans le cycle de vie de développement, afin que les tests d'intégration puissent être convenablement menés et que les besoins particuliers en matière de conception ou autre intégration puissent être correctement assurés. Selon la taille du système, il est permis de scinder pendant le développement la Spécification de Tests d'Intégration Logiciel/Matériel en un certain nombre de sous-documents qui seront naturellement complétés à mesure que les conceptions de type matériel et logiciel évoluent et que les besoins détaillés d'intégration deviennent plus clairs.

7.3.4.35 La Spécification de Tests d'Intégration Logiciel/Matériel doit faire une distinction entre les activités qui peuvent être réalisées par le fournisseur dans ses locaux et celles exigeant l'accès au site de l'utilisateur.

7.3.4.36 La Spécification de Tests d'Intégration Logiciel/Matériel doit traiter de ce qui suit :

a) il doit être montré que le logiciel fonctionne correctement sur le matériel en utilisant le matériel via les interfaces matériel spécifiées;

b) il doit être montré que le logiciel peut gérer les défauts matériels comme demandé;

c) la temporisation et la performance requises doivent être démontrées;

d) les données en entrée requises avec leurs séquences et leurs valeurs doivent être la base des scénarios de test;

e) les données en entrée prévues avec leurs séquences et leurs valeurs doivent être la base des scénarios de test;

f) il doit être indiqué quels résultats de test des composants (voir7.5.4.5) et de test d'intégration du logiciel (voir 7.6.4.3) sont censés être réutilisés pour les tests d'intégration logiciel/matériel.

7.3.4.37 La Spécification de Tests d'Intégration Logiciel/Matériel doit documenter les items suivants :

a) les scénarios de test et les données liées au test;

b) les types de test à effectuer;

c) l'environnement de test y compris les outils, le logiciel de support et la description de la configuration;

d) les critères de test sur lesquels l'achèvement du test sera jugé.

7.3.4.38 La Spécification de Tests d'Intégration Logiciel/Matériel doit être rédigée conformément aux exigences génériques établies pour une Spécification de Tests 6.1.4.4).

7.3.4.39 La Spécification de Tests d'Intégration Logiciel/Matériel doit choisir des techniques et des mesures issues du Tableau A.5). La combinaison choisie doit être justifiée comme un ensemble satisfaisant à 4.8 et 4.9.

7.3.4.40 Un Rapport de Vérification de l'Architecture et de la Conception du Logiciel doit être rédigé, sous la responsabilité du Chargé de vérification, sur la base de la Spécification des Exigences du Logiciel, de la Spécification de l'Architecture du Logiciel, de la Spécification de la Conception du Logiciel, de la Spécification de Tests d'Intégration du Logiciel et de la Spécification de Tests d'Intégration Logiciel/Matériel.

Les exigences de 7.3.4.41 à 7.3.4.43 se rapportent au Rapport de Vérification de l'Architecture et de la Conception du Logiciel.

7.3.4.41 Le Rapport de Vérification de l'Architecture et de la Conception du Logiciel doit être rédigé conformément aux exigences génériques établies pour un Rapport de Vérification (voir 6.2.4.13).

7.3.4.42 Une fois que les Spécifications de l'Architecture, de l'Interface et de la Conception du Logiciel ont été définies, la vérification doit porter sur

a) la cohérence interne des Spécifications de l'Architecture, de l'Interface et de la Conception du Logiciel,

b) l'adéquation des Spécifications de l'Architecture, de l'Interface et de la Conception du Logiciel à satisfaire à la Spécification des Exigences du Logiciel en ce qui concerne la cohérence et la complétude,

c) le fait que la Spécification de l'Architecture du Logiciel satisfait aux exigences générales de lisibilité et de traçabilité de 5.3.2.7 à 5.3.2.10 et de 6.5.4.14 à 6.5.4.16 ainsi qu'aux exigences spécifiques dans 7.3.4.1 à 7.3.4.14,

d) le fait que la Spécification de l'Interface du Logiciel satisfait aux exigences générales de lisibilité et de traçabilité de 5.3.2.7 à 5.3.2.10 et de 6.5.4.14 à 6.5.4.16 ainsi qu'aux exigences spécifiques dans 7.3.4.18 à 7.3.4.19,

e) le fait que la Spécification de la Conception du Logiciel satisfait aux exigences générales de lisibilité et de traçabilité de 5.3.2.7 à 5.3.2.10 et de 6.5.4.14 à 6.5.4.16 ainsi qu'aux exigences spécifiques dans 7.3.4.20 à 7.3.4.24,

f) l'adéquation de la Spécification de l'Architecture du Logiciel et de la Spécification de la Conception du Logiciel pour tenir compte des contraintes relatives au matériel et au logiciel.

Les résultats doivent être enregistrés dans un Rapport de Vérification de l'Architecture et de la Conception du Logiciel.

7.3.4.43 Après que les Spécifications des Tests d'Intégration du Logiciel et d'Intégration Logiciel/Matériel ont été définies, la vérification doit porter sur

a) le fait que la Spécification de Tests d'Intégration du Logiciel satisfait aux exigences générales de lisibilité et de traçabilité de 5.3.2.7 à 5.3.2.10 et de 6.5.4.14 à 6.5.4.16 ainsi qu'aux exigences spécifiques dans 7.3.4.29 à 7.3.4.32,

b) le fait que la Spécification de Tests d'Intégration Logiciel/Matériel satisfait aux exigences générales de lisibilité et de traçabilité de 5.3.2.7 à 5.3.2.10 et de 6.5.4.14 à 6.5.4.16 ainsi qu'aux exigences spécifiques dans 7.3.4.33 à 7.3.4.39.

Les résultats doivent être enregistrés dans un Rapport de Vérification de l'Architecture et de la Conception du Logiciel.

7.4 Conception du Composant

7.4.1 Objectifs

7.4.1.1 Développer une conception de composant logiciel qui respecte les exigences de la Spécification de la Conception du Logiciel dans la limite requise par le niveau d'intégrité de la sécurité logicielle.

7.4.1.2 Développer une Spécification de Tests des Composants Logiciels qui respecte les exigences de la Spécification de la Conception de Composant Logiciel dans la limite requise par le niveau d'intégrité de la sécurité logicielle.

7.4.2 Documents en entrée

1) Spécification de la Conception du Logiciel

7.4.3 Documents en sortie

1) Spécification de la Conception des Composants logiciel

2) Spécification de Tests de Composant Logiciel

3) Rapport de Vérification de la Conception des Composants Logiciels

7.4.4 Exigences

7.4.4.1 Une Spécification de la Conception de Composant Logiciel doit être rédigée pour chaque composant, sous la responsabilité du Concepteur, sur la base de la Spécification de la Conception du Logiciel.

Les exigences de 7.4.4.2 à 7.4.4.6 se rapportent à la Spécification de la Conception des Composants Logiciel.

7.4.4.2 Pour chaque composant logiciel, les informations suivantes doivent être disponibles :

- l'auteur;
- l'historique de la configuration; et
- courte description.

L'historique de la configuration doit inclure une identification précise de la version courante et de toutes les versions antérieures, en spécifiant la version, la date et l'auteur, et une description des changements apportés par rapport à la version précédente.

7.4.4.3 La Spécification de la Conception des Composants Logiciel doit traiter des éléments suivants :

a) l'identification de toutes les unités de plus bas niveau (par exemple : sous-programmes, méthodes, procédures) tracées par rapport au niveau supérieur,

b) leurs interfaces détaillées avec l'environnement et les autres composants avec le détail de leurs entrées/sorties,

c) leur niveau d'intégrité de sécurité sans autre répartition dans le composant lui-même,

d) les algorithmes et les structures de données détaillés.

Chaque Spécification de la Conception des Composants Logiciel doit être auto-consistante et permettre la transformation en code des composants correspondants.

7.4.4.4 Chaque Spécification de la Conception des Composants Logiciel doit être lisible, compréhensible et susceptible d'être testée.

7.4.4.5 La taille et la complexité de chaque Composant Logiciel développé doivent être équilibrées.

7.4.4.6 La Spécification de la Conception des Composants Logiciel doit choisir des techniques et mesures issues du Tableau A.4. La combinaison choisie doit être justifiée comme un ensemble satisfaisant à 4.8 et 4.9.

7.4.4.7 Pour chaque composant, une Spécification de Tests des Composants Logiciel doit être rédigée, sous la responsabilité du Chargé des tests, sur la base de la Spécification de la Conception des Composants Logiciels.

Les exigences de 7.4.4.8 à 7.4.4.10 se rapportent à la Spécification de Tests des Composants Logiciel.

7.4.4.8 La Spécification de Tests des Composants Logiciel doit être rédigée conformément aux exigences génériques établies pour une Spécification de Tests (voir 6.1.4.4).

7.4.4.9 Une Spécification de Tests de Composant Logiciel doit être produite et chaque composant testé par rapport à elle. Ces tests doivent montrer que chaque composant exécute la fonction prévue. La Spécification de Tests de Composant Logiciel doit définir et justifier les critères requis et le degré exigé de couverture des tests dans la limite requise par le niveau d'intégrité logicielle. Les tests doivent être conçus de manière à atteindre trois objectifs :

a) confirmer que le composant exécute les fonctions prévues (tests boîte noire);

b) vérifier comment les parties internes du composant interagissent pour mener à bien les fonctions prévues (tests boîte noire/boîte blanche);

c) confirmer que toutes les parties du composant sont testées (tests boîte blanche).

7.4.4.10 La Spécification de Tests des Composants Logiciel doit choisir des techniques et mesures issues du Tableau A.5. La combinaison choisie doit être justifiée comme un ensemble satisfaisant à 4.8 et 4.9.

7.4.4.11 Un Rapport de Vérification de la Conception des Composants Logiciel doit être rédigé, sous la responsabilité du chargé de Vérification, sur la base de la Spécification de la Conception du Logiciel, de la Spécification de la Conception des Composants Logiciel et de la Spécification de Tests de Composant Logiciel.

Les exigences de 7.4.4.12 à 7.4.4.13 se rapportent au Rapport de Vérification de la Conception des Composants Logiciel.

7.4.4.12 Le Rapport de Vérification de la Conception des Composants Logiciel doit être rédigé conformément aux exigences génériques établies pour un Rapport de Vérification (voir 6.2.4.13).

7.4.4.13 Une fois que chaque Spécification de la Conception de Composant Logiciel a été définie, la vérification doit porter sur :

a) l'adéquation de la Spécification de la Conception de Composant Logiciel pour satisfaire à la Spécification de la Conception du Logiciel,

b) le fait que la Spécification de la Conception des Composants Logiciel satisfait aux exigences générales de lisibilité et de traçabilité de 5.3.2.7 à 5.3.2.10 et de 6.5.4.14 à 6.5.4.17 ainsi qu'aux exigences spécifiques dans 7.4.4.1 à 7.4.4.6,

c) l'adéquation de la Spécification de Tests de Composant Logiciel comme ensemble de scénarios de test pour la Spécification de la Conception de Composant Logiciel,

d) le fait que la Spécification de Tests des Composants Logiciel satisfait aux exigences générales de lisibilité et de traçabilité de 5.3.2.7 à 5.3.2.10 et de 6.5.4.14 à 6.5.4.17 ainsi qu'aux exigences spécifiques dans 7.4.4.7 à 7.4.4.10,

e) la décomposition de la Spécification de la Conception du Logiciel en composants logiciels et les Spécifications de la Conception de Composant Logiciel en faisant référence à

   1) la faisabilité de la performance requise,

   2) la testabilité pour une vérification ultérieure, et

   3) la maintenabilité pour permettre une modification ultérieure.

Les résultats doivent être enregistrés dans un Rapport de Vérification de la Conception des Composants Logiciels.

7.5 Réalisation et Test des composants

7.5.1 Objectifs

7.5.1.1 Réaliser un logiciel pouvant être analysé, testé, vérifié et maintenu. Les tests des composants font aussi partie de cette phase.

7.5.2 Documents en entrée

1) Spécification de la Conception des Composants logiciel

2) Spécification de Tests de Composant Logiciel

7.5.3 Documents en sortie

1) Code Source du Logiciel et Documentation de Support

2) Rapport de Tests des Composants Logiciels

3) Rapport de Vérification du Code Source du Logiciel

7.5.4 Exigences

7.5.4.1 Le code source du logiciel doit être rédigé sous la responsabilité du Réalisateur sur la base de la Spécification de la Conception de Composant Logiciel. Les exigences de 7.5.4.2 à 7.5.4.4 se rapportent au code source du logiciel.

7.5.4.2 La taille et la complexité du code source développé doivent être équilibrées.

7.5.4.3 Le code source du logiciel doit être lisible, compréhensible et susceptible d'être testé.

7.5.4.4 Le code source du logiciel doit être soumis au contrôle de configuration avant le commencement des tests documentés des composants.

7.5.4.5 Un Rapport de Tests de Composant Logiciel doit être rédigé, sous la responsabilité du Chargé des tests, sur la base de la Spécification de Tests de Composant Logiciel et du Code Source du Logiciel.

Les exigences de 7.5.4.6 à 7.5.4.7 se rapportent au Rapport de Tests des Composants Logiciel.

7.5.4.6 Le Rapport de Tests des Composants Logiciel doit être rédigé conformément aux exigences génériques établies pour un Rapport de Tests (voir 6.1.4.5).

7.5.4.7 Le Rapport de Tests de Composant Logiciel doit inclure les caractéristiques suivantes.

a) une déclaration des résultats des tests et si chaque composant remplit les exigences de sa Spécification de la Conception des Composants Logiciels,

b) une déclaration de la couverture des tests doit être fourni pour chaque composant, montrant que le degré requis de couverture des tests a été obtenu pour tous les critères requis.

7.5.4.8 Un Rapport de Vérification du Code Source du Logiciel doit être rédigé, sous la responsabilité du Chargé de vérification, sur la base de la Spécification de la Conception des Composants Logiciels, de la Spécification de tests de Composant Logiciel et du Code Source du Logiciel.

Les exigences de 7.5.4.9 à 7.5.4.10 se rapportent au Rapport de Vérification du Code Source du Logiciel.

7.5.4.9 Le Rapport de Vérification du Code Source du Logiciel doit être rédigé conformément aux exigences génériques établies pour un Rapport de Vérification (voir 6.2.4.13).

7.5.4.10 Après que le Code Source du Logiciel et le Rapport de Tests de Composant Logiciel ont été définis, la vérification doit porter sur

a) l'adéquation du Code Source du Logiciel comme une réalisation de la Spécification de la Conception des Composants Logiciel,

b) l'utilisation correcte des techniques et mesures choisies issues du Tableau A.4 comme un ensemble satisfaisant à 4.8 et 4.9,

c) la détermination de l'application correcte des règles de codage,

d) le fait que le Code Source du Logiciel satisfait aux exigences générales de lisibilité et de traçabilité de 5.3.2.7 à 5.3.2.10 et de 6.5.4.14 à 6.5.4.17 ainsi qu'aux exigences spécifiques dans 7.5.4.1 à 7.5.4.4,

e) l'adéquation du Rapport de Tests de Composant Logiciel comme enregistrement des tests effectués conformément à la Spécification de Tests de Composant Logiciel.

Les résultats doivent être enregistrés dans un Rapport de Vérification du Code Source du Logiciel.

7.6 Intégration

7.6.1 Objectifs

7.6.1.1 Mener à bien l'intégration du logiciel et l'intégration logiciel/matériel.

7.6.1.2 Démontrer que le logiciel et le matériel interagissent correctement pour exécuter les fonctions attendues.

7.6.2 Documents en entrée

1) Tests d'Intégration Logiciel/Matériel

2) Spécification de Tests d'Intégration du Logiciel

7.6.3 Documents en sortie

1) Rapport de Tests d'Intégration du Logiciel

2) Rapport de Tests d'Intégration Logiciel/Matériel

3) Rapport de Vérification de l'Intégration du Logiciel

7.6.4 Exigences

7.6.4.1 L'intégration des composants logiciels doit être le processus de regroupement progressif de chacun des composants logiciels testés au préalable au sein d'une entité de manière à ce que les interfaces des composants et le logiciel assemblé puissent être convenablement testés avant l'intégration et les tests du système.

7.6.4.2 Au cours de l'intégration logiciel/matériel, toute modification ou changement apporté au système intégré doit faire l'objet d'une étude d'impact qui doit identifier tous les composants affectés et les activités indispensables à une nouvelle vérification.

7.6.4.3 Un Rapport de Tests d'Intégration du Logiciel doit être rédigé, sous la responsabilité du Chargé d'Intégration, sur la base de la Spécification de Tests d'Intégration du Logiciel.

Les exigences de 7.6.4.4 à 7.6.4.6 se rapportent au Rapport de Tests d'Intégration du Logiciel.

7.6.4.4 Le Rapport de Tests d'Intégration du Logiciel doit être rédigé conformément aux exigences génériques établies pour un Rapport de Tests (voir 6.1.4.5).

7.6.4.5 Un Rapport de Tests d'Intégration du Logiciel doit être présenté comme suit :

a) un Rapport de Tests d'Intégration du Logiciel doit être présenté; il doit contenir les résultats des tests et préciser si les objectifs et les critères de la Spécification de Tests d'Intégration du Logiciel ont été respectés. En cas d'échec les circonstances doivent être consignées dans le rapport;

b) les scénarios de test et leurs résultats doivent être consignés, de préférence dans un format lisible par la machine en vue d'une analyse ultérieure;

c) Les tests doivent être reproductibles et, dans la mesure du possible, exécutés par des moyens automatiques;

d) le Rapport de Tests d'Intégration du Logiciel doit détailler l'identité et la configuration de tous les éléments concernés.

7.6.4.6 Le Rapport de Tests d'Intégration du Logiciel doit démontrer l'utilisation correcte des techniques et mesures choisies issues du Tableau A.6 comme un ensemble satisfaisant 4.8 et 4.9.

7.6.4.7 Un Rapport de Tests d'Intégration Logiciel/Matériel doit être rédigé, sous la responsabilité du Chargé d'intégration, sur la base de la Spécification de Tests d'Intégration Logiciel/Matériel.

Les exigences de 7.6.4.8 à 7.6.4.10 se rapportent au Rapport de Tests d'Intégration Logiciel/Matériel.

7.6.4.8 Le Rapport de Tests d'Intégration Logiciel/Matériel doit être rédigé conformément aux exigences génériques établies pour un Rapport de Tests (voir 6.1.4.5).

7.6.4.9 Un Rapport de Tests d'Intégration Logiciel/Matériel doit être rédigé comme suit :

a) le Rapport de Tests d'Intégration Logiciel/Matériel doit contenir les résultats des tests et préciser si les objectifs et les critères de la Spécification de Tests d'Intégration Logiciel/Matériel ont été respectés. En cas d'échec les circonstances doivent être enregistrées;

b) les scénarios de test et leurs résultats doivent être consignés, de préférence dans un format lisible par la machine en vue d'une analyse ultérieure;

c) le Rapport de Tests d'Intégration Logiciel/Matériel doit détailler l'identité et la configuration de tous les éléments concernés.

7.6.4.10 Le Rapport de Tests d'Intégration Logiciel/Matériel doit démontrer l'utilisation correcte des techniques et mesures choisies issues du Tableau A.6 comme un ensemble satisfaisant 4.8 et 4.9.

7.6.4.11 Un Rapport de Vérification de l'Intégration du Logiciel doit être rédigé, sous la responsabilité du Chargé de vérification, sur la base des Spécifications des Tests d'Intégration du Logiciel et Logiciel/Matériel et des Rapports de Tests correspondants.

Les exigences de 7.6.4.12 à 7.6.4.13 se rapportent au Rapport de Vérification de l'Intégration du Logiciel.

7.6.4.12 Le Rapport de Vérification de l'Intégration du Logiciel doit être rédigé conformément aux exigences génériques établies pour un Rapport de Vérification (voir 6.2.4.13).

7.6.4.13 Après que le Rapport de Tests d'Intégration du Logiciel et le Rapport de Tests d'Intégration Logiciel/Matériel ont été définis, la vérification doit porter sur

a) l'adéquation du Rapport de Tests d'Intégration du Logiciel comme enregistrement des tests effectués conformément à la Spécification de Tests d'Intégration du Logiciel,

b) si, oui ou non, le Rapport de Tests d'Intégration du Logiciel satisfait aux exigences de lisibilité et de traçabilité de 5.3.2.7 à5.3.2.10 et de 6.5.4.14 à 6.5.4.17 ainsi qu'aux exigences spécifiques dans 7.6.4.3 à 7.6.4.6,

c) l'adéquation du Rapport de Tests d'Intégration du Logiciel/Matériel comme enregistrement des tests effectués conformément à la Spécification de Tests d'Intégration Logiciel/Matériel,

d) si, oui ou non, le Rapport de Tests d'Intégration Logiciel/Matériel satisfait aux exigences générales de lisibilité et de traçabilité de 5.3.2.7 à 5.3.2.10 et de 6.5.4.14 à 6.5.4.17 ainsi qu'aux exigences spécifiques dans 7.6.4.7 à 7.6.4.10.

7.7 Tests d'Ensemble du Logiciel / Validation Finale

7.7.1 Objectifs

7.7.1.1 Analyser et tester l'ensemble intégré logiciel/matériel pour assurer la conformité avec la Spécification des Exigences du Logiciel, en mettant tout particulièrement l'accent sur les aspects fonctionnels et ceux de sécurité selon le niveau d'intégrité de la sécurité logicielle et vérifier qu'il est adapté à son application prévue.

7.7.2 Documents en entrée

1) Spécification des Exigences du Logiciel

2) Spécification de Tests d'Ensemble du Logiciel

3) Plan de Vérification du Logiciel

4) Plan de Validation du Logiciel

5) L'ensemble de la documentation du matériel et logiciel, y compris les résultats intermédiaires de la vérification

6) Spécification des Exigences de Sécurité du Système

7.7.3 Documents en sortie

1) Rapport de Tests d'Ensemble du Logiciel

2) Rapport de Validation du Logiciel

3) Fiche de livraison

7.7.4 Exigences

7.7.4.1 Un Rapport de Tests d'Ensemble du Logiciel doit être rédigé, sous la responsabilité du Chargé des tests, sur la base de la Spécification de Tests d'Ensemble du Logiciel.

Les exigences de 7.7.4.2 à 7.7.4.4 se rapportent au Rapport de Tests d'Ensemble du Logiciel.

7.7.4.2 Le Rapport de Tests d'Ensemble du Logiciel doit être rédigé conformément aux exigences génériques établies pour un Rapport de Tests (voir 6.1.4.5).

7.7.4.3 Le Chargé de validation doit spécifier et exécuter des tests supplémentaires à sa discrétion ou les faire exécuter par le Chargé des tests. Alors que les Tests d'Ensemble du Logiciel sont principalement fondés sur la structure de la Spécification des Exigences du Logiciel, la valeur ajoutée que doit apporter le Chargé de validation est constituée des tests qui sollicitent le système par des scénarios complexes reflétant les besoins de l'utilisateur.

7.7.4.4 Les résultats de l'ensemble des tests et analyses doivent être consignés dans le Rapport de Tests d'Ensemble du Logiciel.

7.7.4.5 Le logiciel doit être mis en œuvre soit en le connectant à des éléments réels de matériel ou de systèmes réels avec lesquels il aurait une interface lors de son fonctionnement, soit par simulation de signaux d'entrée et de charges pilotées par des grandeurs de sortie. Il doit être testé dans les conditions présentes en fonctionnement normal ou présentes lors de situations prévues et de conditions non voulues exigeant une action du système. Lorsque des entrées ou charges simulées sont utilisées, il doit être montré qu'elles ne diffèrent pas sensiblement de celles rencontrées en fonctionnement.

NOTE Les entrées ou charges simulées pourraient remplacer les entrées ou les charges qui ne sont présentes qu'au niveau système ou dans des modes défaillants.

7.7.4.6 Un Rapport de Validation du Logiciel doit être rédigé, sous la responsabilité du Chargé de validation, sur la base du Plan de Validation du Logiciel.

Les exigences de 7.7.4.7 à 7.7.4.11 se rapportent au Rapport de Validation du Logiciel.

7.7.4.7 Le Rapport de Validation du Logiciel doit être rédigé conformément aux exigences génériques établies pour un Rapport de Validation (voir 6.3.4.7 à 6.3.4.11).

7.7.4.8 Une fois l'intégration terminée, ainsi que les Tests d'Ensemble du Logiciel et les analyses achevés; un Rapport de Validation du Logiciel doit être produit comme suit :

a) il doit préciser si les objectifs et les critères du Plan de Validation du Logiciel ont été respectés. Les écarts par rapport au plan doivent être consignés et justifiés;

b) il doit contenir une déclaration résumée des résultats des tests et préciser si le logiciel complet sur sa machine cible remplit les exigences exposées dans la Spécification des Exigences du Logiciel;

c) il doit contenir une évaluation de la couverture des exigences de la Spécification des Exigences du Logiciel, par les tests;

d) une évaluation d'autres activités de vérification conformément au Plan et Rapport de Vérification du Logiciel doit être effectuée accompagnée d'une vérification que la traçabilité des exigences est complètement exécutée et couverte;

e) si le Chargé de validation produit ses propres scénarios de test qui ne sont pas remis au Chargé des tests, le Rapport de Validation du Logiciel doit les documenter selon 6.3.4.7 à 6.3.4.11.

7.7.4.9 Le Rapport de Validation du Logiciel doit contenir la confirmation que chaque combinaison de techniques ou de mesures choisies conformément à l'Annexe A est appropriée pour le niveau défini d'intégrité de sécurité logicielle. Il doit contenir une évaluation de l'efficacité globale de la combinaison de techniques et de mesures adoptée en prenant en compte la taille et la complexité du logiciel produit et en tenant compte des résultats réels des activités de test, de vérification et de validation.

7.7.4.10 Les points suivants doivent être traités dans le Rapport de Validation du Logiciel :

a) documentation de l'identité et de la configuration du logiciel;

b) déclaration identifiant de façon appropriée les logiciels et les équipements de support technique;

c) déclaration identifiant de façon appropriée les modèles de simulation utilisés;

d) déclaration relative à l'adéquation de la Spécification de Tests d'Ensemble du Logiciel;

e) recueil et conservation de la trace des éventuels écarts trouvés;

f) revue et évaluation de tous les écarts en termes de risque (impact);

g) déclaration que le projet a correctement géré les actions correctives conformément aux processus et procédures de gestion des modifications et avec une identification claire de toute divergence décelée;

h) déclaration de chaque restriction donnée par les écarts d'une manière traçable;

i) conclusion indiquant si, oui ou non; le logiciel est adéquat pour son application prévue, en tenant compte des conditions et des contraintes d'application.

7.7.4.11 Les éventuelles divergences constatées, y compris les erreurs et les non-conformités à la présente Norme européenne ou à tout(e) exigence ou plan concernant le logiciel, ainsi que les contraintes et les limitations, doivent être clairement identifiées dans un paragraphe séparé du Rapport de Validation du Logiciel, évaluées quant au niveau d'intégrité de sécurité et incluses dans toute fiche de livraison qui accompagne le logiciel livré.

7.7.4.12 Une fiche de livraison qui accompagne le logiciel livré doit contenir toutes les restrictions dans l'utilisation du logiciel. Ces restriction sont dérivées des

a) erreurs détectées,

b) non-conformités à la présente Norme européenne,

c) degrés de respect des exigences,

d) degrés de respect d'un plan éventuel.

8 Développement de données d'application ou d'algorithmes d'application : systèmes configurés par des données d'application ou par des algorithmes d'application

8.1 Objectifs

8.1.1 Un trait caractéristique de nombreux systèmes ferroviaires est la nécessité de concevoir chaque installation en vue de répondre aux exigences individuelles d'une application spécifique. Un système configuré par des données d'application et/ou par des algorithmes d'application permet d'adapter spécialement un logiciel générique homologué avec des exigences individuelles pour chaque application spécifique.

L'objectif du développement de données d'application est l'obtention correcte des données à partir de l'installation concernée et la vérification du comportement prévu, suivie d'une évaluation du processus de développement utilisé pour les données d'application en question.

Les exigences pour le développement d'algorithmes d'application sont les mêmes que celles concernant le développement d'un logiciel générique comme décrit aux Articles 1 à 7 et 9.

Un exemple type est un système dont le logiciel générique est préconfiguré pour une application ferroviaire générique par un jeu d'algorithmes d'application et qui est en outre configuré ensuite selon chaque installation spécifique par instanciation et interconnexion des algorithmes d'application et par un jeu de données de configuration. Par exemple, les principes de signalisation d'un système d'enclenchement (par exemple : une gestion de signaux, une gestion d'aiguilles) peuvent être mis en œuvre par un jeu d'algorithmes d'application.

Les données d'application se présentent en général sous forme de valeurs de paramètres ou de descriptions (identité, type, emplacement, etc.) d'objets externes. Les algorithmes d'application peuvent se présenter sous forme de diagrammes fonctionnels, de diagrammes d'états et de diagrammes à contacts, qui déterminent la réponse souhaitée du système en fonction de ses entrées, de son état courant et de valeurs de paramètres spécifiques. Les algorithmes d'application comprennent des connexions et opérations logiques à effectuer.

Les données et algorithmes d'application sont en général produits à l'aide d'outils dédiés. Ils peuvent être exprimés aux formats de tableaux ou de diagrammes, qui peuvent être interprétés ou compilés en codes exécutables souvent après conversion en codes sources manipulés via des langages spécialisés (avec syntaxe et sémantique).

La personnalisation des systèmes par l'aptitude à la configuration donne au Concepteur différents degrés de maîtrise sur la fonctionnalité détaillée du logiciel.

8.1.2 Les procédures et les outils utilisés pour leur développement doivent être appropriés au niveau d'intégrité de sécurité du système tel que déterminé par la fonction pour laquelle ils sont développés.

8.1.3 Les paragraphes ci-après décrivent les exigences pour le développement initial d'un système configurable et pour le développement subséquent de chaque jeu de données/algorithmes spécifiques à l'application.

8.2 Documents en entrée

1) Spécification des Exigences de Logiciel pour le logiciel générique

2) Spécification de l'Architecture de Logiciel pour le logiciel générique

3) Conditions d'application du logiciel générique et des outils d'application

4) Manuels de l'utilisateur du logiciel générique et des outils d'application

8.3 Documents en sortie

1) Plan de préparation de l'application

2) Spécification des Exigences de l'Application

3) Architecture et Conception de l'Application

4) Spécification de Tests de l'Application

5) Rapport de Tests d'Application

6) Rapport de Vérification de la Préparation de l'Application

7) Code Source des données/algorithmes d'application

8) Rapport de vérification des données/algorithmes d'application

8.4 Exigences

8.4.1 Processus de développement de l'Application

8.4.1.1 Un Plan de Préparation d'Application doit être rédigé, sous la responsabilité du Gestionnaires des Exigences ou du Concepteur, sur la base des documents en entrée de 8.2.

Les exigences de 8.4.1.2 à 8.4.1.11 se rapportent au Plan de Préparation de l'Application.

8.4.1.2 Un Plan de Préparation de l'Application doit être produit afin de définir et détailler le processus de développement de l'application, y compris toutes les activités, tous les documents à fournir et tous les rôles qui en ont la charge. Il peut être produit soit pour chaque application spécifique, soit pour une classe d'applications spécifiques, c'est-à-dire pour une application générique.

8.4.1.3 Le Plan de Préparation de l'Application doit définir une structure de documentation pour le processus de préparation de l'application.

8.4.1.4 Le Plan de Préparation de l'Application doit choisir des techniques et mesures issues du Tableau A.11. La combinaison choisie doit être justifiée comme un ensemble satisfaisant à 4.8 et 4.9.

8.4.1.5 Le Plan de Préparation de l'Application doit spécifier les procédures et les outils d'application (avec leur classification basée sur 6.7) devant être utilisés dans le processus de développement de l'application.

8.4.1.6 Le Plan de Préparation de l'Application doit inclure des activités de vérification et de validation pour assurer que les données/algorithmes d'application sont complets, corrects et compatibles les uns avec les autres et avec l'application générique, et fournir la preuve que les conditions d'application de l'application générique sont respectées. Ces activités de vérification et de validation peuvent être remplacées par la vérification et la validation exécutées sur les outils qui produisent les données/algorithmes d'application. Les résultats sont regroupés dans le Rapport de Vérification de la Préparation d'Application et le Rapport de Tests d'Application.

8.4.1.7 Le Plan de Préparation de l'Application doit inclure des activités de vérification et de validation pour assurer que les outils d'application et le logiciel générique sont compatibles les uns avec les autres et avec l'application spécifique, et fournir la preuve que leurs conditions d'application sont respectées.

8.4.1.8 Une analyse du risque doit être effectuée et couvrir le processus de développement de l'application, notamment les outils et procédures de l'application afin de valider le Plan de Préparation de l'Application et satisfaire au niveau requis d'intégrité de la sécurité logicielle. Le Plan de Préparation de l'Application doit inclure l'analyse du risque.

8.4.1.9 Le Plan de Préparation de l'Application doit spécifier les exigences d'indépendance entre les équipes réalisant les tâches de vérification, de validation et de préparation selon 5.1

NOTE Les activités de préparation des données sont accomplies par les concepteurs d'application.

8.4.1.10 Le Plan de Préparation de l'Application doit définir une classe d'outils pour tout outil matériel ou logiciel utilisé dans le cycle de vie de la préparation de l'application.

8.4.1.11 Chaque fois que possible, le Plan de Préparation de l'Application doit utiliser, pour spécifier les exigences et la conception, des notations qui sont familières aux ingénieurs d'applications. Lorsque de nouvelles notations sont introduites, la documentation utilisateur nécessaire doit être fournie ainsi que la formation le cas échéant.

8.4.1.12 Un Rapport de Vérification des Données/Algorithmes d'Application doit être rédigé, sous la responsabilité du Chargé de vérification, sur la base des documents en entrée de 8.2.

L'exigence en 8.4.1.13 se rapporte au Rapport de Vérification de Données/Algorithmes d'Application.

8.4.1.13 Une fois que le Plan de Préparation de l'Application a été défini, la vérification doit porter sur

a) le fait que le Plan de Préparation de l'Application satisfait aux exigences générales de lisibilité et de traçabilité de 5.3.2.7 à 5.3.2.10 et de 6.5.4.14 à 6.5.4.17 ainsi qu'aux exigences spécifiques dans 8.4.1.2 à 8.4.1.11,

b) la cohérence interne du Plan de Préparation de l'Application.

Les résultats doivent être consignés dans un Rapport de Vérification des Données/Algorithmes d'Application.

8.4.1.14 La mise en œuvre du Plan de Préparation de l'Application doit être vérifiée et validée pour chaque application spécifique.

8.4.2 Définitions des Exigences de l'Installation

8.4.2.1 Une Spécification des Exigences de l'Application doit être rédigée, sous la responsabilité du Gestionnaires des Exigences, sur la base des documents en entrée de 8.2.

Les exigences de 8.4.2.2 à 8.4.2.3 se rapportent à la Spécification des Exigences de l'Application.

8.4.2.2 Les exigences pour l'application spécifique doivent comprendre les exigences qui sont spécifiques à l'installation à l'étude (par exemple : disposition des voies, emplacements des signaux, vitesses limites pour le système de signalisation), ainsi qu'une récapitulation ou une référence relative aux conditions d'application

du logiciel générique et des outils d'application, et les normes auxquelles il est indispensable que l'application se conforme (par exemple : les principes de signalisation pour un système de signalisation).

8.4.2.3 Les exigences relatives aux données et algorithmes d'application traités par le logiciel générique du système doivent être spécifiées à ce stade.

8.4.2.4 Un Rapport de Vérification des Données/Algorithmes d'Application doit être rédigé, sous la responsabilité du Chargé de vérification, sur la base des documents en entrée de 8.2.

L'exigence en 8.4.2.5 se rapporte au Rapport de Vérification de Données/Algorithmes d'Application.

8.4.2.5 une fois que la Spécification des Exigences de l'Application a été définie, la vérification doit porter sur :

a) le fait que la Spécification des Exigences de l'Application satisfait aux exigences générales de lisibilité et de traçabilité de 5.3.2.7 à 5.3.2.10 et de 6.5.4.14 à 6.5.4.17 ainsi qu'aux exigences spécifiques dans 8.4.2.2 à 8.4.2.3,

b) la cohérence interne de la Spécification des Exigences de l'Application.

Les résultats doivent être consignés dans un Rapport de Vérification des Données/Algorithmes d'Application.

8.4.3 Architecture et Conception

La quantité et le type des composants matériels et logiciels génériques à utiliser dans l'application spécifique doivent être spécifiés. L'emplacement des composants des données d'application et des algorithmes d'application dans l'architecture de l'application spécifique doit être défini. Les données et algorithmes d'application traités par le logiciel générique doivent être conçus à ce stade.

8.4.4 Production des Données/Algorithmes d'Application

8.4.4.1 Le processus de développement de l'application doit inclure la production et la compilation du code source des données/algorithmes génériques et spécifiques, ainsi que les activités de vérification et de tests liées à cette production. L'utilisation de langages en diagramme est recommandée pour produire le code source des algorithmes d'application. Se référer au Tableau A.16.

8.4.4.2 Un Rapport de Tests de l'Application doit être rédigé, sous la responsabilité du Chargé des Tests, sur la base des documents en entrée de 8.2.

L'exigence 8.4.4.3 se rapporte au Rapport de Tests de l'Application.

8.4.4.3 Le Rapport de Tests de l'Application doit documenter l'exécution correcte et complète des tests définis dans la Spécification de Tests de l'Application.

8.4.4.4 Le Rapport de Vérification de la Préparation d'Application doit

a) documenter chaque activité exécutée afin d'assurer le caractère correct et la complétude des données/algorithme et leur cohérence avec les principes de l'application et l'architecture de l'application générique,

b) évaluer la compatibilité des données/algorithmes avec l'application générique.

8.4.4.5 Une Spécification de Tests de l'Application doit être rédigée, sous la responsabilité du Chargé des Tests, sur la base des documents en entrée de 8.2.

L'exigence en 8.4.4.6 se rapporte à la Spécification de Tests de l'Application.

8.4.4.6 La Spécification de Tests de l'Application doit spécifier les tests à réaliser à un stade intermédiaire ou final de la préparation des données/algorithmes, afin d'assurer

a) la cohérence et la complétude des données/algorithmes par rapport aux principes de l'application,

b) la cohérence et la complétude des données/algorithmes par rapport à l'architecture de l'application spécifique.

8.4.4.7 Un Rapport de Vérification des Données/Algorithmes d'Application doit être rédigé, sous la responsabilité du Chargé de vérification, sur la base des documents en entrée de 8.2.

L'exigence en 8.4.4.8 se rapporte au Rapport de Vérification de Données/Algorithmes d'Application.

8.4.4.8 Une fois que la Spécification de Tests de l'Application a été définie, la vérification doit porter sur :

a) le fait que la Spécification de Tests de l'Application satisfait aux exigences générales de lisibilité et de traçabilité de 5.3.2.7 à 5.3.2.10 et de 6.5.4.14 à 6.5.4.17 ainsi qu'aux exigences spécifiques dans 8.4.4.6,

b) la cohérence interne de la Spécification de Tests de l'Application.

Les résultats doivent être consignés dans un Rapport de Vérification des Données/Algorithmes d'Application.

8.4.5 Intégration de l'Application et Acceptation des Tests

8.4.5.1 Dans certains systèmes, les données/algorithmes d'application peuvent être intégré(e)s dans le matériel et le logiciel génériques pour un test en usine avant l'installation sur le système cible. Dans les cas où le niveau de confiance peut être obtenu par d'autres moyens, il n'est pas exclu de supprimer cette démarche. L'application doit alors être installée sur le système cible et les tests d'intégration au sein de l'installation complète doivent être effectués. Enfin, le système cible doit être mis en service comme système pleinement opérationnel et un processus d'acceptation finale du système cible dans l'installation complète doit être réalisé. Le Rapport de Tests de l'Application doit documenter l'exécution correcte et complète des tests définis dans la Spécification de Tests de l'Application. Le Rapport de Vérification de la Préparation de l'Application doit vérifier la complétude et le caractère correct des tests réalisés sur l'installation complète.

8.4.5.2 Une Spécification de Tests de l'Application doit être rédigée, sous la responsabilité du Chargé des Tests, sur la base des documents en entrée de 8.2.

L'exigence en 8.4.5.3 se rapporte à la Spécification de Tests de l'Application.

8.4.5.3 La Spécification de Tests de l'Application doit spécifier les tests à réaliser pour assurer

a) l'intégration correcte des données/algorithmes sur le matériel et le logiciel génériques, si besoin est,

b) l'intégration correcte des données/algorithmes avec l'installation complète.

8.4.5.4 Un Rapport de Vérification des Données/Algorithmes d'Application doit être rédigé, sous la responsabilité du Chargé de vérification, sur la base des documents en entrée de 8.2.

L'exigence en 8.4.5.5 se rapporte au Rapport de Vérification de Données/Algorithmes d'Application.

8.4.5.5 Une fois que la Spécification de Tests de l'Application a été définie, la vérification doit porter sur le fait que la Spécification de Tests de l'Application satisfait aux exigences spécifiques dans 8.4.5.3.

8.4.6 Validation et Évaluation de l'Application

Les activités de validation et d'évaluation doivent contrôler la performance de chaque étape du cycle de vie.

8.4.7 Procédures et outils de préparation de l'application

8.4.7.1 Pour chaque nouveau type de système configuré par des données/algorithmes d'application, des procédures et outils spécifiques doivent être développés pour permettre d'appliquer aux installations du nouveau système le processus de développement d'application spécifié en 8.4.1. Le développement de ces outils doit être réalisé selon la présente Norme européenne en parallèle avec le matériel et le logiciel génériques du système. Les activités de vérification, de validation et d'évaluation doivent assurer que les outils de préparation des données et le logiciel générique sont compatibles.

8.4.7.2 Tout processus de compilation doit être validé et évalué. Il doit être noté que des compilateurs spécialisés sont en général nécessaires pour la conversion des données et des algorithmes.

8.4.7.3 La totalité des données/algorithmes d'application et de la documentation associée pour chaque application spécifique doit être soumise aux exigences de déploiement du logiciel telles que spécifiées en 9.1.

8.4.7.4 La totalité des données/algorithmes d'application et de la documentation associée doit être soumise aux exigences de maintenance du logiciel spécifiées en 9.2.

8.4.7.5 La totalité des données/algorithmes d'application et de la documentation associée doit être placée sous la gestion de configuration selon les exigences spécifiées en 6.5 et 6.7. La gestion de configuration des données/algorithmes d'application peut être séparée de la partie logiciel générique.

8.4.7.6 Le Rapport de Vérification d'Application démontre la couverture et la mise en application des conditions d'application du logiciel générique et des outils d'application.

8.4.8 Développement de logiciel générique

8.4.8.1 Le développement du logiciel générique, qui prend en charge l'exécution des données/algorithmes d'application, doit se conformer aux exigences de 7.1 à 7.7 de la présente Norme européenne. Les exigences supplémentaires suivantes doivent également être respectées.

8.4.8.2 Les types ou classes de fonction qui peuvent être configuré(e)s par des données/algorithmes d'application dans chaque système et sous-système doivent être identifié(e)s dans les documents de la Spécification des Exigences du Logiciel du logiciel générique. Le niveau d'intégrité de la sécurité alloué aux fonctions déterminera les normes à appliquer au développement ultérieur des données/algorithmes d'application pour toutes les installations du système.

8.4.8.3 Au cours de la phase de conception du logiciel, générique, les interfaces détaillées entre le logiciel générique et les données/algorithmes d'application doivent être spécifiées, à moins qu'elles n'aient déjà été spécifiées lors d'une phase précédente du cycle de vie, par exemple en raison d'une exigence d'utilisation d'un langage d'application spécifique existant.

8.4.8.4 Il doit être mis en pratique une séparation rigide entre le logiciel générique et les données/algorithmes d'application, autrement dit il doit être possible de recompiler et de mettre à jour, soit le logiciel générique, soit les données/algorithmes d'application, sans avoir à mettre l'autre à jour, à moins qu'une modification n'ait été apportée dans l'interface définie entre le logiciel générique et les données/algorithmes d'application. De même, les données/algorithmes spécifiques des applications doivent être séparé(e)s des données/algorithmes d'application génériques.

8.4.8.5 Il faut que les procédures de contrôle des modifications assurent que toute modification apportée au logiciel générique peut être installée uniquement quand il a été établi que le logiciel révisé est compatible avec les données/algorithmes d'application d'origine ou que les données/algorithmes d'application ont fait l'objet d'une révision.

8.4.8.6 Il faut accorder une attention particulière au processus de vérification et à la phase de tests de validation afin d'assurer que toutes les combinaisons pertinentes de données et d'algorithmes sont examinées.

Si toutes les combinaisons pertinentes de données et d'algorithmes n'ont pas été examinées dans le processus de vérification, de tests et de validation du logiciel générique, cela doit être clairement identifié comme une limite d'utilisation du logiciel générique. Un complément du processus de vérification, de tests et de validation du logiciel générique doit être exécuté en cas de définition d'un certain nombre des données ou d'algorithmes au-delà de cette limite.

8.4.8.7 Lorsque cela est réalisable, le logiciel générique doit être conçu pour détecter les données/algorithmes d'application altéré(e)s.

8.4.8.8 Les concepteurs doivent éditer la Fiche de Livraison du logiciel générique et des outils d'application au plus tard dans la Phase de Tests d'Ensemble du Logiciel/Validation Finale relative au logiciel générique et aux outils d'application. Le contenu de ces documents doit être soumis aux activités de vérification et de validation.

Les rubriques suivantes doivent être traitées dans le document “Conditions d'application du logiciel générique et des outils d'application” :

1) les références aux manuels d'utilisateur du logiciel générique et outils d'application;

2) les éventuelles contraintes sur les données/algorithmes d'application, par exemple : règles d'architecture ou de codage imposées pour respecter les niveaux d'intégrité de la sécurité.

9 Déploiement et maintenance du logiciel

9.1 Déploiement du logiciel

9.1.1 Objet

9.1.1.1 Assurer que le logiciel fonctionne comme requis, en préservant le niveau requis d'intégrité de la sécurité logicielle et la sûreté de fonctionnement requise, lorsqu'il est déployé dans l'environnement final d'application.

9.1.2 Documents en entrée

Tous les documents de conception, de développement et d'analyses pertinents pour le déploiement.

9.1.3 Documents en sortie

1) Plan de livraison et de déploiement du Logiciel

2) Manuel de Déploiement du Logiciel

3) Fiche de livraison

4) Enregistrements du Déploiement

5) Rapport de Vérification du Déploiement

9.1.4 Exigences

9.1.4.1 Le déploiement doit être effectué sous la responsabilité du chef de projet.

9.1.4.2 Avant de livrer une diffusion de logiciel, le référentiel du logiciel doit être enregistré et maintenu traçable sous le contrôle de la gestion de configuration. Les logiciels préexistants et ceux développés conformément à une version antérieure de la présente Norme européenne doivent également être inclus.

9.1.4.3 La diffusion du logiciel doit être reproductible tout au long du cycle de vie du référentiel du logiciel.

9.1.4.4 Une Fiche de livraison doit être rédigée, sous la responsabilité du Concepteur, sur la base des documents en entrée de 9.1.2.

L'exigence en 9.1.4.5 se rapporte à la Fiche de livraison.

9.1.4.5 Une fiche de livraison doit être fournie et définir

a) les conditions d'application qu'il faut respecter,

b) les informations de compatibilité parmi les composants du logiciel et entre le logiciel et le matériel,

c) toutes les restrictions dans l'utilisation du logiciel (voir 7.7.4.12).

9.1.4.6 Un Manuel de Déploiement du Logiciel doit être rédigé sur la base des documents en entrée de 9.1.2.

L'exigence 9.1.4.7 se rapporte au Manuel de Déploiement du Logiciel.

9.1.4.7 Le Manuel de Déploiement du Logiciel doit définir les procédures pour identifier et installer correctement une diffusion de logiciel.

9.1.4.8 En cas de déploiement incrémentiel (c'est-à-dire le déploiement de composants individuels), il est hautement recommandé pour les SIL 3 et SIL 4 et recommandé pour les SIL 1 et SIL 2 que le logiciel soit conçu de manière à contenir des moyens assurant que l'activation de versions incompatibles du logiciel est exclue.

9.1.4.9 La gestion de configuration doit assurer qu'aucun dommage ne résultera de la présence simultanée de différentes versions des mêmes composants logiciels lorsque cela ne peut être évité.

9.1.4.10 Une procédure de retour en arrière (c'est-à-dire une capacité de retourner à la diffusion précédente) doit être disponible lors de l'installation d'une nouvelle diffusion du logiciel.

9.1.4.11 Le logiciel doit avoir des mécanismes d'auto-identification intégrés, permettant son identification au moment du processus de chargement et après chargement dans la cible. Il convient que le mécanisme d'auto-identification indique les informations de version pour le logiciel et toutes les éventuelles données de configuration ainsi que l'identité du produit.

NOTE Il est recommandé de protéger par codage (voir Tableau A.3 "Codes de détection d'erreur") les données du code contenant les informations sur la diffusion du logiciel.

9.1.4.12 Un Registre de Déploiements doit être rédigé sur la base des documents en entrée de 9.1.2.

L'exigence 9.1.4.13 se rapporte au Registre de Déploiements du Logiciel.

9.1.4.13 Un Registre de Déploiements doit donner la preuve que le logiciel prévu a été chargé, par examen des mécanismes d'auto-identification intégrés (voir 9.1.4.11). Ce registre doit être conservé parmi les documents liés au système livré comme les autres vérifications et fait partie de la mise en service et de l'acceptation.

9.1.4.14 Le logiciel déployé doit être traçable aux installations livrées.

NOTE Ceci revêt une importance spéciale lorsque des défauts critiques sont découverts et nécessitent d'être corrigés dans plus d'une installation.

9.1.4.15 Des informations relatives au diagnostic doivent être fournies par le logiciel, comme partie de la surveillance des défauts.

9.1.4.16 Un Rapport de Vérification de Déploiement doit être rédigé, sous la responsabilité du Chargé de vérification, sur la base des documents en entrée de 9.1.2.

Les exigences de 9.1.4.17 à 9.1.4.19 se rapportent au Rapport de Vérification du Déploiement.

9.1.4.17 Une fois que le Manuel de Déploiement du Logiciel a été défini, la vérification doit porter sur :

a) le fait que le Manuel de Déploiement du Logiciel satisfait aux exigences générales de lisibilité et de traçabilité de 5.3.2.7 à 5.3.2.10 et de 6.5.4.14 à 6.5.4.17 ainsi qu'aux exigences spécifiques dans 9.1.4.7,

b) la cohérence interne du Manuel de Déploiement du Logiciel.

9.1.4.18 Une fois que le Registre de Déploiement a été défini, la vérification doit porter sur :

a) le fait que le Registre de Déploiement du Logiciel satisfait aux exigences générales de lisibilité et de traçabilité de 5.3.2.7 à 5.3.2.10 et de 6.5.4.14 à 6.5.4.17 ainsi qu'aux exigences spécifiques dans 9.1.4.13,

b) la cohérence interne du Registre de Déploiement.

9.1.4.19 Une fois que la Fiche de livraison a été définie, la vérification doit porter sur :

a) le fait que la Fiche de livraison satisfait aux exigences générales de lisibilité et de traçabilité de 5.3.2.7 à 5.3.2.10 et de 6.5.4.14 à 6.5.4.17 ainsi qu'aux exigences spécifiques dans 9.1.4.5,

b) la cohérence interne de la Fiche de livraison.

Les résultats doivent être enregistrés dans un Rapport de Vérification du Déploiement.

9.1.4.20 Des mesures doivent être incluses dans le conditionnement du logiciel pour empêcher ou détecter les erreurs survenant pendant le stockage, le transfert, la transmission ou la duplication du code exécutable ou des données exécutables. Il est recommandé de coder le code exécutable (voir Tableau A.3 “Codes de détection d'erreur”) comme faisant partie de la vérification de l'intégrité du code dans le processus de chargement du logiciel.

9.2 Maintenance du logiciel

9.2.1 Objet

9.2.1.1 Assurer que le logiciel fonctionne comme requis, en préservant le niveau d'intégrité de la sécurité logicielle et la sûreté de fonctionnement lorsqu'on procède à des corrections, des améliorations ou des adaptations sur le logiciel lui-même. Voir aussi 6.6 “Contrôle des modifications et des évolutions” de la présente Norme européenne et la phase 13 “Modification et remise à niveau” dans l'EN 50126-1.

9.2.2 Documents en entrée

L'ensemble approprié des documents de conception, de développement et d'analyse.

9.2.3 Documents en sortie

1. Plan de Maintenance du Logiciel
2. Registres des Modifications du Logiciel
3. Enregistrements de la Maintenance du Logiciel
4. Rapport de Vérification de la Maintenance du Logiciel

9.2.4 Exigences

9.2.4.1 Bien que la présente Norme européenne ne soit pas censée être rétroactive car s'appliquant principalement aux nouveaux développements et ne s'appliquant dans sa totalité qu'à des logiciels existants si ceux-ci sont soumis à des modifications majeures, le 9.2 concernant la maintenance du logiciel s'applique à tous les changements, même mineurs. Par contre, il est hautement recommandé d'appliquer la totalité de la présente Norme européenne pendant les mises à niveau et la maintenance des logiciels existants.

9.2.4.2 Quel que soit le niveau d'intégrité de la sécurité logicielle, le fournisseur doit, avant de commencer tout travail de modification, décider si les actions de maintenance doivent être considérées comme majeures ou mineures ou si les méthodes existantes de maintenance du système sont appropriées. La décision doit être justifiée et enregistrée par le fournisseur et doit être soumise à l'évaluation du Chargé d'évaluation.

9.2.4.3 La maintenance doit être réalisée conformément aux directives exposées dans l'ISO/CEI 90003.

9.2.4.4 La maintenabilité doit être conçue comme un aspect intrinsèque du logiciel, en particulier en respectant les exigences de 7.3, 7.4 et 7.5. La série ISO/CEI 9126 doit être utilisée pour mettre en œuvre et vérifier le niveau minimum de maintenabilité.

9.2.4.5 Un Plan de Maintenance du Logiciel doit être rédigé sur la base des documents en entrée de 9.2.2.

L'exigence en 9.2.4.6 se rapporte au Plan de Maintenance du Logiciel.

9.2.4.6 Les procédures de maintenance du logiciel doivent être définies et enregistrées dans le Plan de Maintenance du Logiciel. Ces procédures doivent également traiter :

a) le contrôle du compte rendu des erreurs, des journaux des erreurs, des enregistrements de maintenance, de l'autorisation de modification et de la configuration logiciel/système ainsi que les techniques et mesures du Tableau A.10,

b) la vérification, la validation et l'évaluation de toute modification,

c) la définition de l'Autorité qui approuve le logiciel modifié, et

d) l'autorisation de la modification.

9.2.4.7 Un Enregistrement de la Maintenance du Logiciel doit être rédigé sur la base des documents en entrée de 9.2.2.

L'exigence 9.2.4.8 se rapporte à l'Enregistrement de Maintenance du Logiciel.

9.2.4.8 Un Enregistrement de la Maintenance du Logiciel doit être établi pour chaque logiciel avant sa première diffusion et doit être mis à jour. Outre les exigences de l'ISO/CEI 90003:2004 concernant les "registres et rapports de maintenance" (voir ISO/CEI 90003:2004, “Maintenance”), cet Enregistrement doit également inclure

a) des références à tous les registres des modifications de logiciel applicables à ce logiciel,

b) l'évaluation de l'impact du changement,

c) des scénarios de test pour les composants, incluant les données de test pour la revalidation et la non-régression, et

d) l'historique de la configuration logicielle.

9.2.4.9 Un Registre des Modifications du Logiciel doit être rédigé sur la base des documents en entrée de 9.2.2.

L'exigence 9.2.4.10 se rapporte au Registre des Modifications du Logiciel.

9.2.4.10 Un Registre des Modifications du Logiciel doit être établi pour chaque activité de maintenance. Cet enregistrement doit inclure

a) la demande de modification ou de changement, la version, la nature du défaut, le changement requis et la source du changement,

b) une analyse de l'impact de l'activité de maintenance sur le système global, y compris l'interaction matérielle, logicielle et humaine ainsi que des interactions avec l'environnement et interactions éventuelles,

c) la spécification détaillée de la modification ou du changement effectué(e), et

d) la revalidation, les tests de non-régression et la réévaluation de la modification ou du changement dans la limite requise par le niveau d'intégrité de la sécurité logicielle. La responsabilité en matière de revalidation peut varier d'un projet à l'autre, selon le niveau d'intégrité de la sécurité logicielle. L'impact de la modification ou du changement sur le processus de revalidation peut aussi se limiter à différents niveaux du système (seulement les composants modifiés, tous les composants affectés identifiés, le système complet). Le Plan de Validation du Logiciel doit donc aborder les deux problèmes, selon le niveau d'intégrité de la sécurité logicielle. Le degré d'indépendance de la revalidation doit être le même que celui de la validation.

9.2.4.11 Un Rapport de Vérification de la Maintenance du Logiciel doit être rédigé, sous la responsabilité du Chargé de vérification, sur la base des documents en entrée de 9.2.2.

Les exigences de 9.2.4.12 à 9.2.4.14 se rapportent au Rapport de Vérification de la Maintenance du Logiciel.

9.2.4.12 Une fois que le Plan de Maintenance du Logiciel a été défini, la vérification doit porter sur :

a) le fait que le Plan de Maintenance du Logiciel satisfait aux exigences générales de lisibilité et de traçabilité de 5.3.2.7 à 5.3.2.10 et de 6.5.4.14 à 6.5.4.17 ainsi qu'aux exigences spécifiques dans 9.2.4.6,

b) la cohérence interne du Plan de Maintenance du Logiciel.

9.2.4.13 Une fois que l'Enregistrement de Maintenance du Logiciel a été défini, la vérification doit porter sur :

a) le fait que l'Enregistrement de Maintenance du Logiciel satisfait aux exigences générales de lisibilité et de traçabilité de 5.3.2.7 à 5.3.2.10 et de 6.5.4.14 à 6.5.4.17 ainsi qu'aux exigences spécifiques dans 9.2.4.8,

b) la cohérence interne de l'Enregistrement de Maintenance du Logiciel.

9.2.4.14 Une fois que le Registre de Modifications du Logiciel a été défini, la vérification doit porter sur :

a) le fait que le Registre de Modifications du Logiciel satisfait aux exigences générales de lisibilité et de traçabilité de 5.3.2.7 à 5.3.2.10 et de 6.5.4.14 à 6.5.4.17 ainsi qu'aux exigences spécifiques dans 9.2.4.10,

b) la cohérence interne du Registre de Modifications du Logiciel.

9.2.4.15 Les activités de maintenance doivent être effectuées selon le Plan de Maintenance du Logiciel.

9.2.4.16 Les techniques et mesures issues du Tableau A.10 doivent être choisies. La combinaison choisie doit être justifiée comme un ensemble satisfaisant à 4.8 et 4.9.

9.2.4.17 La maintenance doit être réalisée avec un niveau de compétence, d'outils, de documentation, de planification et de gestion au moins égal à celui utilisé dans le développement initial du logiciel. Ceci doit s'appliquer également à la gestion de la configuration, au contrôle des modifications, au contrôle des documents et à l'indépendance des parties impliquées.

9.2.4.18 Le contrôle des fournisseurs externes, le compte rendu des problèmes et les actions correctives doivent être gérés en fonction des mêmes critères spécifiés dans les alinéas applicables de l'Assurance Qualité du Logiciel (6.5) que ceux appliqués pour le développement de nouveau logiciel.

9.2.4.19 Pour chaque problème consigné ou chaque mise en application communiquée, une analyse d'impact sur la sécurité doit être effectuée.

9.2.4.20 Pour les logiciels en maintenance, les actions d'atténuation à la hauteur du risque identifié doivent être prises afin d'assurer l'intégrité globale du système pendant que les problèmes communiqués font l'objet d'une investigation et sont corrigés.

Annexe A
(normative)

Critères de sélection des techniques et mesures

Les articles de la présente Norme européenne sont associés dans cette annexe aux tableaux d'articles (voir A.1, Tableau A.1 à Tableau A.11) pour illustrer les moyens d'assurer la conformité. Il existe aussi des tableaux de niveau inférieur, les tableaux détaillés (voir A.2, Tableau A.12 à Tableau A.23), qui développent certaines entrées des tableaux d'articles. Par exemple, “Modélisation” dans le Tableau A.2 est développé dans le Tableau A.17. Il existe aussi une Annexe D informative référencée à partir des tableaux d'articles.

Avec chaque technique ou mesure figurant dans les tableaux, il existe une exigence pour chaque niveau d'intégrité de la sécurité logicielle (SIL). Dans cette version du document, les exigences pour les niveaux 1 et 2 d'intégrité de la sécurité logicielle sont les mêmes pour chaque technique. De la même façon, chaque technique comporte les mêmes exigences aux niveaux 3 et 4 d'intégrité de la sécurité logicielle. Ces exigences peuvent être les suivantes :

'M' ce symbole signifie que l'utilisation d'une technique est obligatoire (*Mandatory*);

'HR' ce symbole signifie que cette technique ou mesure est Hautement Recommandée pour ce niveau d'intégrité de la sécurité. Si cette technique ou mesure n'est pas utilisée, la justification de l'utilisation d'autres techniques doit être détaillée dans le Plan d'Assurance Qualité du Logiciel ou dans un autre document référencé par le Plan d'Assurance Qualité du Logiciel;

'R' ce symbole signifie que la technique ou mesure est Recommandée pour ce niveau d'intégrité de la sécurité. Il s'agit d'un niveau de recommandation inférieur à un symbole 'HR' et il est possible de combiner ces techniques pour former une partie d'un ensemble conditionnel;

'-' ce symbole signifie que la technique ou mesure ne comporte aucune recommandation pour ou contre son utilisation;

'NR' ce symbole signifie que la technique ou mesure est catégoriquement Non Recommandée pour ce niveau d'intégrité de la sécurité. Si cette technique ou mesure est utilisée, la raison de cette décision doit être détaillée dans le Plan d'Assurance Qualité du Logiciel ou dans un autre document référencé par le Plan d'Assurance Qualité du Logiciel.

La combinaison de techniques ou de mesures est à préciser dans le Plan d'Assurance Qualité du Logiciel ou dans un autre document référencé par le Plan d'Assurance Qualité du Logiciel en sélectionnant une ou plusieurs techniques ou mesures, à moins que les notes jointes au tableau n'imposent d'autres exigences. Ces notes peuvent inclure la référence à des techniques approuvées ou à des combinaisons de techniques approuvées. Si de telles techniques ou combinaisons, y compris toutes les techniques obligatoires respectives, sont utilisées, le chargé d'évaluation doit accepter leur validité et ne doit se préoccuper que de leur application correcte. Si un ensemble différent de techniques est utilisé et peut être justifié, il est admis que le chargé d'évaluation les trouve acceptables.

A.1 Tableaux d'articles

Tableau A.1– Problèmes liés au cycle de vie et Documentation (5.3)

DOCUMENTATION	SIL 0	SIL 1	SIL 2	SIL 3	SIL 4
Planification					
1. Plan d'Assurance Qualité du Logiciel	HR	HR	HR	HR	HR
2. Rapport de vérification concernant l'Assurance Qualité du logiciel	HR	HR	HR	HR	HR
3. Plan de Gestion de la Configuration du Logiciel	HR	HR	HR	HR	HR
4. Plan de Vérification du Logiciel	HR	HR	HR	HR	HR
5. Plan de Validation du Logiciel	HR	HR	HR	HR	HR
Exigences relatives au logiciel					
6. Spécification des Exigences du Logiciel	HR	HR	HR	HR	HR
7. Spécification de Tests d'Ensemble du Logiciel	HR	HR	HR	HR	HR
8. Rapport de Vérification des Exigences du Logiciel	HR	HR	HR	HR	HR
Architecture et Conception					
9. Spécification de l'Architecture du Logiciel	HR	HR	HR	HR	HR
10. Spécification de la Conception du Logiciel	HR	HR	HR	HR	HR
11. Spécifications d'Interface du Logiciel	HR	HR	HR	HR	HR
12. Spécification de Tests d'Intégration du Logiciel	HR	HR	HR	HR	HR
13. Spécification de Tests d'Intégration Logiciel/Matériel	HR	HR	HR	HR	HR
14. Rapport de Vérification de l'Architecture et de la Conception du Logiciel	HR	HR	HR	HR	HR
Conception du Composant					
15. Spécification de la Conception des Composants logiciel	R	HR	HR	HR	HR
16. Spécification de Tests de Composant Logiciel	R	HR	HR	HR	HR
17. Rapport de Vérification de la Conception des Composants Logiciels	R	HR	HR	HR	HR
Réalisation et Test des composants					
18. Code Source du Logiciel et Documentation de Support	HR	HR	HR	HR	HR
19. Rapport de Tests des Composants Logiciels	R	HR	HR	HR	HR
20. Rapport de Vérification du Code Source du Logiciel	HR	HR	HR	HR	HR
Intégration					
21. Rapport de Tests d'Intégration du Logiciel	HR	HR	HR	HR	HR
22. Rapport de Tests d'Intégration Logiciel/Matériel	HR	HR	HR	HR	HR
23. Rapport de Vérification de l'Intégration du Logiciel	HR	HR	HR	HR	HR
Tests d'Ensemble du Logiciel / Validation Finale					
24. Rapport de Tests d'Ensemble du Logiciel	HR	HR	HR	HR	HR
25. Rapport de Validation du Logiciel	HR	HR	HR	HR	HR
26. Rapport de Validation des Outils	R	HR	HR	HR	HR
27. Fiche de livraison	HR	HR	HR	HR	HR

Tableau A.1– Problèmes liés au cycle de vie et Documentation (5.3) *(suite)*

DOCUMENTATION	SIL 0	SIL 1	SIL 2	SIL 3	SIL 4
Systèmes configurés par des données/algorithmes d'application					
28. Spécification des Exigences de l'Application	HR	HR	HR	HR	HR
29. Plan de Préparation de l'Application (voir NOTE 2)	HR	HR	HR	HR	HR
30. Spécification de Tests de l'Application (voir NOTE 2)	HR	HR	HR	HR	HR
31. Architecture et Conception de l'Application (voir NOTE 2)	HR	HR	HR	HR	HR
32. Rapport de Vérification de la Préparation de l'Application	HR	HR	HR	HR	HR
33. Rapport de Tests d'Application	HR	HR	HR	HR	HR
34. Code Source des données/algorithmes d'application	HR	HR	HR	HR	HR
35. Rapport de vérification des données/algorithmes d'application	HR	HR	HR	HR	HR
Déploiement du logiciel					
36. Plan de livraison et de déploiement du Logiciel	R	HR	HR	HR	HR
37. Manuel de Déploiement du Logiciel	R	HR	HR	HR	HR
38. Fiche de livraison	HR	HR	HR	HR	HR
39. Enregistrements du Déploiement	R	HR	HR	HR	HR
40. Rapport de Vérification du Déploiement	R	HR	HR	HR	HR
maintenance du logiciel					
41. Plan de Maintenance du Logiciel	R	HR	HR	HR	HR
42. Registres des Modifications du Logiciel	HR	HR	HR	HR	HR
43. Enregistrements de la Maintenance du Logiciel	R	HR	HR	HR	HR
44. Rapport de Vérification de la Maintenance du Logiciel	R	HR	HR	HR	HR
Évaluation du logiciel					
45. Plan d'Évaluation du Logiciel	R	HR	HR	HR	HR
46. Rapport d'Évaluation du Logiciel	R	HR	HR	HR	HR

NOTE 1 Selon 5.3.2.11 et 5.3.2.12, les documents peuvent être regroupés différemment.

NOTE 2 Le fait que les documents 29, 30 et 31 soient HR ou R dépend de l'importance définie dans le processus et de l'endroit où la vérification a lieu. Par exemple, des données peuvent juste avoir besoin d'être vérifiées mais testées dans le domaine du système alors que des propriétés plus fonctionnelles ont besoin à la fois de test et de vérification. Dans ce cas, HR a été marqué mais peut être un R facultatif.

Tableau A.2 – Spécification des Exigences du Logiciel (7.2)

TECHNIQUE/MESURE	Réf.	SIL 0	SIL 1	SIL 2	SIL 3	SIL 4
1. Méthodes formelles (basées sur une approche mathématique)	D.28	-	R	R	HR	HR
2. Modélisation	Tableau A.17	R	R	R	HR	HR
3. Méthodologie structurée	D.52	R	R	R	HR	HR
4. Tables de Décision	D.13	R	R	R	HR	HR

Exigences :

1) La Spécification des Exigences du Logiciel doit inclure une description du problème en langage naturelle et toutes les notations formelles ou non formelles nécessaires.

2) Le tableau reproduit des exigences supplémentaires permettant d'obtenir une spécification claire et précise. Une ou plusieurs de ces techniques doivent être sélectionnées pour satisfaire au niveau d'intégrité de la sécurité logicielle utilisé.

Tableau A.3 – Architecture du Logiciel (7.3)

TECHNIQUE/MESURE	Réf.	SIL 0	SIL 1	SIL 2	SIL 3	SIL 4
1. Programmation défensive	D.14	-	HR	HR	HR	HR
2. Détection des défauts & diagnostic	D.26	-	R	R	HR	HR
3. Codes correcteurs d'erreurs	D.19	-	-	-	-	-
4. Codes de détection d'erreurs	D.19	-	R	R	HR	HR
5. Programmation par assertion	D.24	-	R	R	HR	HR
6. Techniques de sécurité contrôlée (safety bag)	D.47	-	R	R	R	R
7. Programmation diversifiée	D.16	-	R	R	HR	HR
8. Bloc de rattrapage	D.44	-	R	R	R	R
9. Rattrapage par régression	D.5	-	NR	NR	NR	NR
10. Rattrapage par progression	D.30	-	NR	NR	NR	NR
11. Mécanismes de rattrapage par réexécution	D.46	-	R	R	R	R
12. Mémorisation des cas exécutés	D.36	-	R	R	HR	HR
13. Intelligence artificielle - Correction des défauts	D.1	-	NR	NR	NR	NR
14. Reconfiguration dynamique du logiciel	D.17	-	NR	NR	NR	NR
15. Analyse des Effets des Erreurs du Logiciel	D.25	-	R	R	HR	HR
16. Dégradation contrôlée	D.31	-	R	R	HR	HR
17. Masquage d'informations	D.33	-	-	-	-	-
18. Encapsulation d'informations	D.33	R	HR	HR	HR	HR
19. Interface entièrement définie	D.38	HR	HR	HR	M	M
20. Méthodes formelles	D.28	-	R	R	HR	HR
21. Modélisation	Tableau A.17	R	R	R	HR	HR
22. Méthodologie structurée	D.52	R	HR	HR	HR	HR
23. Modélisation prise en charge par conception assistée par ordinateur et par les outils de spécification	Tableau A.17	R	R	R	HR	HR

Exigences :

1) Les combinaisons de techniques approuvées applicables aux niveaux 3 et 4 d'intégrité de la sécurité logicielle sont les suivantes :

   a) 1, 7, 19, 22 et une choisie entre 4, 5, 12 et 21;

   b) 1, 4, 19, 22 et une choisie entre 2, 5, 12, 15 et 21.

2) Les combinaisons de techniques approuvées applicables aux niveaux 1 et 2 d'intégrité de la sécurité logicielle sont les suivantes : 1, 19, 22 et une choisie entre 2, 4, 5, 7, 12, 15 et 21.

3) Il est permis de définir certains de ces problèmes au niveau du système.

4) Des codes de détection d'erreurs peuvent être utilisés selon les exigences de la norme EN 50159.

NOTE La technique/mesure 19 concerne les Interfaces externes.

Tableau A.4– Conception et mise en œuvre du logiciel (7.4)

TECHNIQUE/MESURE	Réf.	SIL 0	SIL 1	SIL 2	SIL 3	SIL 4
1. Méthodes formelles	D.28	-	R	R	HR	HR
2. Modélisation	Tableau A.17	R	HR	HR	HR	HR
3. Méthodologie structurée	D.52	R	HR	HR	HR	HR
4. Approche modulaire	D.38	HR	M	M	M	M
5. Composants	Tableau A.20	HR	HR	HR	HR	HR
6. Règles de conception et de codage	Tableau A.12	HR	HR	HR	M	M
7. Programmes analysables	D.2	HR	HR	HR	HR	HR
8. Langage de programmation fortement typé	D.49	R	HR	HR	HR	HR
9. Programmation structurée	D.53	R	HR	HR	HR	HR
10. Langage de programmation	Tableau A.15	R	HR	HR	HR	HR
11. Sous-ensemble de langage	D.35	-	-	-	HR	HR
12. Programmation orientée objet	Tableau A.22 D.57	R	R	R	R	R
13. Programmation procédurale	D.60	R	HR	HR	HR	HR
14. Métaprogrammation	D.59	R	R	R	R	R

Exigences :

1) Une combinaison approuvée de techniques applicables aux niveaux 3 et 4 d'intégrité de la sécurité logicielle est 4, 5, 6, 8 et une choisie entre 1 et 2.

2) Une combinaison approuvée de techniques applicables aux niveaux 1 et 2 d'intégrité de la sécurité logicielle est 3, 4, 5, 6 et une choisie entre 8, 9 et 10.

3) La métaprogrammation doit être limitée à la production du code du logiciel source avant compilation.

Tableau A.5 – Vérification et Tests (6.2 et 7.3)

TECHNIQUE/MESURE	Réf.	SIL 0	SIL 1	SIL 2	SIL 3	SIL 4
1. Preuve formelle	D.29	-	R	R	HR	HR
2. Analyse statique	Tableau A.19	-	HR	HR	HR	HR
3. Analyse et tests dynamiques	Tableau A.13	-	HR	HR	HR	HR
4. Métriques	D.37	-	R	R	R	R
5. Traçabilité	D.58	R	HR	HR	M	M
6. Analyse des Effets des Erreurs du Logiciel	D.25	-	R	R	HR	HR
7. Couverture de test pour le code	Tableau A.21	R	HR	HR	HR	HR
8. Tests fonctionnels et boîte noire	Tableau A.14	HR	HR	HR	M	M
9. Tests de performance	Tableau A.18	-	HR	HR	HR	HR
10. Tests d'interface	D.34	HR	HR	HR	HR	HR

Exigences :

1) Pour les niveaux 3 et 4 d'intégrité de la sécurité logicielle, la combinaison approuvée de techniques est 3, 5, 7, 8 et une choisie entre 1, 2 et 6.

2) Pour les niveaux 1 et 2 d'intégrité de la sécurité logicielle, la combinaison approuvée de techniques est 5 avec une choisie entre 2, 3 et 8.

NOTE 1 Les techniques/mesures 1, 2, 4, 5, 6 et 7 concernent les activités de vérification.

NOTE 2 Les techniques/mesures 3, 8, 9 et 10 concernent les activités de tests.

Tableau A.6 – Intégration (7.6)

TECHNIQUE/MESURE	Réf.	SIL 0	SIL 1	SIL 2	SIL 3	SIL 4
1. Tests fonctionnels et boîte noire	Tableau A.14	HR	HR	HR	HR	HR
2. Tests de performance	Tableau A.18	-	R	R	HR	HR

Tableau A.7– Tests d'Ensemble du Logiciel (6.2et 7.7)

TECHNIQUE/MESURE	Réf.	SIL 0	SIL 1	SIL 2	SIL 3	SIL 4
1. Tests de performance	Tableau A.18	-	HR	HR	M	M
2. Tests fonctionnels et boîte noire	Tableau A.14	HR	HR	HR	M	M
3. Modélisation	Tableau A.17	-	R	R	R	R
Exigence : 1) Pour le niveau d'intégrité de sécurité logicielle 1 ou 2, une combinaison approuvée de techniques est 1 et 2.						

Tableau A.8 – Techniques d'analyse logicielle (6.3)

TECHNIQUE/MESURE	Réf.	SIL 0	SIL 1	SIL 2	SIL 3	SIL 4
1. Analyse logicielle statique	D.13 D.37 Tableau A.19	R	HR	HR	HR	HR
2. Analyse logicielle dynamique	Tableau A.13 Tableau A.14	-	R	R	HR	HR
3. Schémas de cause et de conséquence	D.6	R	R	R	R	R
4. Analyse par arbre des événements	D.22	-	R	R	R	R
5. Analyse des Effets des Erreurs du Logiciel	D.25	-	R	R	HR	HR
Exigence : 1) Une ou plusieurs de ces techniques doivent être sélectionnées pour satisfaire au niveau d'intégrité de la sécurité logicielle utilisé.						

Tableau A.9 – Assurance Qualité du logiciel (6.5)

TECHNIQUE/MESURE	Réf.	SIL 0	SIL 1	SIL 2	SIL 3	SIL 4
1. Accréditée par EN ISO 9001	7.1	R	HR	HR	HR	HR
2. Conforme à l'EN ISO 9001	7.1	M	M	M	M	M
3. Conforme à la norme ISO/CEI 90003	7.1	R	R	R	R	R
4. Système qualité de l'entreprise	7.1	M	M	M	M	M
5. Gestion de la configuration du logiciel	D.48	M	M	M	M	M
6. Listes de contrôle	D.7	R	HR	HR	HR	HR
7. Traçabilité	D.58	R	HR	HR	M	M
8. Enregistrement et analyse des données	D.12	HR	HR	HR	M	M
Exigences : 1) Ce tableau doit être appliqué aux différents rôles et à toutes les phases.						

Tableau A.10 – Maintenance du Logiciel (9.2)

TECHNIQUE/MESURE	Réf.	SIL 0	SIL 1	SIL 2	SIL 3	SIL 4
1. Analyse d'impact	D.32	R	HR	HR	M	M
2. Enregistrement et analyse des données	D.12	HR	HR	HR	M	M

Tableau A.11 – Techniques de préparation des données (8.4)

TECHNIQUE/MESURE	Réf.	SIL 0	SIL 1	SIL 2	SIL 3	SIL 4
1. Méthodes de spécification en tableaux	D.68	R	R	R	R	R
2. Langage spécifique à l'application	D.69	R	R	R	R	R
3. Simulation	D.42	R	HR	HR	HR	HR
4. Tests fonctionnels	D.42	M	M	M	M	M
5. Listes de contrôle	D.7	R	HR	HR	M	M
6. Inspection de Fagan	D.23	-	R	R	R	R
7. Examens formels de la conception	D.56	R	HR	HR	HR	HR
8. Preuve formelle du caractère correct (des données)	D.29	-	-	-	HR	HR
9. Revue de projet structurée	D.56	R	R	R	HR	HR

Exigences :

1) Pour le niveau d'intégrité de sécurité logicielle 1 ou 2, une combinaison approuvée de techniques est 1 et 4.

2) Pour les niveaux 3 et 4 d'intégrité de la sécurité logicielle, les combinaisons approuvées de techniques sont 1, 4, 5 et 7 ou 2, 3 et 6.

NOTE La description de la référence D.29 concerne les programmes alors que la technique 8 dans ce contexte s'applique à la preuve formelle du caractère correct des données.

A.2 Tableaux détaillés

Tableau A.12 – Normes de codage

TECHNIQUE/MESURE	Réf.	SIL 0	SIL 1	SIL 2	SIL 3	SIL 4
1. Norme de codage	D.15	HR	HR	HR	M	M
2. Guide du style de codage	D.15	HR	HR	HR	HR	HR
3. Pas d'objets dynamiques	D.15	-	R	R	HR	HR
4. Pas de variables dynamiques	D.15	-	R	R	HR	HR
5. Usage limité des pointeurs	D.15	-	R	R	R	R
6. Usage limité de la récursivité	D.15	-	R	R	HR	HR
7. Pas de branchements inconditionnels	D.15	-	HR	HR	HR	HR
8. Taille et complexité limitées des fonctions, sous-programmes et méthodes	D.38	HR	HR	HR	HR	HR
9. Stratégie de Point d'entrée/sortie pour les fonctions, sous-programmes et méthodes	D.38	R	HR	HR	HR	HR
9. Nombre limité de paramètres de sous-programme	D.38	R	R	R	R	R
10. Utilisation limitée de variables globales	D.38	HR	HR	HR	M	M
Exigence : 1) Il est accepté que les techniques 3, 4 et 5 soient présentes comme partie intégrante d'un compilateur ou d'un traducteur validé.						

Tableau A.13 – Analyse et Tests dynamiques

TECHNIQUE/MESURE	Réf.	SIL 0	SIL 1	SIL 2	SIL 3	SIL 4
1. Exécution d'un dossier de tests à partir d'une analyse des valeurs aux limites	D.4	-	HR	HR	HR	HR
2. Exécution d'un dossier de tests à partir de la supposition d'erreurs	D.20	R	R	R	R	R
3. Exécution d'un modèle de tests à partir de l'insertion d'erreurs	D.21	-	R	R	R	R
4. Modélisation des performances	D.39	-	R	R	HR	HR
5. Tests de classes d'équivalence et de partition d'entrée	D.18	R	R	R	HR	HR
6. Tests structurels	D.50	-	R	R	HR	HR
Exigence : 1) Pour les dossiers de tests, l'analyse se fait au niveau du sous-système et repose sur la spécification et/ou sur la spécification et le code.						

Tableau A.14 – Test fonctionnel/boîte noire

TECHNIQUE/MESURE	Réf.	SIL 0	SIL 1	SIL 2	SIL 3	SIL 4
1. Exécution d'un dossier de tests à partir des schémas de cause et de conséquence	D.6	-	-	-	R	R
2. Prototypage/Animation	D.43	-	-	-	R	R
3. Analyse des valeurs aux limites	D.4	R	HR	HR	HR	HR
4. Tests de classes d'équivalence et de partition d'entrée	D.18	R	HR	HR	HR	HR
5. Simulation du processus	D.42	R	R	R	R	R
Exigence : 1) L'exhaustivité de la simulation dépendra de la limite du niveau d'intégrité de la sécurité logicielle, de la complexité et de l'application.						

Tableau A.15 – Langages de programmation textuels

TECHNIQUE/MESURE	Réf.	SIL 0	SIL 1	SIL 2	SIL 3	SIL 4
1. ADA	D.54	R	HR	HR	HR	HR
2. MODULA-2	D.54	R	HR	HR	HR	HR
3. PASCAL	D.54	R	HR	HR	HR	HR
4. C ou C++	D.54 D.35	R	R	R	R	R
5. PL/M	D.54	R	R	R	NR	NR
6. BASIC	D.54	R	NR	NR	NR	NR
7. Assembleur	D.54	R	R	R	R	R
8. C#	D.54 D.35	R	R	R	R	R
9. JAVA	D.54 D.35	R	R	R	R	R
10. Liste d'instructions	D.54	R	R	R	R	R

Exigences :

1) Le choix des langages doit être fondé sur les exigences données en 6.7 et 7.3.

2) Il n'y a aucune exigence de justifier les décisions prises d'exclure des langages de programmation spécifiques.

NOTE 1 Pour des informations relatives à l'évaluation de la pertinence d'un langage de programmation, voir la rubrique D.54 "Langages de programmation adaptés".

NOTE 2 Si un langage spécifique ne figure pas dans le tableau, il n'est pas automatiquement exclu. Il convient toutefois qu'il se conforme à D.54.

NOTE 3 Les systèmes d'exécution associés à des langages sélectionnés qui sont nécessaires pour exécuter des programmes d'application conviennent encore d'être justifiés quant à leur utilisation selon le niveau d'intégrité de sécurité logicielle.

Tableau A.16 – Langages diagrammatiques pour algorithmes d'application

TECHNIQUE/MESURE	Réf.	SIL 0	SIL 1	SIL 2	SIL 3	SIL 4
1. Diagrammes fonctionnels	D.63	R	R	R	R	R
2. Graphes séquentiels de fonction	D.61	-	HR	HR	HR	HR
3. Schémas à contacts	D.62	R	R	R	R	R
4. Graphiques d'états	D.64	R	HR	HR	HR	HR

Tableau A.17 – Modélisation

TECHNIQUE/MESURE	Réf.	SIL 0	SIL 1	SIL 2	SIL 3	SIL 4
1. Modélisation de données	D.65	R	R	R	HR	HR
2. Organigrammes des données	D.11	-	R	R	HR	HR
3. Diagrammes de flux de commande	D.66	R	R	R	HR	HR
4. Automates à état finis/Schémas de transition d'état	D.27	-	HR	HR	HR	HR
5. Réseaux de Pétri temporels	D.55	-	R	R	HR	HR
6. Tables de décision/de vérité	D.13	R	R	R	HR	HR
7. Méthodes formelles	D.28	-	R	R	HR	HR
8. Modélisation des performances	D.39	-	R	R	HR	HR
9. Prototypage/Animation	D.43	-	R	R	R	R
10. Schémas de structure	D.51	-	R	R	HR	HR
11. Diagrammes de séquence	D.67	R	HR	HR	HR	HR
Exigences : 1) Une ligne directrice de modélisation doit être définie et utilisée. 2) Au moins l'une des techniques HR doit être choisie.						

Tableau A.18 – Tests de Performance

TECHNIQUE/MESURE	Réf.	SIL 0	SIL 1	SIL 2	SIL 3	SIL 4
1. Tests en avalanche/en surcharge	D.3	-	R	R	HR	HR
2. Temps de réponse et contraintes de place mémoire	D.45	-	HR	HR	HR	HR
3. Exigences en matière de performance	D.40	-	HR	HR	HR	HR

Tableau A.19 – Analyse statique

TECHNIQUE/MESURE	Réf.	SIL 0	SIL 1	SIL 2	SIL 3	SIL 4
1. Analyse des valeurs aux limites	D.4	-	R	R	HR	HR
2. Listes de contrôle	D.7	-	R	R	R	R
3. Analyse de flux de contrôle	D.8	-	HR	HR	HR	HR
4. Analyse du flot de données	D.10	-	HR	HR	HR	HR
5. Supposition d'erreurs	D.20	-	R	R	R	R
6. Revues/Examens de la conception	D.56	HR	HR	HR	HR	HR

Tableau A.20 – Composants

TECHNIQUE/MESURE	Réf.	SIL 0	SIL 1	SIL 2	SIL 3	SIL 4
1. Masquage d'informations	D.33	-	-	-	-	-
2. Encapsulation d'informations	D.33	R	HR	HR	HR	HR
3. Limite du nombre de paramètres	D.38	R	R	R	R	R
4. Interface entièrement définie	D.38	R	HR	HR	M	M

Exigence :

1) Le masquage et l'encapsulation d'informations sont seulement hautement recommandés s'il n'existe pas de stratégie générale pour l'accès aux données.

NOTE La technique/mesure 4 concerne les Interfaces internes.

Tableau A.21 – Couverture des tests pour le code

Critère de couverture des tests	Réf.	SIL 0	SIL 1	SIL 2	SIL 3	SIL 4
1. Instruction	D.50	R	HR	HR	HR	HR
2. Branchement	D.50	-	R	R	HR	HR
3. Conditions composées	D.50	-	R	R	HR	HR
4. Flux de données	D.50	-	R	R	HR	HR
5. Chemin	D.50	-	R	R	HR	HR

Exigences :

1) Pour chaque SIL, une mesure quantifiée de couverture doit être développée pour les tests entrepris. Cela peut soutenir le jugement sur la confiance acquise dans les tests et la nécessité de techniques supplémentaires.

2) Pour les SIL 3 ou 4, il convient de mesurer la couverture des tests au niveau composant selon

   - 2 et 3; ou
   - 2 et 4; ou
   - 5

   ou il convient de mesurer la couverture des tests au niveau d'intégration selon l'un ou plusieurs choisis parmi 2, 3, 4 ou 5.

3) D'autres critères peuvent être utilisés pour la couverture des tests, étant donné que cela peut être justifié. Ces critères dépendent de l'architecture du logiciel (voir Tableau A.3) Tableau A.3) et du langage de programmation (voir Tableau A.15 et Tableau A.16).

4) Tout code qu'il n'est pas pratique de tester doit être démontré correct en utilisant une technique appropriée, par exemple une analyse statique issue du Tableau A.19.

NOTE 1 La couverture des instructions est automatiquement obtenue par les éléments 2 à 5.

NOTE 2 Les critères de couverture des tests dans ce tableau sont utilisés pour des tests structurels (basés sur code, boîte blanche). Les techniques/mesures pour les tests fonctionnels (basés sur spécification, boîte noire) sont données au Tableau A.14.

NOTE 3 Il est en général difficile d'obtenir un haut pourcentage de couverture. L'utilisation de l'exécution de scénarios de test à partir de valeurs aux limites (D.4) et des tests de classes d'équivalence et de partition d'entrée (D.18) peut permettre d'obtenir une couverture suffisante avec un plus petit nombre de tests.

NOTE 4 La différence entre 2 et 3 dépend dans la pratique du niveau du langage de programmation et de l'utilisation de conditions composées. Lorsque des conditions simples sont utilisées par exemple comme résultat d'une compilation, 2 et 3 sont considérées comme étant identiques.

Tableau A.22 – Architecture de logiciel orienté objet

TECHNIQUE/MESURE	Réf.	SIL 0	SIL 1	SIL 2	SIL 3	SIL 4
1. Traçabilité du concept du domaine d'application aux classes de l'architecture	-	R	R	R	HR	HR
2. Utilisation de cadres adaptés, de combinaisons de classes et de modèles de conception communément utilisées	-	R	R	R	HR	HR
3. Conception détaillée orientée objet	Tableau A.23	R	R	R	HR	HR

Exigences :

1) Lorsque des cadres et des modèles de conception existants sont utilisés, les exigences relatives aux logiciels préexistants s'appliquent à ces cadres et modèles

NOTE 1 L'approche orientée objet présente les informations différemment des approches procédurales; la liste ci-après contient les recommandations qui requièrent une considération spécifique :

- compréhension des hiérarchies de classes et identification des fonction(s) logicielle(s) dont l'exécution aura lieu sur invocation d'une méthode donnée (y compris lors de l'utilisation d'une bibliothèque de classes existante);
- tests structurels (Tableau A.13).

La traçabilité depuis le domaine d'application jusqu'à l'architecture de classes est moins importante.

NOTE 2 Pour une partie du logiciel visé, il pourrait exister un cadre issu de logiciels préexistants qui a résolu avec succès une tâche similaire et qui est bien connu du personnel de développement. L'utilisation de ce cadre est alors recommandée.

Tableau A.23 – Conception détaillée orientée objet

TECHNIQUE/MESURE	Réf.	SIL 0	SIL 1	SIL 2	SIL 3	SIL 4
1. Il convient que les classes n'aient qu'un seul objectif	-	R	R	R	HR	HR
2. Héritage utilisé uniquement si la classe dérivée est un affinement de sa classe de base	-	R	HR	HR	HR	HR
3. Profondeur de l'héritage limitée par des normes de codage	-	R	R	R	HR	HR
4. Neutralisation d'opérations (méthodes) sous contrôle strict	-	R	R	R	HR	HR
5. Héritage multiple utilisé uniquement pour les classes d'interface	-	R	HR	HR	HR	HR
6. Héritage à partir de classes inconnues	-	-	-	-	NR	NR

Exigences :

1) Une classe donnée est caractérisée par le fait d'avoir une responsabilité donnée, c'est-à-dire la prise en charge de données étroitement liées et des opérations sur ces données.

2) Il faut veiller à éviter les dépendances circulaires entre objets.

Annexe B
(normative)

Principaux rôles et responsabilités relatifs au logiciel

Tableau B.1 Gestionnaire des Exigences

Tableau B.2 (Designer) Concepteur

Tableau B.3 Fiabilité, disponibilité, maintenabilité et sécurité

Tableau B.4 (Tester) Chargé des tests

Tableau B.5 Chargé de vérification

Tableau B.6 Chargé d'intégration

Tableau B.7 (Validator) Chargé de validation

Tableau B.8 Chargé d'évaluation

Tableau B.9 Chef de projet

Tableau B.10 Gestionnaire de la Configuration

Tableau B.1 — Spécification du Rôle du Gestionnaire des Exigences

Rôle : Gestionnaire des Exigences
Responsabilités : 1. doit être chargé de spécifier les exigences relatives au logiciel 2. doit être le propriétaire de la Spécification des Exigences du Logiciel 3. doit établir et maintenir la traçabilité vers et depuis les exigences du niveau système 4. doit assurer que les exigences relatives aux spécifications et aux logiciels sont prises en compte dans la gestion des modifications et de la configuration, y compris l'état, la version et le statut d'autorisation 5. doit assurer la cohérence et la complétude dans la Spécification des Exigences du Logiciel (avec référence aux exigences de l'utilisateur et de l'environnement final d'application) 6. doit développer et maintenir les documents d'exigences relatives au logiciel
Principales compétences : 1. doit être compétent en ingénierie des exigences 2. doit avoir l’expérience du domaine d'application 3. doit avoir l’expérience des critères de sécurité dans le domaine d'application 4. doit comprendre le rôle global du système et l'environnement d'application 5. doit comprendre les techniques analytiques et leurs résultats 6. doit comprendre les réglementations applicables 7. doit comprendre les exigences de l'EN 50128

Tableau B.2 — Spécification du Rôle du Concepteur

Rôle : Concepteur
Responsabilités : 1. doit transformer les exigences spécifiées relatives au logiciel en solutions acceptables 2. doit être propriétaire des solutions d'architecture et de conception descendante 3. doit définir ou sélectionner les méthodes de conception et les outils 4. doit appliquer les principes et normes appropriés de conception de la sécurité 5. doit développer des spécifications de composants, le cas échéant 6. doit maintenir la traçabilité vers et depuis les exigences spécifiées relatives au logiciel 7. doit développer et maintenir la documentation de la conception 8. doit assurer que les documents de conception sont pris en compte par la gestion des modifications et de la configuration
Principales compétences : 1. doit être compétent en l'ingénierie appropriée au domaine d'application 2. doit être compétent en matière de principes de conception de la sécurité 3. doit être compétent en analyse de conception et en méthodologies des tests de conception 4. doit être capable de travailler dans les limites des contraintes de conception dans un environnement donné 5. doit être compétent pour comprendre le domaine du problème 6. doit comprendre toutes les contraintes imposées par la plateforme matérielle, le système d'exploitation et les systèmes d'interfaçage 7. doit comprendre les parties correspondantes de l'EN 50128

Tableau B.3 — Spécification du Rôle du Réalisateur

Rôle : Réalisateur
Responsabilités : 1. doit transformer les solutions de conception en données/code source/autres représentations de conception 2. doit transformer le code source en code exécutable/autre représentation de conception 3. doit appliquer les principes de conception de la sécurité 4. doit appliquer les normes spécifiées de préparation/codage des données 5. doit mener à bien l'analyse pour vérifier le résultat intermédiaire 6. doit intégrer le logiciel sur la machine cible 7. doit développer et maintenir les documents de mise en œuvre comprenant les méthodes, types de données et listings appliqués 8. doit maintenir la traçabilité vers et depuis la conception 9. doit maintenir le(s) code/données généré(es) ou modifié(es) sous le contrôle des modifications et de la configuration
Principales compétences : 1. doit être compétent en l'ingénierie appropriée au domaine d'application 2. doit être compétent dans le langage de réalisation et les outils 3. doit être capable d'appliquer les normes de codage et styles de programmation spécifiés 4. doit comprendre toutes les contraintes imposées par la plateforme matérielle, le système d'exploitation et les systèmes d'interfaçage 5. doit comprendre les parties correspondantes de l'EN 50128

Tableau B.4 — Spécification du Rôle du Chargé des tests

Rôle : Chargé des tests
Responsabilités : 1. doit assurer que les activités de tests sont planifiées 2. doit développer les spécifications des tests (objectifs et scénarios) 3. doit assurer la traçabilité des objectifs de test par rapport aux exigences spécifiées relatives au logiciel et celle des scénarios de test par rapport aux objectifs spécifiés des tests 4. doit assurer que les tests planifiés sont réalisés les tests spécifiés menés à bien 5. doit identifier les écarts par rapport aux résultats attendus et les consigner dans des rapports de tests 6. doit communiquer des écarts avec l'organisme compétent de gestion des modifications en vue de l'évaluation et d'une prise de décision 7. doit consigner les résultats dans des rapports 8. doit sélectionner les équipements de test du logiciel
Principales compétences : 1. doit être compétent dans le domaine où les tests sont effectués, par exemple : exigences relatives au logiciel, données, code, etc. 2. doit être compétent en diverses approches/méthodologies de test et de vérification et être capable d'identifier la méthode la plus adaptée dans un contexte donné 3. doit être capable de déduire des scénarios de test à partir des spécifications données 4. doit avoir une capacité de réflexion analytique et un bon sens de l'observation 5. doit comprendre les parties correspondantes de l'EN 50128

Tableau B.5 — Spécification du Rôle du Chargé de vérification

Rôle : Chargé de vérification
Responsabilités : 1. doit développer un Plan de Vérification du Logiciel (qui peut inclure des questions relatives à la qualité) mentionnant ce qui nécessite une vérification et quel type de processus (par exemple : revue, analyse, etc.) et de test est requis comme preuve 2. doit vérifier l'adéquation (complétude, pertinence et traçabilité) des preuves documentées à partir de revue, d'intégration et de tests avec les objectifs de vérification spécifiés 3. doit identifier les anomalies, les évaluer en termes de risque (impact); les consigner et les communiquer à l'organisme compétent de gestion des modifications en vue de l'évaluation et d'une prise de décision 4. doit gérer le processus de vérification (revue, intégration et tests) et assurer l'indépendance des activités selon les besoins 5. doit développer et maintenir des enregistrements sur les activités de vérification 6. doit rédiger un Rapport de vérification indiquant le résultat des activités de vérification
Principales compétences : 1. doit être compétent dans le domaine où la vérification est effectuée, par exemple : exigences relatives au logiciel, données, code, etc. 2. doit être compétent en diverses approches/méthodologies de vérification et être capable d'identifier la méthode ou combinaison de méthodes la plus adaptée dans un contexte donné 3. doit être capable de déduire les types de vérification des spécifications données 4. doit avoir une capacité de réflexion analytique et un bon sens de l'observation 5. doit comprendre les parties correspondantes de l'EN 50128

Tableau B.6 — Spécification du Rôle du Chargé d'intégration

Rôle : Chargé d'intégration
Responsabilités : 1. doit gérer le processus d'intégration en utilisant la ligne de base du logiciel 2. doit développer la Spécification de Tests d'Intégration Logiciel et Logiciel/Matériel. pour les composants logiciels sur la base des spécifications et de l'architecture des composants du Concepteur indiquant quels sont les composants d'entrée nécessaires, la séquence d'activités d'intégration et les composants intégrés résultants 3. doit développer et maintenir des enregistrements sur les activités d'intégration 4. doit identifier les anomalies d'intégration, les consigner et les communiquer à l'organisme compétent de gestion des modifications en vue de l'évaluation et d'une prise de décision 5. doit rédiger un rapport d'intégration de composants et du système global indiquant le résultat de l'intégration
Principales compétences : 1. doit être compétent dans le domaine où l'intégration des composants est effectuée, par exemple les langages de programmation concernés, les interfaces logicielles, les systèmes d'exploitation, les données, les plateformes, les codes, etc. 2. doit être compétent en diverses approches/méthodologies d'intégration et être capable d'identifier la méthode ou combinaison de méthodes la plus adaptée dans un contexte donné 3. doit être compétent pour comprendre la conception et la fonctionnalité requises à divers niveaux intermédiaires 4. doit être capable de déduire les types de test d'intégration à partir d'un ensemble de fonctions intégrées 5. doit avoir une capacité de réflexion analytique et un bon sens de l'observation qui permettent la compréhension du système dans sa globalité 6. doit comprendre les parties correspondantes de l'EN 50128

Tableau B.7 — Spécification du Rôle du Chargé de Chargé de validation

Rôle : Chargé de validation
Responsabilités : 1. doit développer une compréhension système du logiciel au sein de l'environnement prévu d'application 2. doit développer un plan de validation et spécifier les tâches et activités essentielles pour la validation du logiciel et convenir de ce plan avec le Chargé d'évaluation 3. doit examiner les exigences relatives au logiciel par rapport à l'environnement/usage prévu 4. doit examiner le logiciel par rapport aux exigences relatives au logiciel afin d'assurer que la totalité de celles-ci est respectée 5. doit évaluer la conformité du processus logiciel et du logiciel développé par rapport aux exigences de la présente Norme européenne, y compris les SIL attribués 6. doit examiner la validité, la cohérence et l'adéquation de la vérification et des tests 7. doit vérifier la validité, la cohérence et l'adéquation des scénarios de test et des tests exécutés 8. doit assurer que toutes les activités du plan de validation ont été effectuées 9. doit examiner et classer tous les écarts en termes de risque (impact), les consigner et les soumettre à l'organisme chargé de la gestion des modifications et de la prise de décisions 10. doit donner une recommandation sur l'adéquation du logiciel pour l'utilisation prévue et indiquer toute contrainte d'application le cas échéant 11. doit produire des écarts à partir du plan de validation 12. doit réaliser des audits, examens ou revues sur le projet global (comme instanciations du processus de développement générique) selon ce qui est approprié dans diverses phases du développement 13. doit examiner et analyser les rapports de validation relatifs à des applications antérieures selon ce qui est approprié 14. doit examiner si les solutions développées sont traçables à des exigences relatives au logiciel 15. doit assurer que les registres des situations dangereuses associés et les non-conformités restantes sont examinés et toutes les situations dangereuses sont maîtrisées de la manière appropriée par des mesures d'élimination ou de transfert/maîtrise du risque 16. doit rédiger un rapport de validation 17. doit signifier son accord/désaccord pour la diffusion du logiciel
Principales compétences : 1. doit être compétent dans le domaine où la validation est effectuée 2. doit avoir l'expérience des critères de sécurité dans le domaine d'application 3. doit être compétent en diverses approches/méthodologies de validation et être capable d'identifier la méthode ou combinaison de méthodes la plus adaptée dans un contexte donné 4. doit être capable de déduire les types de preuves de validation requises à partir des spécifications données en gardant à l'esprit l'application prévue 5. doit être capable de combiner différents types et sources de preuves et de synthétiser une vue globale sur l'adéquation aux besoins ou les contraintes et limitations de l'application 6. doit avoir une capacité de réflexion analytique et un bon sens de l'observation 7. doit avoir une compréhension et une représentation du logiciel global, notamment la compréhension de l'environnement d'application 8. doit comprendre les exigences de l'EN 50128.

Tableau B.8 — Spécification du Rôle du Chargé d'évaluation

Rôle : Chargé d'évaluation
Responsabilités : 1. doit développer une compréhension système du logiciel au sein de l'environnement prévu d'application 2. doit rédiger un plan d'évaluation et le communiquer à l'autorité de tutelle et l'organisation du client (organisme contractant du Chargé d'évaluation) 3. doit évaluer la conformité du processus logiciel et du logiciel développé par rapport aux exigences de la présente Norme européenne, y compris les SIL attribués 4. doit évaluer la compétence de l'équipe et de l'organisation du projet pour le développement du logiciel 5. doit évaluer les activités de vérification et de validation et les pièces justificatives 6. doit évaluer les systèmes de management de la qualité adoptés pour le développement du logiciel 7. doit évaluer le système de gestion des modifications et de la configuration ainsi que la preuve de son utilisation et de son application 8. doit identifier et évaluer en termes de risque (impact) tout écart par rapport aux exigences relatives au logiciel dans le rapport d'évaluation 9. doit assurer que le plan d'évaluation est mis en œuvre 10. doit effectuer des audits de la sécurité et des examens sur le processus de développement global, le cas échéant, à diverses phases du développement 11. doit donner un avis circonstancié sur l'adéquation du logiciel développé pour son utilisation prévue en détaillant toutes les contraintes, conditions d'applications et observations pour la maîtrise du risque selon le cas 12. doit rédiger un rapport d'évaluation et maintenir des enregistrements sur le processus d'évaluation.
Principales compétences : 1. doit être compétent dans le domaine ou les technologies où l'évaluation est effectuée 2. doit avoir l'habilitation/licence de la part d'une autorité de tutelle reconnue 3. doit avoir / s'efforcer en permanence d'acquérir des niveaux suffisants d'expérience en matière de principes de sécurité et d'application de ces principes à l'intérieur du domaine d'application 4. doit être compétent pour vérifier qu'une méthode ou combinaison de méthodes adaptée dans un contexte donné a été appliquée 5. doit être compétent pour comprendre les processus pertinents de gestion de la sécurité, des ressources humaines, techniques et de la qualité pour satisfaire aux exigences de l'EN 50128 6. doit être compétent en matière d'approches/méthodologies d'évaluation 7. doit avoir une capacité de réflexion analytique et un bon sens de l'observation 8. doit être capable de combiner différents types et sources de preuves et de synthétiser une vue globale sur l'adéquation aux besoins ou les contraintes et limitations imposées à l'application 9. doit avoir une compréhension et une représentation du logiciel global, notamment la compréhension de l'environnement d'application 10. doit être capable de juger de l'adéquation de tous les processus de développement (comme la gestion de la qualité, la gestion de la configuration, les processus de validation et de vérification) 11. doit comprendre les exigences de l'EN 50128.

Tableau B.9 — Spécification du Rôle du Chef de projet

Rôle : Chef de projet
Responsabilités : 1. doit assurer que le système de gestion de la sécurité et l'indépendance des rôles selon5.1 sont en place pour le projet et que l'avancement est contrôlé par rapport aux plans 2. doit allouer un nombre suffisant de ressources compétentes dans le projet pour mener à bien les tâches essentielles, y compris les activités de sécurité, en gardant à l'esprit l'indépendance nécessaire des rôles 3. doit assurer qu'un Chargé de validation approprié a été désigné pour le projet comme défini dans l'EN 50128 4. doit être responsable de la livraison et du déploiement du logiciel et assurer que les exigences de sécurité issues des différentes parties prenantes sont aussi satisfaites et transmises 5. doit accorder un temps suffisant pour la mise en œuvre et le respect appropriés des tâches de sécurités 6. doit entériner les produits de sécurité complets et partiels délivrés par le processus de développement 7. doit assurer que des enregistrements et une traçabilité suffisants sont maintenus dans la prise de décision relative à la sécurité
Principales compétences : 1. doit comprendre les exigences de l'EN 50128 relatives à la qualité, aux compétences, à l'organisation et à la gestion 2. doit comprendre les exigences du processus de sécurité 3. doit être capable de peser les différentes options et comprendre l'impact sur la performance sécurité d'une décision ou d'options choisies 4. doit comprendre les exigences du processus de développement du logiciel 5. doit comprendre les exigences d'autres normes applicables

Tableau B.10 — Spécification du Rôle du Gestionnaire de la Configuration

Rôle : Gestionnaire de la Configuration
Responsabilités : 1. doit être responsable du plan de gestion de la configuration du logiciel 2. doit être propriétaire du système de gestion de la configuration 3. doit établir que toutes les exigences relatives au logiciel sont clairement identifiées et mises en versions de manière indépendante à l'intérieur du système de gestion de la configuration 4. doit préparer des Fiches de Livraison qui mentionnent les versions incompatibles des composants logiciels
Principales compétences : 1. doit être compétent en gestion de configuration de logiciel 2. doit comprendre les exigences de l'EN 50128

Annexe C
(informative)

Résumé du contrôle des documents

Le Tableau C.1 fournit un résumé du document.

Tableau C.1 — Résumé du Contrôle des Documents

PHASE	DOCUMENTATION	Rédigé par	1ère vérification	2nde vérification
Planification	1. Plan d'Assurance Qualité du Logiciel	[a]	VER	VAL
	2. Rapport de vérification concernant l'Assurance Qualité du logiciel	VER		VAL
	3. Plan de Gestion de la Configuration du Logiciel	voir B.10	VER	VAL
	4. Plan de Vérification du Logiciel	VER		VAL
	5. Plan de Validation du Logiciel	VAL	VER	
Exigences relatives au logiciel	6. Spécification des Exigences du Logiciel	REQ	VER	VAL
	7. Spécification de Tests d'Ensemble du Logiciel	TST	VER	VAL
	8. Rapport de Vérification des Exigences du Logiciel	VER		VAL
Architecture et conception	9. Spécification de l'Architecture du Logiciel	DES	VER	VAL
	10. Spécification de la Conception du Logiciel	DES	VER	VAL
	11. Spécifications d'Interface du Logiciel	DES	VER	VAL
	12. Spécification de Tests d'Intégration du Logiciel	INT	VER	VAL
	13. Tests d'Intégration Logiciel/Matériel	INT	VER	VAL
	14. Rapport de Vérification de l'Architecture et de la Conception du Logiciel	VER		VAL
Conception du composant	15. Spécification de la Conception des Composants logiciel	DES	VER	VAL
	16. Spécification de Tests de Composant Logiciel	TST	VER	VAL
	17. Rapport de Vérification de la Conception des Composants Logiciels	VER		
Réalisation et test des composants	18. Code Source du Logiciel et Documentation de Support	IMP	VER	VAL
	19. Rapport de Vérification du Code Source du Logiciel	VER		VAL
	20. Rapport de Tests des Composants Logiciels	TST	VER	VAL
Intégration	21. Rapport de Tests d'Intégration du Logiciel	INT	VER	VAL
	22. Rapport de Tests d'Intégration Logiciel/Matériel	INT	VER	VAL
	23. Rapport de Vérification de l'Intégration du Logiciel	VER		
Tests d'ensemble du logiciel / Validation finale	24. Rapport de Tests d'Ensemble du Logiciel	TST	VER	VAL
	25. Rapport de Validation du Logiciel	VAL	VER	
	26. Rapport de Validation des Outils	[a]	VER	
	27. Fiche de livraison	[a]	VER	VAL

Tableau C.1 – Résumé du contrôle des documents *(suite)*

PHASE	DOCUMENTATION	Rédigé par	1ère vérification	2nde vérification
Systèmes configurés par des données/algorithmes d'application	28. Spécification des Exigences de l'Application	REQ	VER	VAL
	29. Plan de préparation de l'application	REQ ou DES	VER	VAL
	30. Spécification de Tests de l'Application	TST	VER	VAL
	31. Architecture et Conception de l'Application	DES	VER	VAL
	32. Rapport de Vérification de la Préparation de l'Application	VER		
	33. Rapport de Tests d'Application	TST	VER	VAL
	34. Code Source des données/algorithmes d'application	DES	VER	VAL
	35. Rapport de vérification des données/algorithmes d'application	VER		VAL
Déploiement du logiciel	36. Plan de livraison et de déploiement du Logiciel	[a]	VER	VAL
	37. Manuel de Déploiement du Logiciel	[a]	VER	VAL
	38. Fiche de livraison	[a]	VER	VAL
	39. Enregistrements du Déploiement	[a]	VER	VAL
	40. Rapport de Vérification du Déploiement	VER		
Maintenance du logiciel	41. Plan de Maintenance du Logiciel	[a]	VER	VAL
	42. Registres des modifications du logiciel	[a]	VER	VAL
	43. Enregistrements de la Maintenance du Logiciel	[a]	VER	VAL
	44. Rapport de Vérification de la Maintenance du Logiciel	[a]	VER	VAL
Évaluation du logiciel	45. Plan d'Évaluation du Logiciel	ASR	VER	
	46. Rapport d'Évaluation du Logiciel	ASR	VER	

a Aucun rôle spécifique n'est défini.

Annexe D
(informative)

Bibliographie des techniques

D.1 Intelligence artificielle - Correction des défauts

Objectif

Être capable de réagir aux situations dangereuses possibles d'une manière très souple en introduisant un mélange (une combinaison) de méthodes et de modèles de processus et un certain type d'analyse en ligne de la sécurité et de la fiabilité.

Description

En particulier, il est admis que la prévision des défauts (calcul des tendances), la correction des défauts et les actions de maintenance et de surveillance soient supportées de manière très efficace par des systèmes basés Iintelligence Artificielle dans divers canaux d'un système, puisque les règles seraient éventuellement dérivées directement des spécifications et vérifiées par rapport à celles-ci. Il n'est pas exclu d'éviter efficacement, grâce à cette approche, certains défauts communs qui sont introduits dans les spécifications du fait d'avoir déjà implicitement présentes à l'esprit certaines règles de conception et d'implémentation, particulièrement lorsqu'on applique une combinaison de modèles et de méthodes d'une manière fonctionnelle ou descriptive.

Les méthodes sont sélectionnées de manière à permettre que les défauts soient corrigés et les effets des défaillances minimisés en vue de satisfaire à la sécurité et à la fiabilité recherchées.

D.2 Programmes analysables

Objectif

Concevoir un programme de sorte que l'analyse du programme soit facilement réalisable. Le comportement du programme doit pouvoir être testé complètement sur la base de l'analyse.

Description

L'objectif consiste à produire des programmes faciles à analyser à l'aide des méthodes d'analyse statique. Pour atteindre ce but, il convient de respecter les règles de la programmation structurée, par exemple :

- il convient que le flux de contrôle du composant soit composé d'éléments structurés, autrement dit de séquences, d'itérations et de sélections;
- il convient que les composants soient de petites dimensions;
- le nombre de chemins possibles via un composant est faible;
- il faut que les parties du programme soient conçues de manière à être indépendantes autant que possible;
- il convient que la relation entre les paramètres d'entrée et de sortie soit aussi simple que possible;
- il convient de ne pas utiliser des calculs complexes comme base des décisions de branchements et de boucles;
- il convient que les décisions de branchements et de boucles soient liées aux paramètres d'entrée du composant de façon simple;
- les limites entre les différents types d'organisation doivent être simples.

D.3 Tests en avalanche/en surcharge

Objectif

Charger l'objet testé avec une charge de travail exceptionnellement élevée de manière à montrer qu'il résisterait facilement à des charges de travail normales.

Description

Il existe une variété de conditions de test qui peuvent être appliquées aux tests de surcharge. Certaines de ces conditions de test sont énumérées ci-dessous :

- si vous travaillez en mode interrogation, l'objet de test reçoit beaucoup plus de changements d'entrée par unité de temps que dans des conditions normales;
- si vous travaillez à la demande, le nombre de demandes par unité de temps adressées à l'objet testé est augmenté au-delà des conditions normales;
- si la taille de la base de données joue un rôle important, elle est augmentée au-delà des conditions normales;
- les périphériques influents sont réglés sur leur vitesse maximum ou minimum respectivement;
- dans les cas extrêmes, tous les facteurs influents sont, dans la mesure du possible, réglés sur leurs conditions limites en même temps.

Dans ces conditions de test, le comportement temporel de l'objet testé peut être évalué. L'influence des variations de travail peut être observée. Le dimensionnement correct des tampons internes ou des variables dynamiques, des piles, etc., peut être vérifié.

D.4 Analyse des valeurs aux limites

Objectif

Supprimer les erreurs logicielles se produisant aux limites ou frontières des paramètres.

Description

Le domaine d'entrée du programme est divisé en un nombre de catégories d'entrées. Il convient que les tests englobent les limites et les extrêmes des catégories. Les tests vérifient que les limites du domaine d'entrée de la spécification coïncident avec celles du programme. L'utilisation de la valeur zéro, dans une traduction directe aussi bien qu'indirecte, est souvent susceptible d'erreur et exige une attention spéciale :

- diviseur zéro;
- caractères de contrôle qui ne s'impriment pas;
- pile ou élément de liste vide;
- matrice nulle;
- entrée de tableau nulle.

En principe, les limites de l'entrée ont une correspondance directe avec les limites de la gamme de sortie. Il convient d'écrire des scénarios de test de manière à forcer la sortie dans ses valeurs limites. Examiner aussi, s'il est possible de spécifier un scénario de test dans lequel la sortie dépasserait les valeurs limites de la spécification.

Si la sortie est une séquence de données, par exemple un tableau imprimé, il convient d'apporter une attention toute spéciale au premier et au dernier élément et aux listes contenant aucun, 1 et 2 éléments.

D.5 Rattrapage par régression

Objectif

Permettre une exploitation fonctionnelle correcte en présence d'un ou de plusieurs défauts.

Description

Si un défaut a été détecté, le système est réinitialisé sur un état interne précédent dont la cohérence a déjà été éprouvée. Cette méthode implique une sauvegarde fréquente de l'état interne en ce que l'on appelle des points de reprises bien définis. Il est permis d'effectuer cette opération de façon globale (pour la base de données complète) ou incrémentielle (seulement les changements entre les points de reprise). Il faut alors que le système compense les changements qui se sont produits dans l'intervalle en utilisant la journalisation (trace des actions), la compensation (tous les effets de ces changements sont annulés) ou l'interaction (manuelle) externe.

D.6 Schémas de cause et de conséquence

Objectif

Modéliser, sous forme de schémas, la séquence d'événements pouvant se développer dans un système comme conséquence d'une combinaison d'événements de base.

Description

Cette technique peut être considérée comme une combinaison de l'analyse de l'arbre des causes et de l'arbre des événements. Commençant à partir d'un événement critique, un graphe cause-conséquence est tracé vers l'arrière et vers l'avant. Dans la direction arrière, il équivaut à un arbre des causes avec l'événement critique figurant tout en haut. Dans la direction avant, les conséquences possibles de cet événement sont identifiées. Le graphe peut contenir des symboles de nœuds qui décrivent les conditions de propagation le long des différentes branches partant du nœud. Des temporisations peuvent également être incluses. Ces conditions peuvent aussi être décrites à l'aide des arbres des causes. Les lignes de propagation peuvent être combinées à des symboles logiques afin de rendre le schéma plus compact. Un ensemble de symboles standards est défini pour être utilisé dans les schémas de cause et de conséquence. Les schémas peuvent être utilisés pour calculer la probabilité d'une occurrence de certaines conséquences critiques.

D.7 Listes de contrôle

Objectif

Stimuler l'évaluation critique sur tous les aspects du système plutôt que d'imposer des exigences spécifiques.

Description

Ensemble de questions qui doit être établi par la personne élaborant la liste de contrôle. La plupart des questions sont d'ordre général et il faut que le Chargé d'Évaluation les interprète de la façon la mieux adaptée au système spécifique en cours d'évaluation.

Pour tenir compte de la grande diversité des logiciels et des systèmes qui doivent être validés, la plupart des listes de contrôle contiennent des questions applicables à de nombreux types de systèmes. En conséquence, il n'est pas exclu que certaines questions figurant sur la liste ne soient pas applicables au système en question. Si tel est le cas, il convient de les ignorer. De la même façon, il est admis que, pour un système spécifique, il soit nécessaire de compléter la liste de contrôle standard par des questions directement adaptées au système en question.

En tout cas, il convient qu'il soit clair que l'utilisation des listes de contrôle dépend surtout de la compétence et du jugement de l'ingénieur qui sélectionne et met la liste de contrôle en application. En conséquence, il convient que les décisions prises par l'ingénieur par rapport à la liste de contrôle sélectionnée (aux listes de contrôle sélectionnées) et à toute question complémentaire ou superflue soient entièrement documentées et justifiées. L'objectif consiste à assurer que l'application des listes de contrôle puisse être passée en revue et que les mêmes résultats soient obtenus à moins que des critères différents ne soient utilisés.

Lorsque l'on complète une liste de contrôle, l'objectif est d'être aussi concis que possible. Quand une justification approfondie s'avère nécessaire, il convient de l'apporter par renvoi à une documentation complémentaire. Il convient d'utiliser des mentions telles que Satisfaisant, Insuffisant, Peu concluant ou un ensemble limité de symboles similaires pour enregistrer les résultats de chaque question. Cette concision simplifie beaucoup le processus permettant d'arriver à une conclusion globale à partir des résultats de l'évaluation de la liste de contrôle.

D.8 Analyse de Flux de Contrôle

Objectif

Détecter des structures faibles et potentiellement incorrectes dans le programme.

Description

L'analyse de flux de contrôle identifie les zones suspectes du code qui ne respectent pas les bonnes pratiques de programmation. Le programme est analysé pour former un graphe orienté qui peut être analysé pour

- un code inaccessible, par exemple, branchements inconditionnels qui rendent les blocs de code inaccessibles,
- un code arborescent, qui est un code bien structuré dont le graphe de contrôle est réductible en un simple nœud par réductions de graphes successifs. Un code faiblement structuré ne peut être réduit qu'à une nodosité composée de plusieurs nœuds.

D.9 Analyse des défaillances de mode commun

Objectif

Identifier dans les systèmes ou les sous-systèmes redondants les défaillances potentielles qui amoindriraient les avantages de la redondance en raison de l'apparition simultanée des mêmes défaillances dans les parties redondantes.

Description

Les systèmes informatiques conçus pour protéger la sécurité d'une unité de production utilisent souvent la redondance dans le matériel et la décision à la majorité. Cette technique est utilisée pour éviter les défaillances aléatoires des composants qui risqueraient d'empêcher le traitement correct des données dans un système informatique.

Certaines défaillances, pourtant, peuvent être communes à plusieurs composants. Par exemple, si un système informatique est installé dans une même salle, des imperfections dans la climatisation risqueraient éventuellement de réduire les avantages de la redondance. Il en va de même pour des effets externes sur le système informatique tels que : les incendies, les inondations, les interférences électromagnétiques, les accidents d'avion et les tremblements de terre. Il n'est pas exclu, en outre, que le système informatique soit affecté par des incidents liés à l'exploitation et à la maintenance. Il est donc essentiel de proposer des procédures adéquates et bien documentées pour l'exploitation et la maintenance. La formation approfondie du personnel d'exploitation et de maintenance est également essentielle.

Les effets internes également contribuent largement aux défaillances ayant une cause commune (CCF, *Common-Cause Failure*). Elles peuvent découler d'erreurs de conception dans des composants communs ou

identiques et dans leurs interfaces, aussi bien que du vieillissement des composants. L'analyse CCF doit examiner le système à la recherche de défaillances potentielles communes. Les méthodes d'analyse CCF sont le contrôle général de la qualité, les examens de la conception, la vérification et les tests effectués par une équipe indépendante et l'analyse des incidents réels avec le retour d'expérience acquise sur des systèmes similaires. Cependant, le domaine d'application de l'analyse va au delà du matériel. Même si des "logiciels variés" sont utilisés dans les chaînes complexes d'un système informatique redondant, il n'est pas exclu qu'un certain degré de standardisation dans les approches logicielles puisse éventuellement donner lieu à une analyse CCF. Il pourrait s'agir, par exemple, d'erreurs dans la spécification commune.

Quand les défaillances ayant une cause commune ne se produisent pas exactement en même temps, des précautions peuvent être prises en utilisant des méthodes de comparaison entre les chaînes redondantes et il convient que ces méthodes mènent à la détection d'une défaillance avant que celle-ci ne devienne commune à toutes les chaînes. Il convient que l'analyse CCF prenne en compte cette technique.

D.10 Analyse du flux de données

Objectif

Détecter des structures faibles et potentiellement incorrectes dans le programme.

Description

L'analyse du flot de données combine l'information obtenue à partir de l'analyse du flux de contrôle à l'information concernant les variables qui sont lues ou écrites dans différentes portions du code. L'analyse peut rechercher

- des variables qui sont lues avant d'être écrites. Il est fort probable qu'il s'agisse d'une erreur et en tout cas d'une mauvaise pratique de programmation,
- des variables qui sont écrites plusieurs fois sans être lues, ce qui pourrait indiquer qu'un code a été omis,
- des variables qui sont écrites mais jamais lues, ce qui pourrait indiquer qu'un code est redondant.

Il existe une extension de l'analyse du flot de données, connue sous le nom d'analyse du flux d'informations dans laquelle les flots de données réels (à la fois à l'intérieur et entre les procédures) sont comparés au projet de conception. Cette analyse est, en principe, réalisée par un outil informatisé dans lequel les flots de données projetés sont définis à l'aide d'un commentaire structuré qui peut être lu par l'outil.

D.11 Organigrammes des données

Objectif

Décrire le flot de données dans un programme sous forme de schéma.

Description

Les organigrammes de données documentent la façon dont les données d'entrée sont transformées en sorties, chaque étape du schéma représentant une transformation distincte.

Les composants de base d'un organigramme de données comprennent

- les fonctions, représentées par des bulles,
- les flots de données, représentés par des flèches,
- magasins de données représentés par des boîtes ouvertes,
- les entrées/sorties, représentées par des types spéciaux de boîtes.

Les organigrammes de données décrivent la façon dont une entrée est transformée en sortie. Comme il convient, ils ne contiennent pas des informations de contrôle ou des informations de séquencement. Chaque bulle peut être considérée comme une boîte noire autonome qui, dès que ses entrées sont disponibles, les transforme en ses sorties.

L'un des principaux avantages des organigrammes de données est qu'ils montrent les transformations sans émettre aucune hypothèse sur la manière dont ces transformations sont mises en œuvre.

La meilleure approche vis-à-vis de la préparation des organigrammes de données est de prendre en considération les entrées système et d'évoluer vers les sorties système. Il faut que chaque bulle représente une transformation distincte - il convient que sa sortie soit, d'une certaine façon, différente de son entrée. Il n'existe aucune règle permettant de déterminer la structure globale du schéma et la construction d'un organigramme de données est l'un des aspects créatifs de la conception système. A l'instar de toute conception, il s'agit d'un processus itératif composé de tentatives précoces affinées par étapes pour produire un schéma final.

D.12 Enregistrement et analyse des données

Objectif

Faciliter l'amélioration du processus logiciel par l'enregistrement, la validation et l'analyse de données pertinentes issues de projets individuels et de personnes. La pertinence des données est déterminée par les objectifs stratégiques de l'organisation. Les objectifs peuvent être orientés vers l'évaluation d'une méthode particulière de développement de logiciel relative aux demandes de modification la concernant, par exemple par rapport à l'efficacité de la prévention des défauts.

Description

L'enregistrement et l'analyse des données constituent une part essentielle de l'amélioration du processus logiciel. L'enregistrement de données valides représente une part importante pour en savoir plus sur le processus de développement de logiciel et pour évaluer des méthodes alternatives de développement de logiciel.

Des comptes rendus détaillés qui concernent à la fois le projet lui-même et les questions individuelles sont actualisés pendant un projet. Par exemple, un ingénieur est tenu de conserver des comptes rendus pouvant inclure :

- l'effort développé sur les composants individuels,
- les tests effectués sur chaque composant,
- les décisions et le raisonnement les sous-tendant,
- la réalisation des jalons du projet,
- les problèmes et leurs solutions.

Pendant le projet et au moment de sa conclusion, ces comptes rendus peuvent être analysés pour établir une grande variété d'informations. En particulier, l'enregistrement des données est très important pour la maintenance de systèmes informatiques puisque le raisonnement sous-tendant certaines décisions prises au cours du projet de développement n'est pas toujours connu des ingénieurs de maintenance.

En raison d'une médiocre planification, l'enregistrement des données tend souvent à occuper un volume excessif et à être hors propos. Cela peut être évité en suivant le principe selon lequel il convient de piloter l'enregistrement de données par des objectifs, des questions et des métriques de pertinence par rapport à ce qui est stratégiquement important pour l'organisation.

Pour atteindre l'exactitude souhaitée, il convient de procéder à l'enregistrement des données et au processus de validation concomitamment avec le développement, par exemple comme partie du processus de la maîtrise de la configuration.

D.13 Tables de décision (Tables de vérité)

Objectif

Fournir une spécification et une analyse claires et cohérentes des relations et combinaisons logiques complexes.

Description

Ces méthodes connexes utilisent des tableaux à deux dimensions pour décrire de façon concise les relations logiques entre les variables booléennes du programme.

La concision et la nature tabulaire des deux méthodes en font des moyens parfaitement adaptés pour l'analyse des combinaisons logiques complexes exprimées dans un code.

Les deux méthodes sont potentiellement exécutables si elles sont utilisées comme spécifications.

D.14 Programmation défensive

Objectif

Générer des programmes qui détectent un flux de contrôle, un flot de données ou des valeurs de données anormaux pendant leur exécution et qui réagissent envers eux d'une manière prédéterminée et acceptable.

Description

Beaucoup de techniques peuvent être utilisées au cours de la programmation pour rechercher les anomalies du contrôle ou des données. Ces techniques peuvent être appliquées systématiquement tout au long de la programmation d'un système pour diminuer la vraisemblance d'un traitement erroné des données.

On peut identifier deux zones de chevauchement des techniques défensives. Le logiciel intrinsèquement protégé contre les erreurs est conçu pour prendre en charge ses propres imperfections de conception. Il n'est pas exclu que celles-ci soient provoquées par une simple erreur de conception ou de codage ou par des exigences erronées. Certaines techniques défensives sont énumérées ci-dessous :

- il convient de vérifier l'étendue des variables;
- chaque fois que possible, il convient de vérifier la plausibilité des valeurs;
- il convient de vérifier le type, la dimension et l'étendue des paramètres des procédures à l'entrée de la procédure.

Ces trois premières recommandations aident à assurer que les nombres manipulés par le programme sont fondés, à la fois en termes de fonction du programme et de signification physique des variables.

Il convient de séparer les paramètres en lecture seule et en lecture-écriture et de vérifier leur accès. Il convient que les fonctions traitent tous les paramètres comme s'ils étaient en lecture seule. Il convient que les constantes littérales ne soient pas accessibles en écriture. Ceci aide à détecter un écrasement accidentel ou une utilisation erronée des variables.

Un logiciel tolérant aux erreurs est conçu pour "s'attendre" à des défaillances dans son propre environnement ou à un emploi hors des conditions nominales ou prévues et se comporter d'une manière prédéfinie. Ces techniques incluent ce qui suit :

- il convient de vérifier la plausibilité des variables d'entrée et des variables intermédiaires avec la signification physique;
- il convient de vérifier l'effet des variables de sortie, de préférence par l'observation directe des changements d'état du système associé;
- il convient que le logiciel vérifie sa propre configuration. Ceci peut inclure à la fois l'existence et l'accessibilité du matériel attendu et aussi l'état complet du logiciel lui-même. Ces vérifications sont particulièrement importantes pour conserver l'intégrité après des procédures de maintenance.

Certaines de ces techniques de programmation défensive telles que la vérification des séquences du flux de contrôle traitent également les défaillances externes.

D.15 Normes de codage et Guide de style

Objectif

Assurer une présentation uniforme des documents de conception et du code généré, imposer une programmation cohérente et une méthode de conception sûre standard qui évite les erreurs.

Description

Les *Normes de codage* sont des règles et des restrictions sur un langage de programmation donné pour éviter les défauts potentiels qui peuvent être commis en utilisant le langage en question.

Il convient que le contenu de la Norme de codage comprenne

- la justification du langage,
- le domaine d'application et la norme de base, si disponibles,

  NOTE Pour les langages spécifiques à un domaine, des normes de base peuvent ne pas être disponibles.
- la procédure de modification de la norme de codage,
- l'analyse de défauts potentiels et le traitement recommandé,
- les restrictions pour éviter les défauts,
- la portabilité.

Les *Guides de style* traitent de questions telles que les conventions de formatage et de désignation, et bien que cela puisse être fortement subjectif, le style plus que tout affecte la lisibilité du code. L'établissement d'un style commun et cohérent pour un projet facilitera la compréhension et la maintenance du code développé par plus d'un programmeur et rendra plus aisé pour plusieurs personnes de coopérer dans le développement du même programme.

D.16 Programmation diversifiée

Objectif

Détecter et masquer les défauts résiduels de conception du logiciel pendant l'exécution d'un programme de manière à empêcher les défaillances critiques de sécurité du système et à continuer en direction d'une plus grande fiabilité.

Description

Dans la programmation diversifiée, une spécification de programme donnée est réalisée *N* fois de diverses manières. Les mêmes valeurs d'entrée sont données aux N versions et les résultats produits par les N versions sont comparés. Le résultat, s'il est considéré valide, est transmis aux sorties informatiques.

Les N versions peuvent tourner en parallèle sur des ordinateurs distincts, ou encore toutes les versions peuvent être exécutées sur le même ordinateur et les résultats soumis à une décision interne. Différentes stratégies de décisions peuvent être utilisées sur les N versions selon les exigences de l'application.

- Si le système a un état sûr, il est possible d'exiger l'accord complet (les *N* versions s'accordent); dans le cas contraire, une valeur intrinsèquement sûre de la sortie est utilisée. Pour les systèmes à simple déclenchement, la décision peut être influencée, en faveur de la sécurité. Dans ce cas, la sécurité serait de déclencher le système, si l'une ou l'autre version l'exigeait. En principe, cette approche n'utilise que deux versions (N=2).
- Pour les systèmes sans état sûr, les stratégies de décision à la majorité peuvent être employées. Dans les cas où il n'existe aucun accord collectif, les approches probabilistes peuvent être utilisées afin de maximiser la chance de sélection de la valeur correcte, par exemple en prenant la valeur moyenne, par le maintien en l'état temporaire des sorties jusqu'au retour de l'accord, etc.

Cette technique n'élimine pas les défauts résiduels de conception du logiciel, mais propose une mesure permettant de les détecter et de les masquer avant qu'ils puissent avoir une incidence sur la sécurité.

Malencontreusement, des expérimentations et des études analytiques montrent que la programmation en *N* versions n'est pas toujours aussi efficace qu'on le souhaite. Même si des algorithmes différents sont utilisés, les diverses versions du logiciel "plantent" trop souvent sur les mêmes entrées.

Deux alternatives à la programmation en *N* versions sont la diversité de la conception et la diversité fonctionnelle. La diversité de conception implique l'utilisation de plusieurs composants, chacun de ceux-ci étant conçu d'une manière différente mais mettant en œuvre la même fonction. La diversité fonctionnelle implique la résolution du même problème de plusieurs manières fonctionnellement différentes. Quelle que soit l'approche, aucune méthode efficace d'évaluation du niveau de diversité n'est actuellement disponible.

D.17 Reconfiguration dynamique

Objectif

Conserver la fonctionnalité du système malgré un défaut interne.

Description

Il faut que l'architecture logique du système soit telle qu'elle puisse être configurée sur un sous-ensemble des ressources disponibles du système. Il est nécessaire que l'architecture soit capable de détecter une défaillance dans une ressource physique, puis de reconfigurer l'architecture logique sur les ressources limitées qui fonctionnent toujours. Bien que le concept soit plus traditionnellement limité au rétablissement pour cause d'unités matérielles défaillantes, il est également applicable à des unités logicielles défaillantes s'il y a "une redondance d'exécution" suffisante pour permettre une nouvelle tentative du logiciel ou s'il existe des données redondantes suffisantes pour isoler la défaillance.

Bien que traditionnellement appliquée au matériel, cette technique est en cours de développement pour être appliquée au logiciel et donc au système dans son ensemble. Il faut qu'elle soit prise en considération dès la première étape de conception du système.

D.18 Tests de classes d'équivalence et de partition d'entrée

Objectif

Tester le logiciel de façon adéquate en utilisant un minimum de cas de test. Les cas de test sont obtenus en sélectionnant les partitions du domaine d'entrée requis pour tester le logiciel.

Description

Cette stratégie de test repose sur la relation d'équivalence des entrées qui détermine une partition du domaine d'entrée.

Les scénarios de test sont sélectionnés dans le but de couvrir tous les sous-ensembles de cette partition. Un scénario de test minimum est pris dans chaque classe d'équivalence.

Les deux possibilités de base existant pour le partitionnement d'entrée sont les suivantes :

- les classes d'équivalence peuvent être définies au travers de la spécification. L'interprétation de la spécification peut être orientée entrée, par exemple les valeurs sélectionnées sont traitées de la même façon, ou orientée sortie, par exemple l'ensemble de valeurs conduisant au même résultat fonctionnel, et
- les classes d'équivalence sont éventuellement définies sur la structure interne du programme. Dans ce cas, les résultats des classes d'équivalence sont déterminés à partir de l'analyse statique du programme, par exemple l'ensemble de valeurs conduisant au même chemin en cours d'exécution.

D.19 Codes de détection et de correction d'erreurs

Objectif

Détecter et corriger des erreurs dans les informations sensibles.

Description

Pour une information de n bits, un bloc codé de *k* bits est généré Il permet la détection et la correction des erreurs. Les différents types de code incluent :

- les codes de Hamming;
- les codes cycliques;
- codes polynomiaux;
- codes de hachage;
- codes cryptographiques.

D.20 Supposition d'erreurs

Objectif

Supprimer des erreurs de programmation courantes.

Description

Il n'est pas exclu que l'expérience des tests et l'intuition combinées à la connaissance et à la curiosité envers le système testé ajoutent certains scénarios de test inclassables dans l'ensemble des scénarios de test prévus. Il n'est pas exclu que des valeurs ou des combinaisons de valeurs spéciales soient sujettes à

erreurs. Il est permis de déduire certains scénarios de test intéressants à partir des listes de contrôle d'inspection. Il n'est pas exclu d'examiner aussi si le système est suffisamment robuste. Peut-on pousser les boutons du panneau avant trop rapidement ou trop fréquemment ? Que se passe-t-il si l'on pousse deux boutons simultanément ?

D.21 Insertion d'erreurs

Objectif

Vérifier si un ensemble de scénarios de test est adéquat.

Description

Certains types d'erreurs connus sont insérés dans le programme et le programme est exécuté avec les scénarios de test dans les conditions de test. Si seules quelques-unes des erreurs insérées sont trouvées, l'ensemble des scénarios de test n'est pas adéquat. Le rapport entre les erreurs insérées trouvées et le nombre total d'erreurs insérées est une estimation du rapport entre les erreurs réelles trouvées et le nombre total d'erreurs. Cela donne une possibilité d'estimation des erreurs restantes et donc de l'effort de test qu'il reste à fournir.

$$\frac{\text{Erreurs insérées trouvées}}{\text{Nombre total d'erreurs insérées}} = \frac{\text{Erreurs réelles trouvées}}{\text{Nombre total d'erreurs réelles}}$$

Il est admis que la détection de toutes les erreurs insérées indique que l'ensemble des scénarios de test est adéquat ou que les erreurs insérées étaient trop faciles à trouver. Cette méthode comporte des limitations car, pour obtenir un résultat utilisable, il faut que le type d'erreur ainsi que la position des insertions correspondent à la distribution statistique des erreurs réelles.

Si l'insertion d'erreurs est utilisée, l'emplacement de toutes les erreurs doit assurer que toutes les erreurs insérées ont été retirées avant la diffusion du logiciel.

D.22 Analyse par arbre des événements

Objectif

Modéliser, sous forme de schéma, la séquence des événements qui peuvent se développer dans un système après un événement déclencheur et, par ce moyen, indiquer comment des conséquences graves peuvent survenir.

Description

En haut du schéma sont écrites les conditions de la séquence ayant un rapport avec le déroulement suivant l'événement déclencheur qui constitue la cible de l'analyse. En commençant sous l'événement déclencheur, on tire un trait vers la première condition de la séquence. Là le schéma se divise en une branche "oui" et une branche "non", en décrivant la manière dont les développements futurs dépendent de la condition. Pour chacune de ces branches, on continue vers la prochaine condition de la même façon. Toutes les conditions, toutefois, ne sont pas pertinentes pour toutes les branches. On continue jusqu'à la fin de la séquence et chaque branche de l'arbre structuré de cette façon représente une conséquence possible. L'arborescence des événements peut être utilisé pour calculer la probabilité des diverses conséquences reposant sur la probabilité et sur le nombre de conditions dans la séquence.

D.23 Inspection de Fagan

Objectif

Mettre à jour des erreurs dans toutes les phases du développement du programme.

Description

L'audit "formel" des documents d'assurance qualité avait pour objectif de trouver des erreurs ou des omissions. Ce processus de contrôle se compose de cinq phases : la planification, la préparation, l'inspection, la correction et le suivi. Chacune de ces phases comporte son propre objectif distinct. Il faut que le développement complet du système (spécification, conception, codage et tests) soit contrôlé.

D.24 Programmation par assertion des défaillances

Objectif

Détecter les défauts résiduels au cours de l'exécution d'un programme logiciel.

Description

La méthode de programmation par assertion repose sur le concept de vérification d'une précondition (la validité des conditions initiales est vérifiée avant qu'une séquence d'instructions ne soit exécutée) et d'une postcondition (les résultats sont vérifiés après l'exécution d'une séquence d'instructions). Si la précondition ou la postcondition n'est pas conforme, le traitement s'arrête avec une erreur.

Par exemple,

```
assertion <précondition>;
    action 1;
        :
        :
    action x;
assertion <postcondition>;
```

D.25 AEEL – Analyse des Effets des Erreurs du Logiciel

Objectif

Identifier les composants logiciel, leur criticité; proposer un moyen de détecter des erreurs de logiciel et renforcer la robustesse du logiciel; évaluer la quantité de validation nécessaire sur les divers composants logiciel.

Description

L'analyse se fait en trois phases :

- Identification des composants logiciels vitaux

  Détermination de la profondeur de l'analyse (au niveau d'une simple ligne d'instruction, d'un groupe d'instructions, d'un composant, etc.) requise pour chaque composant, à partir de sa spécification.

- Analyse des erreurs du logiciel

  Le résultat de cette phase est présenté dans un tableau répertoriant les informations suivantes :

  - nom du composant;
  - erreur examinée;
  - conséquences de l'erreur au niveau du module;
  - conséquences au niveau du système;
  - critère de sécurité violé;
  - gravité de l'erreur;
  - moyen proposé de détection d'erreur;
  - critère violé si le moyen de détection proposé est mis en application;
  - gravité résiduelle si le moyen de détection est mis en application.

- Synthèse

  La synthèse identifie les scénarios anti-sécuritaire restants et l'effort de validation nécessaire étant donné l'importance de chaque module.

L'analyse AEEL (Analyse des Effets des Erreurs du Logiciel), en tant qu'analyse en profondeur effectuée par une équipe indépendante, constitue une méthode puissante de détection des anomalies.

D.26 Détection des défauts et diagnostic

Objectif

Détecter dans un système les défauts qui peuvent conduire à une défaillance, et fournir ainsi la base de contre-mesures de manière à minimiser les conséquences des défaillances.

Description

La détection des défauts est le processus consistant à rechercher des états erronés dans un système (qui sont provoqués, comme il est expliqué plus haut, par un défaut à l'intérieur du (sous-)système à vérifier). L'objectif principal de la détection des défauts consiste à inhiber l'effet des résultats faux. Un système qui fournit des résultats corrects ou pas de résultats du tout, est appelé "autocorrecteur".

La détection de défauts repose sur les principes de la redondance (principalement pour détecter les défauts matériels) et de la diversité (défauts logiciels). Un certain type de vote est nécessaire pour déterminer le bien fondé des résultats. Les méthodes spéciales applicables sont la programmation par assertion, la programmation en *N* versions et la technique sécurité contrôlée, et au niveau matériel on peut introduire des capteurs, des boucles de contrôle, des codes de contrôle d'erreur, etc.

Il est admis que la détection de défauts soit réalisée au moyen de contrôles du domaine des valeurs ou du domaine du temps sur différents niveaux, spécialement au niveau physique (température, tension etc.), logique (codes de détection d'erreur), fonctionnel (assertions) ou externe (contrôles de plausibilité). Il est admis que les résultats de ces contrôles soient stockés et associés aux données affectées pour permettre le suivi des défaillances.

Les systèmes complexes sont composés de sous-systèmes. L'efficacité de la détection des défauts, du diagnostic et de la compensation des défauts dépend de la complexité des interactions entre les sous-systèmes qui influencent la propagation des défauts.

Le diagnostic des défauts isole le plus petit sous-système pouvant être identifié. Des sous-systèmes plus petits autorisent un diagnostic plus détaillé des défauts (identification des états erronés).

D.27 Automates à états finis/Schémas de transitions d'état

Objectif

Définir ou mettre en œuvre la structure de contrôle d'un système.

Description

De nombreux systèmes peuvent être définis par rapport à leurs états, leurs entrées et leurs actions. Ainsi, quand le système est dans l'état S1, la réception de l'entrée I provoquerait éventuellement l'exécution de l'action A et le passage à l'état S2. En définissant les actions d'un système pour chaque entrée dans chaque état, nous pouvons définir complètement un système. Le modèle résultant du système porte le nom d'automate à états finis (FSM, *Finite State Machine*). Il est souvent représenté par un schéma appelé schéma de transition d'état montrant comment le système passe d'un état à un autre, ou par une matrice dont les dimensions sont l'état et l'entrée et dont les cellules contiennent l'action et le nouvel état résultant de la réception de l'entrée dans l'état donné.

Quand un système est compliqué ou a une structure naturelle, cette situation peut être simulée par des automates à états finis sur plusieurs niveaux.

Il est possible de vérifier l'état complet (le système doit avoir une action et un nouvel état pour chaque entrée dans chaque état), la cohérence (un seul changement d'état est défini pour chaque paire état/entrée) et l'accessibilité (s'il est possible ou non de passer d'un état à l'autre par une quelconque séquence d'entrées) d'une spécification ou une conception exprimée sous forme d'automate à état fini. Ces propriétés sont importantes pour les systèmes critiques et peuvent être vérifiées. Les outils permettant de supporter ces contrôles sont faciles à écrire. Il existe aussi des algorithmes autorisant la génération automatique des scénarios de test afin de vérifier la réalisation de l'automate à état fini ou l'animation d'un modèle d'automate à état fini.

Plusieurs extensions des automates à états finis ont été conçues pour améliorer la description du comportement d'un système complexe. Les dits graphiques d'état ajoutent de la hiérarchie, de la composition (parallélisme), des transitions inter-niveaux, des états antérieurs, etc. Un trait particulièrement utile est l'imbrication des états et transitions internes, donnant la possibilité de révéler ou cacher les états internes selon les besoins. Les graphiques d'état font partie du langage UML (*Unified Modeling Language*, langage de modélisation unifié) et, à ce titre, sont pris en charge par de nombreux outils commerciaux.

D.28 Méthodes formelles

Objectif

Les "Méthodes Formelles" se réfèrent à des techniques et outils mathématiquement rigoureux pour la spécification, la conception et la vérification des systèmes logiciels et matériels.

Description

"Mathématiquement rigoureux" signifie que les spécifications utilisées dans les méthodes formelles sont des énoncés bien formés dans une logique mathématique et que les vérifications formelles sont des déductions rigoureuses dans la logique en question (c'est-à-dire que chaque étape provient d'une règle d'inférence et donc peut être vérifiée par un processus mécanique). L'intérêt des méthodes formelles est de fournir un moyen d'examiner symboliquement l'ensemble de l'espace des états d'une conception numérique (qu'elle soit matérielle ou logicielle) et d'établir une assertion ou une propriété de sécurité qui soit vraie pour toutes les entrées possibles. Cependant, il en est rarement fait usage dans la pratique actuellement (sauf pour les composants critiques des systèmes critiques pour la sécurité) en raison de la complexité énorme des systèmes réels.

Plusieurs approches sont utilisées pour surmonter les espaces d'états de taille astronomique associés aux systèmes réels :

- appliquer des méthodes formelles aux exigences et aux conceptions de haut niveau où la plupart des détails sont soustraits;
- appliquer des méthodes formelles aux composants les plus critiques seulement;
- analyser des modèles de logiciel et de matériel où les variables sont discrétisées et les plages réduites de manière drastique;
- analyser des modèles de système d'une manière hiérarchique qui permet de "diviser pour régner";
- automatiser autant que possible la vérification.

Bien que l'utilisation de la logique mathématique soit un thème fédérateur dans la discipline des méthodes formelles; il n'existe pas une meilleure "méthode formelle" unique. Chaque domaine d'application requiert différentes méthodes de modélisation et différentes approches de preuve. En outre, même au sein d'un domaine d'application particulier, des outils et techniques différents peuvent être mis en œuvre pour répondre au mieux aux spécificités des différentes phases du cycle de vie. Par exemple, il vaudrait mieux utiliser un démonstrateur de théorèmes pour analyser la validité d'une description de niveau de transfert de registre (RTL, *register transfer level*) d'un circuit de Transformation de Fourier Rapide alors qu'il vaudrait mieux utiliser des méthodes de dérivation algébriques pour analyser la validité des affinements de conception en conception au niveau grille. Par conséquent un grand nombre de méthodes formelles sont en développement à travers le monde.

Plusieurs exemples de méthodes formelles sont décrits dans les paragraphes suivants de la présente bibliographie. La liste des exemples donnés n'est pas exhaustive. Les méthodes formelles décrites sont CSP, CCS, HOL, LOTOS, OBJ, Logique temporelle, VDM, la méthode Z, la méthode B et la vérification du modèle.

D.28.1 CSP - Processus Séquentiels de Communication

Objectif

La méthode CSP est une technique appliquée à la spécification des systèmes logiciels concurrents, autrement dit des systèmes de processus de communication fonctionnant simultanément.

Description

La méthode CSP propose un langage pour la spécification des systèmes de processus et une preuve permettant de vérifier que la réalisation des processus satisfait à leurs spécifications (décrites comme une trace - séquences permises d'événements).

Un système est modélisé comme un réseau de processus indépendants. Chaque processus est décrit au travers de tous ses comportements possibles. Un système est modélisé en composant des processus en série ou en parallèle. Les processus peuvent communiquer (synchroniser ou échanger des données) via des canaux, la communication n'ayant lieu que lorsque les deux processus sont prêts. La synchronisation relative des événements peut être modélisée.

La théorie sous-tendant les processus CSP a été directement incorporée dans l'architecture du transputeur INMOS[1)], et le langage OCCAM [2)] autorise la réalisation directe d'un système spécifié CSP sur un réseau de transputeurs

---

1) Inmos était une entreprise britannique de semi-conducteurs qui produisait dans les années 1980 une architecture innovante de microprocesseurs visant le traitement parallèle, appelée transputer. Plus tard, Inmos est devenu une partie de SGS-Thomson puis STMicroelectronics.

2) occam est un langage de programmation concurrent dont le nom dérive de celui de William of Ockham du fameux rasoir d'Occam. Il est le langage de programmation natif du transputeur d'inmos.

D.28.2 CCS - Algèbre des Systèmes de Transmission

Objectif

La méthode CCS est un moyen de décrire et de raisonner sur le comportement des systèmes de processus parallèles et communicants.

Description

Similaire à la méthode CSP, la méthode CCS est une algèbre appliquée au comportement de systèmes. La conception du système est modélisée comme un réseau de processus indépendants fonctionnant en série ou en parallèle. Les processus peuvent communiquer via des ports (similaires aux canaux CSP), la communication ne se produit que lorsque les deux processus sont prêts. Le non-déterminisme peut être modélisé. En commençant par une représentation très abstraite de l'ensemble du système (connue sous le nom de trace), il est possible de réaliser un affinement par étapes du système pour arriver à composer des processus de communication dont le comportement global est celui requis par l'ensemble du système. De la même façon, il est possible d'adopter une méthode ascendante, combinant des processus et déduisant les propriétés du système résultant à l'aide de règles d'inférence liées aux règles de composition.

D.28.3 HOL - Logique d'Ordre Supérieur

Objectif

Il s'agit d'un langage logique devant servir de base à la spécification et à la vérification du matériel.

Description

HOL (*Higher Order Logic*) fait référence à une notation logique spécifique et à son système de support machine, les deux ayant été développés dans les laboratoires informatiques de l'Université de Cambridge. La notation logique s'inspire principalement de la Simple théorie des types de Church (*Simple Theory of Types*) et le système du support machine repose sur le système des fonctions LCF (Logique des fonctions calculables - *Logic of Computable Functions*).

D.28.4 LOTOS

Objectif

Le langage LOTOS est un moyen de décrire et de raisonner sur le comportement des systèmes de processus parallèles et communicants.

Description

Le langage LOTOS (*Language for Temporal Ordering Specification*) repose sur CCS en intégrant des caractéristiques supplémentaires issues des algèbres connexes CSP et CIRCAL (*Circuit Calculus*). Il surmonte la faiblesse de CCS en matière de manipulation des structures des données et des expressions des valeurs en la combinant avec un deuxième composant basé sur le langage des types de données abstraits ACT ONE. Le composant LOTOS de définition du processus pourrait, toutefois, être utilisé avec d'autres formalismes pour la description des types de données abstraits.

D.28.5 OBJ

Objectif

Fournir une spécification précise du système avec une remontée de l'information utilisateur et une validation du système préalable à la mise en œuvre.

Description

OBJ est un langage de spécification algébrique. Les utilisateurs spécifient des exigences en termes d'équations algébriques. Les aspects de comportement, ou constructifs, du système sont spécifiés en termes d'opérations agissant sur les types de données abstraits (ADT, *abstract data types*). Un type de données abstrait est semblable à un ensemble ADA [3)] dans lequel le comportement de l'opérateur est visible tandis que les détails de mise en œuvre sont "cachés".

Une spécification OBJ et la mise en œuvre par étapes consécutive relèvent des mêmes techniques de preuves formelles que les autres approches formelles. En outre, puisque les aspects constructifs de la spécification OBJ sont exécutables par la machine, il est simple d'accomplir la validation du système à partir de la spécification elle-même. L'exécution est essentiellement l'évaluation d'une fonction par une substitution d'équation (réécriture) qui continue jusqu'à ce qu'une valeur de sortie spécifique soit obtenue. Cette "exécutabilité" permet aux utilisateurs finaus du système envisagé d'avoir un "aperçu" du système final au stade de la spécification du système sans avoir la nécessité de connaître les techniques de spécification formelles sous-jacentes.

Comme pour toutes les autres techniques ADT, le langage OBJ n'est applicable qu'aux systèmes séquentiels ou aux aspects séquentiels des systèmes concurrents. Ce langage OBJ a été largement utilisé pour la spécification des applications industrielles de petite et de grande échelle.

D.28.6 Logique temporelle

Objectif

Expression directe des exigences en matière de sécurité et de fonctionnement et démonstration formelle que ces propriétés sont préservées dans les étapes de développement suivantes.

Description

La logique standard des prédicats (du premier ordre) ne contient aucun concept de temps. La logique temporelle étend la logique du premier ordre en ajoutant des opérateurs modaux (par exemple, "Dorénavant" et "Finalement"). Ces opérateurs peuvent être utilisés pour qualifier des assertions sur le système. Par exemple, il se peut que des propriétés de sécurité soient tenues de contenir "dorénavant", tandis qu'il sera peut-être demandé à d'autres états du système d'être atteints "finalement" à partir de quelque autre état d'initialisation. Les formules temporelles sont interprétées sur des séquences d'états (comportements). Ce qui constitue un "état" dépend du niveau choisi de description. Cela peut faire référence à l'ensemble du système, à un composant du système ou à un programme informatique. Les intervalles temporels quantifiés et les contraintes ne sont pas explicitement gérés dans la logique temporelle. La synchronisation absolue doit être gérée par la création d'états temporels supplémentaires comme partie intégrante de la définition de l'état.

[3)] Ada est un langage de programmation informatique de haut niveau, structuré, statiquement typé, impératif, large de spectre et orienté objet, qui est une extension du Pascal et d'autres langages.

D.28.7 VDM - Méthode de Développement de Vienne

Objectif

Spécification et mise en œuvre des programmes séquentiels.

Description

La méthode VDM (*Vienna Development Method*) est une technique de spécification à base mathématique et une technique de raffinement des réalisations d'une manière autorisant la preuve de leur correction par rapport à la spécification.

La technique de spécification est basée sur un modèle en ce sens que l'état du système est modélisé en termes de structures de la théorie des ensembles sur lesquelles sont définis des invariants (prédicats), et les opérations sur cet état sont modélisées en spécifiant leurs préconditions et postconditions en termes d'état du système. Il peut être montré que les opérations préservent les invariants du système.

La réalisation de la spécification est réalisée par la réification de l'état du système en termes de structures de données dans le langage cible et par le raffinement des opérations en terme d'un programme dans le langage cible. Les étapes de réification et de raffinement donnent naissance à des obligations de preuve qui établissent leur validité. Que ces obligations soient ou non menées à bien est un choix réservé au concepteur.

La méthode VDM est principalement utilisée au stade de la spécification mais peut être employée dans les stades de conception et de réalisation conduisant au code source. Elle peut aussi être appliquée aux programmes séquentiels ou aux processus séquentiels dans les systèmes concurrents.

D.28.8 Méthode Z

Objectif

Z est une notation de langage de spécification destinée aux systèmes séquentiels et une technique de conception permettant au concepteur d'évoluer d'une spécification Z vers des algorithmes exécutables d'une manière permettant d'établir la preuve de leur validité par rapport à la spécification.

La notation Z est surtout utilisée au stade de la spécification, mais une méthode a été mise au point pour passer de la spécification à la conception et à la réalisation. Cette notation est parfaitement adaptée au développement des systèmes séquentiels orientés donnée.

Description

A l'instar de la méthode VDM, la technique de spécification est basée sur un modèle en ce sens que l'état du système est modélisé en termes de structures de la théorie des ensembles sur lesquelles sont définis des invariants (prédicats), et les opérations sur cet état sont modélisées en spécifiant leurs préconditions et postconditions en termes d'état du système. Il peut être montré que les opérations préservent les invariants du système, démontrant par là leur cohérence. La partie formelle de la spécification est divisée en schémas permettant la structuration des spécifications par le biais du raffinement.

En principe, une spécification Z est un mélange de Z formel et de texte explicatif informel en langage naturel. (Le texte formel par lui-même est parfois trop laconique pour permettre une lecture facile et il est souvent nécessaire d'expliquer ses objectifs, en revanche le langage naturel informel peut aisément devenir vague et imprécis).

A la différence de la méthode VDM, Z est une notation plutôt qu'une méthode complète. Toutefois, il a été développé une méthode associée (appelée B) qui peut être utilisée conjointement à Z.

D.28.9 Méthode B

Objectif

Tout comme la méthode VDM, la méthode B a pour but de modéliser formellement un système ou un logiciel et de prouver que le comportement de ce système ou logiciel respecte les propriétés qui ont été rendues explicites au cours de la modélisation.

Description

La modélisation B fait appel à des éléments mathématiques de la théorie des ensembles. D'une part, les invariants (c'est-à-dire prédicats) définissent les propriétés statiques du modèle. D'autre part, les opérations établissent des postconditions, définissant ainsi son comportement dynamique. La spécification d'un système ou logiciel complexe est rendue possible par décomposition du modèle en "machines" liées entre elles par des liens de sémantique différente.

On peut distinguer deux catégories principales de modélisation avec le formalisme B.

- La première (historiquement la première) vise à développer les logiciels : Dans ce cas, le but est de produire un programme qui respecte sa spécification. Le modèle est constitué de machines abstraites (pas nécessairement déterministes) et d'affinements par étapes de ces machines, ce qui conduit à des mises en œuvre déterministes écrites dans un pseudo-code appelé “B0”. Ce pseudo-code peut alors être traduit automatiquement en un langage de programmation cible.
- La seconde catégorie vise à modéliser les systèmes et en l'occurrence on parle de “ B événementiel” ("Event B") : le but est de spécifier, sans ambiguïté et en toute cohérence, un système qui respectent des propriétés explicites. Le modèle tient compte du système lui-même et de son environnement. La dynamique du système est modélisée par "événements"' et la technique d'affinement est utilisé pour préciser les interactions entre le système et son environnement.

Un ensemble d'Obligations de preuve (assertions logiques qui sont formellement prouvées à partir d'hypothèses qui avaient été extraites du modèle formel B) est généré automatiquement. Ces Obligations de preuve garantissent

- l'existence de données qui respectent les propriétés statiques et dynamiques du modèle,
- que les opérations (comportement dynamique du modèle) respectent l'invariant,
- que l'affinement des données et des opérations (et du pseudo-code B0 si cela s'avère nécessaire) ne contredit pas la spécification écrite dans les machines abstraites,
- que chaque opération est appelée dans le contexte de sa précondition,
- dans le cas de la modélisation de logiciels, que le programme se termine (en particulier, que chaque boucle se termine).

D'autres Obligations de preuve (par exemple : vérification du dépassement ou du soupassement d'entier) sont aussi générées.

D.28.10 Vérification du modèle

Objectif

Étant donné un modèle d'un système, tester automatiquement si ce modèle satisfait ou non à une spécification donnée.

Description

La vérification du modèle est le processus consistant à vérifier si une structure donnée est un modèle d'une formule logique donnée. Le concept est général et s'applique à toutes sortes de logiques et de structures appropriées. Un simple problème de vérification de modèle est de contrôler si, oui ou non, une formule donnée en logique des propositions est satisfaite par une structure.

Une classe importante de méthodes de vérification de modèle a été développée pour vérifier par algorithme les systèmes formels. Cela est réalisé en vérifiant si la structure, souvent dérivée d'une conception logicielle ou matérielle, satisfait ou non à la spécification formelle, typiquement une formule en logique temporelle.

La vérification de modèle est le plus souvent appliquée à des conceptions de matériels. Pour les logiciels, en raison de l'indécidabilité (voir la théorie de la Calculabilité), l'approche ne peut pas être complètement algorithmique; typiquement, il peut échouer à prouver ou infirmer une propriété donnée.

La structure est en général donnée sous dorme d'une description de code source dans un langage de description de matériel industriel ou un langage à usage spécial. Un tel programme correspond a un automate à états finis, c'est-à-dire un graphe orienté constitué de nœuds (ou sommets) et d'arcs. Un jeu de propositions atomiques est associé à chaque nœud, indiquant typiquement quels éléments de mémoires sont un. Les nœuds représentent les états d'un système, les arcs représentent les transitions possibles susceptibles de modifier l'état, alors que les propositions atomiques représentent les propriétés de base qui sont valides à un instant de l'exécution.

Formellement, le problème peut être posé comme suit : étant donné une propriété souhaitée, exprimée comme une formule en logique temporelle p, et une structure M avec un état initial s, décider si. Si M est finie, comme c'est le cas dans les matériels, la vérification de modèle se réduit ou non à une recherche de graphe.

D.29 Preuve formelle

Objectif

En utilisant des modèles et des règles théoriques et mathématiques, il est possible de prouver la validité d'un programme ou modèle sans l'exécuter.

Description

Un certain nombre d'assertions sont déclarées à divers endroits du programme et elles sont utilisées comme préconditions et postconditions vers les différents chemins du programme. La preuve consiste à montrer que le programme transfère les préconditions dans les postconditions selon un ensemble de règles logiques, et que le programme prend fin.

Plusieurs méthodes formelles sont décrites dans la présente bibliographie, par exemple : CCS, CSP, HOL, LOTOS, OBJ, Logique temporelle, VDM et Z. Leurs descriptions figurent à l'Article D.28 "Méthodes formelles".

D.30 Rattrapage par progression

Objectif

Permettre une exploitation fonctionnelle correcte en présence d'un ou de plusieurs défauts.

Description

Si un défaut a été détecté, l'état actuel du système est manipulé pour obtenir un état qui deviendra cohérent un peu plus tard. Ce concept est spécialement adapté aux systèmes en temps réel, dotés d'une petite base de données et d'une grande vitesse de changement de l'état interne. Il est supposé que, au minimum, une partie de l'état du système s'impose finalement sur l'environnement et que seule une partie des états du système est influencée (contrainte) par l'environnement.

D.31 Dégradation contrôlée

Objectif

Maintenir la disponibilité des fonctions cruciales du système malgré les défaillances en éliminant les fonctions moins cruciales.

Description

Cette technique attribue des priorités aux diverses fonctions que le système doit exécuter. La conception assure, s'il arrivait que les ressources soient insuffisantes pour exécuter toutes les fonctions du système, que les fonctions de plus haute priorité seraient alors exécutées de préférence aux fonctions moins prioritaires. Par exemple, il est admis que des fonctions de consignation des erreurs et des événements soient moins prioritaires que les fonctions de contrôle du système, auquel cas le contrôle du système pourrait continuer si le matériel associé à la consignation d'erreurs devait échouer.

Un autre exemple serait un système de signalisation dans lequel en cas de perte de communication avec le centre de commande le matériel local côté ligne établit automatiquement les itinéraires disponibles pour la direction prise par le trafic de la plus haute priorité. Cela serait une dégradation contrôlée car les trains sur les itinéraires prioritaires seraient capables de passer par la zone affectée par la perte de communication avec le centre de commande mais d'autres mouvements, tels que les mouvements de manœuvre, ne seraient pas possibles.

D.32 Analyse d'impact

Objectif

Identifier l'effet provoqué par une modification ou une amélioration apportée à un logiciel sur les autres composants du logiciel en question ainsi que sur les autres systèmes.

Description

Avant d'apporter toute modification ou amélioration au logiciel, une analyse doit être effectuée en vue d'identifier l'impact de la modification ou de l'amélioration sur le logiciel ainsi que d'identifier les systèmes et composants logiciels affectés.

Une fois l'analyse terminée, une décision doit être prise concernant la nouvelle vérification du système logiciel. Cette vérification dépend du nombre de composants affectés, de la criticité de ces composants et de la nature du changement. Les décisions possibles sont les suivantes :

a) seul le composant changé doit être revérifié;

b) tous les composants affectés identifiés sont revérifiés; et

c) le système complet est revérifié.

D.33 Masquage d'informations/Encapsulation

Objectif

Augmenter la robustesse et la maintenabilité du logiciel.

Description

Les données globalement accessibles par tous les composants logiciels peuvent être modifiées accidentellement ou incorrectement par l'un ou l'autre de ces composants. Il n'est pas exclu que tout

changement apporté à ces structures de données nécessite un examen détaillé du code et des modifications importantes.

Le masquage d'informations constitue une approche générale permettant de minimiser ces difficultés. Les structures de données clés sont "masquées" et ne peuvent être manipulées que par le biais d'un ensemble défini de procédures d'accès. Ceci permet de modifier des structures internes et d'ajouter de nouvelles procédures sans affecter le comportement fonctionnel du reste du logiciel. Par exemple, un répertoire nommé peut avoir des procédures d'accès Insertion, Suppression et Recherche. Les procédures d'accès et les structures de données internes peuvent être réécrites (par exemple, pour utiliser une différente méthode de consultation ou pour enregistrer les noms sur un disque dur) sans altérer le comportement logique du reste du logiciel utilisant ces procédures.

Ce concept de type de données abstrait est directement pris en charge dans un certain nombre de langages de programmation, mais le principe de base peut être appliqué quel que soit le langage utilisé.

D.34 Tests d'interface

Objectif

Démontrer que les interfaces des sous-programmes ne contiennent pas d'erreurs ou pas d'erreurs amenant à des défaillances dans une application particulière du logiciel ou détecter toutes les erreurs éventuellement significatives.

Description

Plusieurs niveaux de détail ou d'état complet des tests sont réalisables. Les niveaux les plus importants sont les tests de

- toutes les variables d'interface dans leurs positions extrêmes,
- toute variable d'interface individuellement sur ses valeurs extrêmes avec les autres variables d'interface sur leurs valeurs normales,
- toutes les valeurs du domaine de chaque variable d'interface avec les autres variables d'interface sur leurs valeurs normales,
- toutes les valeurs de toutes les variables combinées (il n'est pas exclu que ceci ne soit réalisable que pour les petites interfaces),
- les conditions de test spécifiées applicables à chaque appel de chaque sous-routine.

Ces tests sont particulièrement importants si leurs interfaces ne contiennent pas d'assertions détectant les valeurs incorrectes des paramètres. Ils sont également essentiels après la génération de nouvelles configurations de sous-programmes préexistants.

D.35 Sous-ensemble de langage

Objectif

Réduire la probabilité d'introduction de défauts de programmation et augmenter la probabilité de détection des défauts restants.

Description

Le langage est examiné pour identifier les structures de programmation qui sont sujettes à erreurs ou difficiles à analyser, en utilisant par exemple des méthodes d'analyse statique. Un sous-ensemble de langage excluant ces structures est alors défini.

D.36 Mémorisation des cas exécutés

Objectif

Forcer le logiciel dans un état sûr s'il exécute un chemin non autorisé.

Description

Pendant le processus d'autorisation, tous les détails importants de chaque exécution du programme sont enregistrés. Pendant le fonctionnement normal, chaque exécution du programme est comparée à l'ensemble des exécutions autorisées. Si elle diffère, une action de sécurité est décidée.

L'enregistrement de l'exécution peut être la séquence de chemins de décision-à-décision individuels (DDpaths, decision-to-decision paths) ou la séquence d'accès individuels aux tableaux, aux enregistrements ou aux volumes, ou les deux.

Différentes méthodes de stockage des chemins d'exécution sont possibles. Les méthodes de codage de hachage peuvent être utilisées pour mettre en correspondance la séquence d'exécution avec un seul grand nombre ou une séquence de nombres. Pendant le fonctionnement normal, il faut vérifier la valeur du chemin d'exécution par rapport aux modèles enregistrés avant toute opération de sortie.

Puisqu'il existe un très grand nombre de combinaisons possibles des chemins de décision-à-décision au cours d'un seul programme, il n'est parfois pas réalisable de traiter les programmes comme un tout. Dans ce cas, la technique peut être appliquée au niveau du composant.

D.37 Métriques

Objectif

Prédire les attributs des programmes à partir des propriétés du logiciel lui-même plutôt que de le faire à partir de son développement ou de son historique de test.

Description

Ces modèles évaluent certaines propriétés structurelles du logiciel et les associent à un attribut recherché tel que la complexité. Des outils logiciels sont requis pour évaluer la plupart des mesures. Certaines des métriques pouvant être appliquées sont énumérées ci-dessous :

- complexité théorique du graphe : cette mesure pouvant être appliquée au début du cycle de vie pour évaluer les compromis repose sur la complexité du graphe de contrôle du programme, représenté par un nombre cyclomatique;
- nombre de chemins permettant d'activer un certain composant (accessibilité) : plus le composant est accessible, plus il est facile à mettre au point;
- mesures de la complexité de Halstead : cette mesure calcule la longueur du programme en comptant le nombre d'opérateurs et d'opérandes. Elle fournit une mesure de la complexité et estime les ressources de développement;
- nombre d'entrées et de sorties par composant : la minimisation du nombre de points d'entrée/sortie est une caractéristique clé de la conception structurée et des techniques de programmation.

D.38 Approche modulaire

Objectif

Décomposition d'un système logiciel en petites parties compréhensibles afin de limiter la complexité du logiciel.

Description

Une approche modulaire ou modularisation contient plusieurs règles applicables aux phases de conception, de codage et de maintenance d'un projet logiciel. Ces règles varient selon la méthode de conception employée au cours de la conception. La plupart des méthodes intègre les règles suivantes :

- un module/composant doit n'avoir qu'une seule tâche ou fonction bien définie à remplir;
- les connexions entre les modules/composants doivent être limitées et strictement définies, la cohérence à l'intérieur d'un module/composant doit être forte;
- les collections de sous-programmes doivent être construites de manière à fournir plusieurs niveaux de modules/composants;
- les sous-programmes ne doivent comporter qu'une seule entrée et une seule sortie;
- les modules/composants doivent communiquer avec d'autres modules/composants via leurs interfaces. Lorsqu'on utilise des variables globales ou communes, elles doivent être bien structurées, l'accès doit être contrôlé et leur utilisation justifiée dans chaque instance;
- toutes les interfaces des modules/composants doivent être pleinement documentées;
- toutes les interfaces des modules/composants doivent contenir le nombre minimum de paramètres nécessaires pour la fonction des modules/composants; et
- une restriction adaptée du nombre des paramètres doit être spécifiée; en règle générale ils sont au nombre de 5.

D.39 Modélisation des performances

Objectif

Assurer que la capacité de fonctionnement du système est suffisante pour satisfaire aux exigences spécifiées.

Description

La spécification des exigences inclut les exigences relatives à la capacité de traitement et de réponse de fonctions spécifiques, combinées parfois à des contraintes applicables à l'utilisation des ressources totales du système. La conception suggérée du système est comparée aux besoins requis en

- définissant un modèle des processus du système et de leurs interactions,
- identifiant l'utilisation des ressources par chaque processus (par exemple : le temps processeur, la largeur de bande des communications, les périphériques de stockage, etc.),
- identifiant la distribution des demandes exigées du système dans des conditions moyennes et dans les conditions les plus mauvaises,
- calculant la capacité de traitement moyenne et la plus défavorable et les temps de réponse pour chaque fonction du système.

Pour les systèmes simples, il n’est pas exclu qu’une solution analytique soit possible. En revanche, pour les systèmes plus complexes, une certaine forme de simulation est indispensable pour obtenir des résultats précis.

Avant la modélisation détaillée, il est possible d'utiliser un contrôle du "budget des ressources" plus simple qui additionne les exigences applicables aux ressources de tous les processus. Si les exigences excèdent la capacité prévue du système, la conception n'est pas réalisable. Même si la conception réussit ce contrôle, il n'est pas exclu que la modélisation des performances montre que des retards et des temps de réponse

excessifs se produisent en raison du manque de ressources. Pour éviter cette situation, les ingénieurs imaginent souvent des systèmes utilisant une certaine fraction (50 % par exemple) des ressources totales de manière à limiter la probabilité manque de ressources.

D.40 Exigences en matière de performance

Objectif

Établir que les exigences du logiciel en matière de performance ont été observées.

Description

Une analyse est effectuée sur les spécifications des exigences du système et du logiciel afin d'identifier toutes les exigences générales et spécifiques, explicites et implicites en matière de performance.

A tour de rôle, toutes les exigences de performance sont examinées pour déterminer

- les critères de réussite auxquels il faut satisfaire,
- si une mesure de la satisfaction aux critères de réussite peut être définie,
- la précision potentielle de telles mesures,
- les étapes du projet au cours desquelles les mesures peuvent être estimées,
- les étapes du projet au cours desquelles les mesures peuvent être effectuées.

La faisabilité de chaque exigence de performance est alors analysée de manière à obtenir une liste des exigences de performance, des critères de réussite et des mesures potentielles. Les principaux objectifs sont les suivants :

a) chaque exigence de performance est associée à une mesure au moins;

b) chaque fois que possible, on sélectionne des mesures précises et efficaces qui peuvent être utilisées aussitôt que possible dans le processus de développement;

c) les exigences essentielles et optionnelles des performances et les critères de réussite sont identifiés;

d) chaque fois possible, on doit tirer parti de la possibilité d'utiliser une seule mesure pour plusieurs exigences de performance.

D.41 Tests probabilistes

Objectif

Obtenir un chiffre quantitatif concernant les propriétés de fiabilité du logiciel à l'étude. Il est admis que ce chiffre concerne les niveaux de confiance et de signification associées à

a) une probabilité de défaillance par demande,

b) une probabilité de défaillance pendant une certaine période de temps,

c) une probabilité de limitation d'erreur.

A partir de ces chiffres, il est permis de déduire d'autres paramètres, parmi lesquels

- la probabilité d'une exécution sans défaillance,
- la probabilité de survie,
- la disponibilité,
- le MTBF ou le taux de défaillance,
- la probabilité d'une exécution sûre.

Description

Les considérations probabilistes reposent sur un test probabiliste ou sur une expérience d'exploitation. En général, il existe un grand nombre de scénarios de test pour les cas d'exploitation observés.

Pour faciliter les tests, des aides automatiques sont en principe utilisés. Ils concernent les détails de la fourniture de données de test et la surveillance des résultats des tests. Des tests poussés sont exécutés sur les grands ordinateurs hôtes avec le contexte approprié de simulation des processus. Les données de test sont sélectionnées à la fois selon des points de vue systématiques et aléatoires. Les premiers concernent le contrôle global du test, par exemple, en garantissant un profil des données de test. La sélection aléatoire prend chaque scénario de test individuellement en détail.

Les conditions de test, les exécutions des tests et les surveillances des tests sont déterminées individuellement selon les objectifs détaillés des tests décrits plus haut. Les autres conditions importantes sont définies par les conditions mathématiques préalables à remplir pour autoriser l'évaluation du test dans l'optique de l'objectif voulu du test.

Il est admis que des chiffres probabilistes sur le comportement de tout objet de test soient également dérivés de l'expérience d'exploitation. Sous réserve de remplir les mêmes conditions, on peut appliquer les mêmes mathématiques que pour l'évaluation des résultats des tests.

D.42 Simulation de processus

Objectif

Tester la fonction d'un logiciel ainsi que de son interface avec le monde extérieur, sans lui permettre de modifier le monde réel d'aucune façon.

Description

La création, aux seules fins de tests, d'un système qui simule le comportement du système à contrôler par le système testé.

Il est admis que la simulation soit de type logiciel seulement ou une combinaison de logiciel et de matériel. Elle doit

- fournir toutes les entrées du système testé qui existeront quand le système sera installé,
- répondre aux sorties du système d'une façon représentant fidèlement l'équipement contrôlé,
- prévoir des entrées opérateur afin de fournir toutes les perturbations que le système testé est tenu de gérer.

Quand le logiciel est testé, il est admis que la simulation soit une simulation du matériel cible avec ses entrées et sorties.

D.43 Prototypage/Animation

Objectif

Vérifier la faisabilité de la réalisation du système par rapport aux contraintes définies. Communiquer au client l'interprétation du système par l'auteur des spécifications afin de repérer les méprises.

Description

Un sous-ensemble de fonctions, de contraintes et d'exigences de performance du système est sélectionné. Un prototype est élaboré à l'aide d'outils de haut niveau. À ce stade, des contraintes telles que la machine cible, le langage d'implémentation, la taille du programme, la maintenabilité et la robustesse peuvent ne pas

être prises en compte. Le prototype est évalué par rapport aux critères du client et il est permis de modifier les exigences du système à la lumière de cette évaluation.

D.44 Bloc de rattrapage

Objectif

Améliorer la vraisemblance du programme exécutant la fonction pour laquelle il est conçu.

Description

Plusieurs sections de programme différentes sont écrites, souvent indépendamment, dans le but que chacune d'elle exécute la même fonction souhaitée. Le programme final est construit à partir de ces sections. La première section, appelée section principale, est exécutée en premier. Cette exécution est suivie d'un test d'acceptation du résultat calculé. Si le test est réussi, le résultat est alors accepté et transmis aux parties suivantes du système. Si le test échoue, tous les effets secondaires de la première section sont réinitialisés et la seconde section appelée première alternative est exécutée. Cette exécution, également suivie d'un test d'acceptation, est traitée comme dans le premier cas. Une deuxième, une troisième et même des alternatives supplémentaires peuvent être proposées le cas échéant.

D.45 Temps de réponse et contraintes de place mémoire

Objectif

Assurer que le système satisfera à ses exigences temporelles et de mémoire.

Description

La spécification des exigences destinées au système et au logiciel intègre des exigences relatives à la mémoire et à la réponse de fonctions spécifiques, parfois combinées à des contraintes sur l'utilisation des ressources totales du système. Une analyse est effectuée pour identifier la répartition des demandes dans des conditions moyennes et dans les conditions les plus défavorables. Cette analyse exige des estimations de l'usage des ressources et du temps requis pour chaque fonction du système. Ces estimations peuvent être obtenues de diverses manières, par exemple, par comparaison avec un système existant ou par prototypage et évaluation des performances des systèmes à temps critique.

D.46 Rattrapage par ré-exécution

Objectif

Tenter un rétablissement fonctionnel à partir d'une condition de défaut détectée, à l'aide de mécanismes de tentative de ré-exécution.

Description

Au cas où une condition de défaut ou d'erreur est détectée, des tentatives sont effectuées pour rétablir la situation en ré-exécutant le même code. Le rattrapage par réexécution peut être aussi complet qu'une procédure de réinitialisation et de redémarrage ou n'être qu'une petite tâche de re-planification et de relance, après expiration d'un délai ou action d'un chien de garde concernant le logiciel. Les techniques de réexécution sont communément utilisées pour le rattrapage des défauts ou des erreurs de communication et les conditions de réexécution peuvent être indiquées par une erreur du protocole de communication (somme de contrôle, etc.) ou par une temporisation de la réponse d'acquittement de la communication.

D.47 Sécurité Contrôlée

Objectif

Protéger le logiciel contre les défauts résiduels de spécification et de réalisation qui nuisent à la sécurité.

Description

La Sécurité Contrôlée repose sur un moniteur externe, mis en œuvre sur un ordinateur indépendant d'après une spécification différente. Ce contrôle a pour seul objectif de s'assurer que l'ordinateur principal exécute des actions en sécurité, mais pas nécessairement des actions correctes. Le contrôle supervise en permanence l'ordinateur principal. Il empêche le système d'entrer dans un état anti-sécuritaire. En outre, si le contrôle détecte que l'ordinateur principal entre dans un état potentiellement dangereux, le système doit être ramené à un état sécuritaire soit par le contrôle, soit par l'ordinateur principal lui-même.

D.48 Gestion de la configuration du logiciel

Objectif

La gestion de la configuration du logiciel vise à assurer la cohérence des livraisons des groupes de logiciel développés à mesure que ces livraisons évoluent. La gestion de la configuration s'applique, en général, tant au développement matériel que logiciel.

Description

La gestion de la configuration du logiciel est une technique utilisée tout au long du développement. Pour l'essentiel, elle exige l'enregistrement de la production de toute version de chaque livraison "importante" et de toute relation entre les différentes versions des différentes livraisons. Les enregistrements résultants permettent au concepteur de déterminer l'effet, sur les autres livraisons, d'un changement apporté à une livraison. En particulier, les systèmes ou sous-systèmes peuvent être reconstruits de façon fiable à partir d'ensembles cohérents de versions de composants.

D.49 Langages de programmation à fort typage

Objectif

Réduire la probabilité de défauts en utilisant un langage autorisant un haut niveau de contrôle par le compilateur.

Description

De tels langages, en général, autorisent que des types de données définis par l'utilisateur soient définis à partir des types de données du langage de base ("INTEGER" (entier), "REAL" (réel), par exemple). Ces types peuvent alors être utilisés exactement de la même façon que les types de base, mais des contrôles stricts sont imposés pour assurer que le type correct est utilisé. Ces contrôles sont imposés sur l'ensemble du programme, même si celui-ci est construit à partir d'unités compilées distinctes. Les contrôles assurent aussi que le nombre et le type des arguments des procédures correspondent même lorsqu'ils sont référencés à partir de composants compilés distincts.

Les langages de programmation à fort typage supportent aussi d'autres aspects de la bonne pratique du génie logiciel tels que des structures de contrôle facilement analysables (par exemple : IF … THEN … ELSE, DO … WHILE, etc.) qui conduisent à des programmes bien structurés.

Des exemples types de langage à fort typage sont Pascal, Ada et Modula-2.

D.50 Tests structurels

Objectif

Effectuer des tests permettant de tester certains sous-ensembles de la structure du programme.

Description

Basé sur une analyse du programme, un ensemble de données d'entrée est choisi de manière à ce qu'une grande fraction des éléments du programme sélectionnés soit testée. Les éléments du programme testé peuvent varier en fonction du niveau de rigueur requis.

- Instructions : il s'agit du test le moins rigoureux puisqu'il est possible d'exécuter toutes les instructions du code sans tester les deux branches d'une instruction conditionnelle.
- Branchements : il convient de contrôler les deux côtés de chaque branche. Il n'est pas exclu que cette vérification soit peu pratique pour certains types de codes défensifs.
- Conditions composées : chaque condition d'un branchement conditionnel composé (c'est-à-dire reliée par AND/OR) est testée.
- LCSAJ (*Linear Code Sequence And Jump*, section de code linéaire terminée par un saut) : une LCSAJ est toute section linéaire d'instructions de code comprenant des sauts conditionnels terminée par un saut. Beaucoup de sous-chemins potentiels seront irréalisables en raison des contraintes sur les données d'entrée imposées par l'exécution du code précédent.
- Flot de données : les chemins d'exécution sont sélectionnés sur la base de l'usage des données, par exemple un chemin dans lequel la même variable lue et écrite à la fois.
- Graphe d'appel : un programme est composé de sous-routines qui sont éventuellement appelées à partir d'autres sous-routines. Le graphe d'appel est l'arborescence des appels des sous-routines du programme. Les tests sont destinés à couvrir tous les appels dans l'arborescence.
- Chemin entier : exécute tous les chemins possibles du code. Des tests complets sont normalement irréalisables en raison du très grand nombre de chemins potentiels.

D.51 Schémas de structure

Objectif

Représenter la structure d'un programme sous forme de schéma.

Description

Les schémas de structure sont une notation venant compléter les organigrammes de données. Ils décrivent le système de programmation et la hiérarchie des parties qu'ils représentent sous forme d'un schéma arborescent. Ils documentent la manière dont les éléments des organigrammes de données peuvent être réalisés sous forme d'une hiérarchie d'unités de programmation.

Un schéma de structure décrit les relations entre les unités du programme sans inclure une quelconque information sur l'ordre d'activation de ces unités. Il est dessiné à l'aide des trois symboles suivants :

a) un rectangle annoté avec le nom de l'unité;

b) une flèche reliant ces rectangles;

c) une flèche entourée d'un cercle, annotée du nom des données entrant et sortant des éléments du schéma de structure. En principe, la flèche entourée d'un cercle est dessinée parallèlement à la flèche reliant les rectangles du tableau.

A partir de n'importe quel organigramme de données non ordinaire, il est possible de déduire un nombre de schémas de structures différentes.

Les schémas de structure dérivés des organigrammes de données représentent une structure de premier niveau du système dans laquelle chaque boîte du schéma correspond à une bulle de l'organigramme de données. Naturellement, des niveaux plus profonds peuvent être décrits en utilisant la même technique.

D.52 Méthodologie structurée

Objectif

Le principal objectif des méthodologies structurées est de promouvoir la qualité du développement logiciel en portant l'attention sur les premières parties du cycle de vie. Les méthodes visent à accomplir cela à l'aide de procédures intuitives et précises et de notations (assistées par ordinateurs) permettant d'identifier l'existence d'exigences et de caractéristiques de réalisation dans un ordre logique et d'une manière structurée.

Description

Il existe toute une gamme de méthodes structurées. Certaines telles que les méthodes SSADM et LBMS sont conçues pour le traitement de données traditionnel et les fonctions de traitement transactionnel, tandis que d'autres (MASCOT, JSD, Yourdon en temps réel) sont plus orientées vers les applications de contrôle de processus et temps réel (qui sont plus facilement critique de sécurité).

Les méthodes structurées sont essentiellement des "supports de réflexion" permettant de percevoir et de fractionner systématiquement un problème ou un système. Leurs principales caractéristiques sont les suivantes :

- ordre logique de réflexion décomposant un vaste problème en étapes faciles à manier,
- identification du système total, y compris l'environnement ainsi que le système prescrit,
- décomposition des données et fonctions dans le système prescrit,
- listes de contrôle, autrement dit liste des types d'éléments exigeant une définition,
- faible investissement intellectuel - Simples, intuitives, pragmatiques.

Les notations correspondantes ont tendance à être précises dans l'identification d'un problème et des entités du système (par exemple, les processus et les flots de données), mais les fonctions de traitement exécutées par ces entités ont tendance à être exprimées par des notations informelles. Certaines méthodes utilisent toutefois en partie des notations (mathématiquement) formelles (JSD, par exemple, emploie des expressions régulières : Yourdon, SOM et SDL utilisent des automates à état finis). Cette précision non seulement réduit la possibilité de méprise, mais elle offre la possibilité du traitement automatique.

La visibilité est un autre avantage de la notation structurée, car elle permet à l'utilisateur de contrôler intuitivement la spécification ou la conception d'après sa connaissance élevée mais non formalisée.

D.53 Programmation structurée

Objectif

Concevoir et réaliser le composant logiciel de manière qu'il soit pratique à analyser. Il convient que cette analyse permette de découvrir tous les comportements significatifs du composant.

Description

Il convient que le composant logiciel soit caractérisé par une complexité structurelle minimale et il convient qu'il évite les branchements complexes. Il convient (lorsque c'est possible) d'associer simplement les contraintes et conditions de boucles aux paramètres d'entrée. Il convient de diviser le composant logiciel en modules de taille appropriée et que l'interaction de ces modules soit explicite. Il convient que les caractéristiques du langage de programmation qui favorisent l'approche ci-dessus soient utilisées de préférence à d'autres caractéristiques supposées être plus efficaces, excepté lorsque l'efficacité constitue une priorité absolue (par exemple, certains systèmes critiques de sécurité).

D.54 Langages de programmation adaptés

Objectif

Soutenir, autant que possible, les exigences de la présente Norme européenne, notamment la programmation défensive, le fort typage, la programmation structurée et les assertions. Il convient que le langage de programmation choisi conduise à un code facilement vérifiable avec un minimum d'effort et facilite le développement, la vérification et la maintenance du programme.

Description

Il convient que le langage soit défini intégralement et sans ambiguïté. Il convient qu'il soit orienté utilisateur ou orienté problème plutôt qu'orienté machine. Les langages largement utilisés et leurs sous-ensembles sont préférés aux langages spécialisés.

Outre les caractéristiques déjà référencées, il convient que le langage propose

- une structure de bloc,
- une vérification du temps de compilation,
- une vérification à l'exécution des types et des limites des tableaux,
- une vérification des paramètres.

Il convient que le langage favorise

- l'utilisation de petits composants faciles à gérer,
- la limitation de l'accès aux données dans les composants définis,
- la définition des sous-domaines des variables, et
- tout autre type de structure limitant les erreurs.

Il est souhaitable que le langage soit supporté par un compilateur adapté, des bibliothèques appropriées de composants préexistants, un outil de mise au point et des outils adaptés au contrôle de version et au développement.

Les caractéristiques qui rendent la vérification difficile et qu'il convient donc d'éviter sont les suivantes :

- les branchements inconditionnels à l'exception des appels de sous-routines;
- la récursivité;
- pointeurs, collections ou tout autre type d'objets ou de variables dynamiques;
- la gestion des interruptions au niveau du code source;
- les entrées ou sorties multiples des boucles, blocs ou sous-programmes;
- l'initialisation ou la déclaration implicite de variables;
- les enregistrements variables et l'équivalence; et
- les paramètres procéduraux.

Les langages de bas niveau, notamment les langages assembleurs, posent des problèmes en raison de leur nature orientée machine.

D.55 Réseaux de Pétri temporels

Objectif

Modéliser les aspects pertinents du comportement du système et évaluer et améliorer, le cas échéant, les exigences en matière de sécurité et de fonctionnement par le biais de l'analyse et de la re-conception.

Description

Les réseaux de Pétri appartiennent à une catégorie de modèles théoriques de graphes, adaptés à la représentation de l'information et du flux de contrôle dans les systèmes présentant des caractéristiques de parallélisme et de comportement asynchrone.

Un réseau de Pétri est un réseau de places et de transitions. Les places sont "marquées" ou "non marquées". Une transition est "activée" quand toutes ses places d'entrée sont marquées. Quand elle est activée, elle est autorisée (mais pas obligée) à "tirer". Dans ce cas, les marques d'entrée sont retirées et chaque place de sortie de la transition est marquée.

Les dangers potentiels sont représentés sous forme d'états particuliers (marquages) dans le modèle. Les réseaux de Pétri étendus autorisent la modélisation des caractéristiques temporelles du système. S'il est vrai que les réseaux de Pétri "classiques" se concentrent sur les aspects du flux de contrôle, plusieurs extensions ont été proposées en vue d'incorporer le flot de données dans le modèle.

D.56 Révisions structurées/ Revues de la conception

Objectif

Détecter les erreurs présentes dans un produit du processus de développement aussi tôt et aussi économiquement que possible.

Description

Le CE 56 de la CEI a publié un Guide sur les examens formels de la conception (*Guide on Formal Design Reviews*) qui comprend une description générale des examens formels de la conception, leurs objectifs, le détail des divers types d'examens de la conception, la composition d'une équipe pratiquant ce type d'examen et les devoirs et responsabilités qui lui incombent. Le document CEI fournit également des lignes directrices relatives à la planification et à la conduite des examens de la conception, ainsi que des détails spécifiques concernant le rôle des spécialistes indépendants au sein d'une équipe d'examen.

La CEI recommande "un examen formel de la conception doit être effectué pour l'ensemble des nouveaux produits/processus, des nouvelles applications et des révisions apportées aux produits existants et aux processus de fabrication ayant une incidence sur la fonction, la performance, la sécurité, la fiabilité et la faculté de vérifier la maintenabilité, la disponibilité, la possibilité à établir le coût et d'autres caractéristiques affectant le produit/processus final, les utilisateurs ou les tiers".

Une revue du code consiste en une équipe de revue sélectionnant un petit ensemble de scénarios de test sur papier, représentatifs d'entrées et des sorties prévues correspondantes pour le programme. Les données de test sont alors manuellement analysées au travers de la logique du programme.

D.57 Programmation orientée objet

Objectif

Permettre un prototypage rapide, faciliter la réutilisation des composants logiciels, effectuer le masquage d'informations, réduire la vraisemblance d'erreurs pendant toute la durée du cycle de vie, réduire l'effort nécessaire pendant la phase de maintenance, décomposer des problèmes complexes en petits problèmes

plus faciles à gérer, réduire les dépendances entre les composants, créer des applications plus facilement extensibles.

Description

La programmation orientée objet est essentiellement une nouvelle manière de penser les logiciels, reposant sur les abstractions qui existent en situation réelle plutôt que sur des abstractions de calcul. La programmation orientée objet organise le logiciel comme un ensemble d'objets qui incorporent à la fois la structure des données et le comportement. Cette façon de penser contraste avec la programmation classique dans laquelle la structure des données et le comportement ne sont que vaguement reliés.

Objet : Un objet se compose d'une zone de données privées et d'un ensemble d'opérations - appelées méthodes - sur cet objet. Les méthodes peuvent être publiques ou privées. Aucun autre composant logiciel n'est autorisé à lire ou à changer directement les données privées d'un objet. Tous les autres composants logiciels doivent utiliser les méthodes publiques sur cet objet pour lire ou écrire des données dans la zone des données privées d'un objet.

Classe d'objets : le fait de spécifier une classe d'objets (souvent sous la forme d'une définition de type) active l'instanciation de nombreux objets de la même classe, à savoir, toutes les instanciations ont le domaine de données privées et les méthodes définies dans la classe d'objets.

Héritage (multiple) : une classe d'objets peut hériter du domaine de données privées et des méthodes d'une (ou plusieurs) superclasses (classes d'objets au-dessus d'elle dans la hiérarchie des classes) en étant autorisée à ajouter un certain nombre de données privées, à ajouter un certain nombre de méthodes ou à modifier les mises en œuvre des méthodes héritées. L'héritage permet de construire plusieurs arbres de classes d'objets.

Polymorphisme : la même opération peut se comporter différemment sur des classes d'objets différentes, par exemple, l'opération d'écriture pour un objet terminal écrit des caractères sur le terminal en question et une opération d'écriture sur un objet fichier écrit des caractères sur le fichier en question.

Inconvénient : Les langages de programmation orientée objet peuvent conduire à un besoin de ressources supplémentaires avec un impact négatif sur la performance du système.

D.58 Traçabilité

Objectif

La traçabilité a pour objectif d'assurer qu'il est possible de montrer que toutes les exigences sont respectées et qu'aucun matériau non traçable n'a été introduit.

Description

La traçabilité par rapport aux exigences doit être un facteur important dans la validation d'un système et des moyens doivent être mis à disposition pour en permettre la démonstration durant toutes les phases du cycle de vie.

La traçabilité doit être considérée comme applicable à la fois aux exigences fonctionnelles et non fonctionnelles et doit particulièrement concerner

a) la traçabilité des exigences jusqu'à la conception et autres objets qui y répondent,

b) la traçabilité des objets de la conception jusqu'aux objets de l'implémentation qui les instancient,

c) la traçabilité des exigences et des objets de conception jusqu'aux objets opérationnels et de maintenance qu'il est nécessaire d'appliquer pour que l'utilisation du système soit sûre et adéquate,

d) la traçabilité des exigences, des items de conception, de réalisation, de fonctionnement et de maintenance par rapport aux plans de vérification et de tests et aux spécifications qui détermineront leur acceptabilité,

e) la traçabilité des plans de vérification et de test et des spécifications par rapport aux rapports de tests ou autres qui enregistrent les résultats de leur application.

Lorsque les exigences, la conception et d'autres items sont instanciés dans plusieurs documents séparés, la traçabilité doit être conservée dans les structures de documents et d'une manière hiérarchique.

La sortie du processus de traçabilité doit être soumise à la Gestion de Configuration formelle.

D.59 Métaprogrammation

Objectif

La métaprogrammation permet aux programmeurs d'avoir plus de travail effectué dans la même durée que cela aurait pris pour écrire manuellement l'ensemble du code.

Description

La métaprogrammation est l'écriture de programmes informatiques qui écrivent ou manipulent d'autres programmes (ou eux-mêmes) comme leurs données ou qui font une partie du travail au cours du temps de compilation qui est autrement effectuée au moment de l'exécution.

Le langage dans lequel le métaprogramme est appelé le métalangage. Le langage des programmes qui sont manipulés est appelé le langage objet. La capacité d'un langage de programmation à être son propre métalangage est appelée réflexion ou réflexivité.

La réflexion est une caractéristique précieuse du langage pour faciliter la métaprogrammation. Il est également très utile d'avoir le langage de programmation lui-même comme un type de données de première classe (comme dans Lisp). La programmation générique invoque un moyen de métaprogrammation au sein d'un langage, dans les langages qui la prennent en charge.

La métaprogrammation fonctionne en général de l'une de deux façons. La première façon est d'exposer les parties internes du moteur d'exécution au code de programmation à travers les interfaces de programmation d'application (les API). La seconde approche est l'exécution dynamique d'expressions de chaînes qui contiennent des commandes de programmation. Ainsi, "des programmes peuvent écrire des programmes". Bien que les deux approches puissent être utilisées, la plupart des langages ont tendance à pencher vers l'une ou vers l'autre.

D.60 Programmation procédurale

Objectif

Spécifier les étapes que le programme doit prendre pour atteindre l'état souhaité.

Description

Programmation procédurale basée sur le concept d'appel de procédure. Les procédures, également appelées routines, sous-routines, méthodes ou fonctions (ne sont pas à confondre avec les fonctions mathématiques mais sont similaires à celles utilisées en programmation fonctionnelle) contiennent simplement une série d'étapes de calculs devant être menées à bien. Toute procédure est susceptible d'être appelée à n'importe quel point de l'exécution du programme, y compris par d'autres procédures ou par elle-même.

D.61 Graphes séquentiels de fonction

Objectif

Décrire des algorithmes de programme par des diagrammes.

Description

Les éléments de SFC permettent de partitionner une unité d'algorithmes d'application en un ensemble d'étapes et de transitions interconnectées par des liaisons orientées. Un ensemble d'actions est associé à chaque étape et une condition de transition est associée à chaque transition. Sachant que les éléments de SFC requièrent le stockage d'informations d'état, les seules unités d'algorithmes d'application qui peuvent être structurées en utilisant ces éléments sont les blocs fonctionnels.

Voir EN 61131-3:2003, 2.6.

D.62 Schéma à contact

Objectif

Décrire un programme par des diagrammes.

Description

Voir EN 61131-3:2003, 4.2.

D.63 Diagramme fonctionnel

Objectif

Décrire une fonction entre des variables d'entrée et des variables de sortie par des diagrammes.

Description

Voir EN 61131-3:2003, 4.3.

D.64 Graphe d'états ou Diagramme d'états

Objectif

Décrire le comportement d'un système par des diagrammes.

Description

On utilise un graphe d'états ou des diagrammes d'états pour décrire le comportement d'un système. Le diagramme d'états peut décrire les états possibles d'un objet à mesure que les événements se produisent. Chaque diagramme représente habituellement des objets d'une seule classe et suit les différents états de ses objets dans le système.

Le diagramme d'états peut être utilisé pour représenter graphiquement des automates à états finis. Cela avait été introduit par Taylor Booth dans son livre de 1967 "*Sequential* Machines and Automata Theory". Une autre représentation possible est la table des transitions d'états.

Une forme classique d'un diagramme d'états pour un automate à états finis est un graphe orienté.

D.65 Modélisation de données

Objectif

Création d'un modèle de données.

Description

La modélisation des données en science informatique est le processus de création d'un modèle de données en appliquant des descriptions de modèles de données formelles utilisant des techniques de modélisation de données.

Un modèle de données en génie logiciel est un modèle abstrait qui décrit les modalités de représentation et d'accès aux données. Les modèles de données définissent formellement des objets de donnée et les relations parmi les objets de donnée pour un domaine d'intérêt. Certaines applications types des modèles de base de données comprennent de prendre en charge le développement de bases de données et de permettre l'échange de données pour un domaine d'intérêt particulier. Les modèles de données sont spécifiés dans un langage de modélisation de données.

D.66 Diagramme de flux de commande/Graphe de flux de commande

Objectif

Décrire le comportement d'un système par des diagrammes.

Description

En science informatique, un diagramme de flux de commande ou graphe de flux de commande (CFG, control flow graph) est une représentation, à l'aide d'une notation graphique, de tous les chemins susceptibles d'être traversés dans un programme au cours de son exécution. Chaque nœud du graphe représente un bloc de base, c'est-à-dire, tout morceau de code linéaire, sans saut ou étiquette de saut. Les étiquettes de saut débutent un bloc, et les sauts terminent un bloc. Des arcs orientés sont utilisés pour représenter les sauts dans un flux de commande. Il existe dans la plupart des présentations deux blocs spécialement désignés : le bloc d'entrée, par lequel la commande entre dans le graphe de flux, et le bloc de sortie, par lequel tout le flux de commandes.

Le CFG est essentiel pour un grand nombre d'optimisations de compilation et outils d'analyse statique.

L'accessibilité est une autre propriété des graphes qui est utile en optimisation. Si un bloc/sous-graphe n'est pas relié à partir du sous-graphe contenant le bloc d'entrée, le bloc en question est inaccessible au cours de n'importe quelle exécution; et il est de même du code inaccessible; il peut être retiré en toute sécurité. Si le bloc de sortie est inaccessible à partir du bloc d'entrée, cela indique une boucle infinie. Ici encore, les codes morts et certaines boucles infinies sont possibles, même si le programmeur n'a pas codé explicitement dans ce sens : les optimisations comme la propagation des constantes ("*constant propagation*") et la réduction des constantes ("*constant folding*") suivies de "*jump threading*" pourraient réunir plusieurs blocs élémentaires en un seul, provoquer le retrait d'arcs d'un CFG, etc., et, de ce fait, éventuellement déconnecter des parties du graphe.

terminologie
ces termes sont communément utilisés dans la discussion concernant les graphes de flux de commande

bloc d'entrée
bloc par lequel l'ensemble du flux de commande pénètre dans le graphe

bloc de sortie
bloc par lequel l'ensemble du flux de commande quitte le graphe

arc de retour
arc qui pointe vers un ancêtre dans une traversée en profondeur d'abord (DFS, *depth*-first) du graphe

arc critique
arc qui n'est ni le seul arc partant de son bloc source ni le seul arc entrant dans son bloc destination. Il est nécessaire de diviser ces arcs (il est nécessaire de créer un nouveau bloc au milieu de l'arc) afin d'insérer des calculs sur l'arc

arc anormal
arc dont la destination est inconnue. Ces arcs tendent à empêcher l'optimisation. Les éléments de traitement des exceptions peuvent en produire

arc impossible
(également appelé arc fictif) arc qui a été ajouté au graphe uniquement dans le but de préserver la caractéristique que le bloc de sortie prédomine sur tous les blocs. Il ne peut pas être traversé

dominant
le bloc M domine le bloc N si chaque chemin partant de l'entrée qui atteint le bloc N doit passer par le bloc M. Le bloc d'entrée domine tous les blocs

post-dominant
le bloc M post-domine le bloc N si chaque chemin partant N vers la sortie doit passer par le bloc M. Le bloc de sortie post-domine tous les blocs

dominant immédiat
le bloc M domine immédiatement le bloc N si M domine N et il n'y a aucun bloc P intervenant tel que M domine P et P domine N. Autrement dit, M est le dernier dominant sur tout chemin allant de l'entrée jusqu'à N. Chaque bloc a un dominant immédiat unique, s'il en a même un

post-dominant immédiat
analogue au dominant immédiat

arbre de dominant
structure de données auxiliaires décrivant les relations de dominant. Il y a un arc allant du bloc M au bloc N si M est un dominant immédiat de N. Ce graphe est un arbre car chaque bloc a un dominant immédiat unique. Cet arbre a sa racine au bloc d'entrée

arbre post-dominant
analogue à l'arbre dominant. Cet arbre a sa racine au bloc de sortie

en-tête de boucle
dominant, parfois appelé point d'entrée de la boucle, qui est la cible d'un arc de retour formant boucle. Il domine tous les blocs dans le corps de la boucle

pré-en-tête de boucle
supposons que le bloc M soit un dominant comportant plusieurs arcs entrants, dont certains sont des arcs de retour (donc M est un en-tête de boucle). Il est avantageux pour plusieurs passes d'optimisation de décomposer M en deux blocs Mpre et Mloop. Le contenu de M et des arcs de retour est déplacé vers Mloop, le reste des arcs est déplacé pour se diriger dans Mpre, et un nouvel arc allant de Mpre à Mloop est inséré (et donc Mpre est le dominant de Mloop). Au départ, Mpre serait vide mais des passes telles que le mouvement de code invariant de boucle peut le garnir. Mpre est appelé le pré-en-tête de boucle et Mloop serait l'en-tête de boucle

D.67 Diagramme de séquence

Objectif

Décrire l'interaction entre les processus ou les composants par des diagrammes.

Description

Un diagramme de séquence est une sorte de diagramme d'interactions qui montre comment les processus ou composants fonctionnent les uns avec les autres et dans quel ordre.

D.68 Méthodes de spécification en tableaux

Objectif

Le but est de fournir un moyen normalisé et bien structuré pour définir les fonctions d'un système guidé par les données.

Description

Les notations tabulées telles que les tableaux de contrôle de signalisation constituent une méthode bien établie de documentation des exigences spécifiques à une installation pour un système de signalisation ferroviaire.

La technique est pertinente lorsque les types de relations entre éléments du système sont normalisés.

Avantage : Le format du tableau et des rubriques possibles dans chaque champ peut servir de liste de contrôle pendant la vérification.

D.69 Langage spécifique à l'application

Objectif

Le but est de fournir un moyen permettant de spécifier la fonctionnalité d'un système guidé par les données utilisant des concepts et la technologie qui sont facilement assimilables par les ingénieurs d'applications qui peuvent ne pas être familiarisés avec les langages de programmation conventionnels.

Description

Un langage spécifique à l'application combine typiquement des constructions de contrôle qui sont semblables à des langages de programmation conventionnels de haut niveau avec les opérateurs qui sont spécifiques au type de système.

La technique est pertinente lorsqu'il est nécessaire de spécifier des décisions booléennes mais elle peut également être applicable ailleurs.

Avantage : Flexibilité, permettant de produire des données pour des circonstances inhabituelles qui peuvent ne pas avoir été prévues au moment de la conception initiale du système.

D.70 UML (Unified Modeling Language, langage de modélisation unifié)

Objectif

Représenter des programmes logiciels et les artéfacts connexes d'une manière permettant la réduction de la complexité au moyen de l'abstraction. En permettant la modélisation d'une conception existante ou planifiée en termes de diversité des types de diagrammes, le langage UML facilite l'évaluation des caractéristiques principales de la conception sur la base de représentations à des niveaux de détail appropriés. Le langage UML est fréquemment utilisé dans le dit développement guidé par modèle, pris en charge par des produits commerciaux. Ce style de développement vise à l'amélioration de la qualité des logiciels et la productivité des développeurs par l'utilisation de langages de modélisation de haut niveau.

Description

Le langage UML est un langage de modélisation polyvalent normalisé, provenant de l'utilisation de langages de spécification orientée graphique et de langages de programmation orientée objet. S'appuyant sur cette tradition, le langage UML réutilise plusieurs des concepts et méthodes de ses prédécesseurs. Les modèles sont écrits en termes d'un ou plusieurs types de diagrammes, classés en diagrammes de structure et diagrammes de comportement, les seconds comprenant également quatre types de diagrammes classés en diagrammes d'interaction.

Schémas de structure

- Diagrammes de paquetages : Montrent le contenu de différents paquetages et la relation entre eux, chacun contenant des éléments de modèle associés.
- Diagrammes de classes : Spécifient les types d'objets avec leurs différentes caractéristiques et leurs relations avec d'autres types d'objets, sur la base d'une adaptation de diagrammes traditionnels entités-relations.
- Diagrammes d'objets : Montrent comment différents objets (instances de classes) sont reliés les uns aux autres.
- Diagrammes de structures composites : Montrent la structure interne d'un classifieur (tel qu'une classe ou un composant) et ses points d'interaction avec d'autres parties du système.
- Diagrammes de composants : Montrent les composants qui constituent le système, leurs interrelations, interactions et interfaces externes.
- Diagrammes de déploiements : Spécifient comment le logiciel est distribué à travers la plateforme d'exécution.

Diagrammes de comportements

- Diagrammes d'activités : Décrivent les comportements algorithmiques, en utilisant une adaptation d'organigrammes traditionnels qui permet de modéliser le transfert de données et l'exécution concomitante.
- Diagrammes de machines à états : Décrivent le comportement événementiel à l'aide d'automates à états finis (diagrammes d'états).
- Diagrammes de scénarios d'utilisation : Modélisent les acteurs en interaction avec le système pour obtenir des scénarios d'utilisation spécifiques.
- Diagrammes d'interactions (diagrammes de communications, diagrammes d'aperçu des interactions, diagrammes de séquences, diagrammes des temps) : Décrivent les scénarios comprenant des activités accomplies par des objets en communication.

Alors que le langage UML est un langage de modélisation générique, des interprétations spécifiques à un domaine sont rendues possibles avec l'aide de profils. Par affinage des concepts du langage UML standard, les profils permettent de réaliser ces interprétations en utilisant les extensions définies dans le profil. Ainsi, le langage UML est utilisé comme base pour la définition de langages spécifiques à un domaine.

D.71 Langages spécifiques à un domaine

Objectif

Représenter des programmes logiciels et les artéfacts connexes dans un langage adapté en fonction du domaine particulier.

Description

Un langage spécifique à un domaine (DSL, *domain-specific language*) est un langage de programmation, de spécification ou de modélisation créé spécifiquement pour résoudre des problèmes dans un domaine d'application particulier ou un domaine de problèmes particulier, ou bien avec une technique particulières. Le langage repose sur des concepts et caractéristique pertinents pour ce domaine. Les langages spécifiques à un domaine sont également appelés "langages spécialisés", par opposition aux langages de programmation ou langages de modélisation d'usage général comme Java et UML.

L'un des avantages importants des langages spécifiques à un domaine est la possibilité de représenter et résoudre des problèmes au sein d'un domaine particulier sans la nécessité de connaître la programmation, la spécification ou la modélisation générale. Il en résulte qu'il est possible de produire des programmes, spécifications ou modèles à un haut niveau, éventuellement par l'utilisateur final. En fournissant des constructions adaptées pour ce domaine, et éventuellement des moyens pour la création automatisée de codes, un DSL augmente en général la productivité du programmeur et la qualité du produit qui en résulte. La création de codes est typiquement mise en œuvre sous la forme d'un générateur d'application utilisant le DSL comme entrée.

Bibliographie

EN 50159	Applications ferroviaires - Systèmes de signalisation, de télécommunication et de traitement - Partie 1: Communication de sécurité sur des systèmes de transmission
EN 61131-3:2003	Automates programmables - Partie 3: Langages de programmation (CEI 61131-3:2003)
EN 61158-2:2010	Réseaux de communication industriels - Spécifications des bus de terrain - Partie 2: Spécification des couches physiques et définition des services (CEI 61158-2:2010)

Applications électriques ferroviaires. Signalisation, communications **UTE/UC 9XA**

Liste des organismes représentés dans la commission de normalisation

Secrétariat : UTE

BNF (BUREAU DE NORMALISATION FERROVIAIRE)

FIF (FEDERATION DES INDUSTRIES FERROVIAIRES)

RATP (REGIE AUTONOME DES TRANSPORTS PARISIENS)

SERCE (SYNDICAT DES ENTREPRISES DE GENIE ELECTRIQUE ET CLIMATIQUE)

SNCF (SOCIETE NATIONALE DES CHEMINS DE FER FRANCAIS)